



Bài tập lớn cuối kì - Quán Game điện tử

Lập trình .NET (Học viện Ngân hàng)



messages.pdf_cover_qr_code_label

HỌC VIỆN NGÂN HÀNG
KHOA CÔNG NGHỆ THÔNG TIN VÀ KINH TẾ SỐ



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN LẬP TRÌNH .NET

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG QUẢN LÝ CỬA HÀNG GAME
ĐIỆN TỬ

Giảng viên hướng dẫn : **ThS.Trần Thị Huế**
Lớp học phần : **242IS25A01**
Nhóm : **7**

Hà Nội - 06/2025

HỌC VIỆN NGÂN HÀNG
KHOA CÔNG NGHỆ THÔNG TIN VÀ KINH TẾ SỐ

BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN LẬP TRÌNH .NET

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG QUẢN LÝ CỬA HÀNG GAME
ĐIỆN TỬ

Giảng viên hướng dẫn : ThS.Trần Thị Huế

Nhóm : 7

Bùi Thị Hoa

Nguyễn Thị Phương Anh

Nguyễn Thị Hồng Ngọc

Nguyễn Thị Hải Yến

Nguyễn Thị Hải Yến

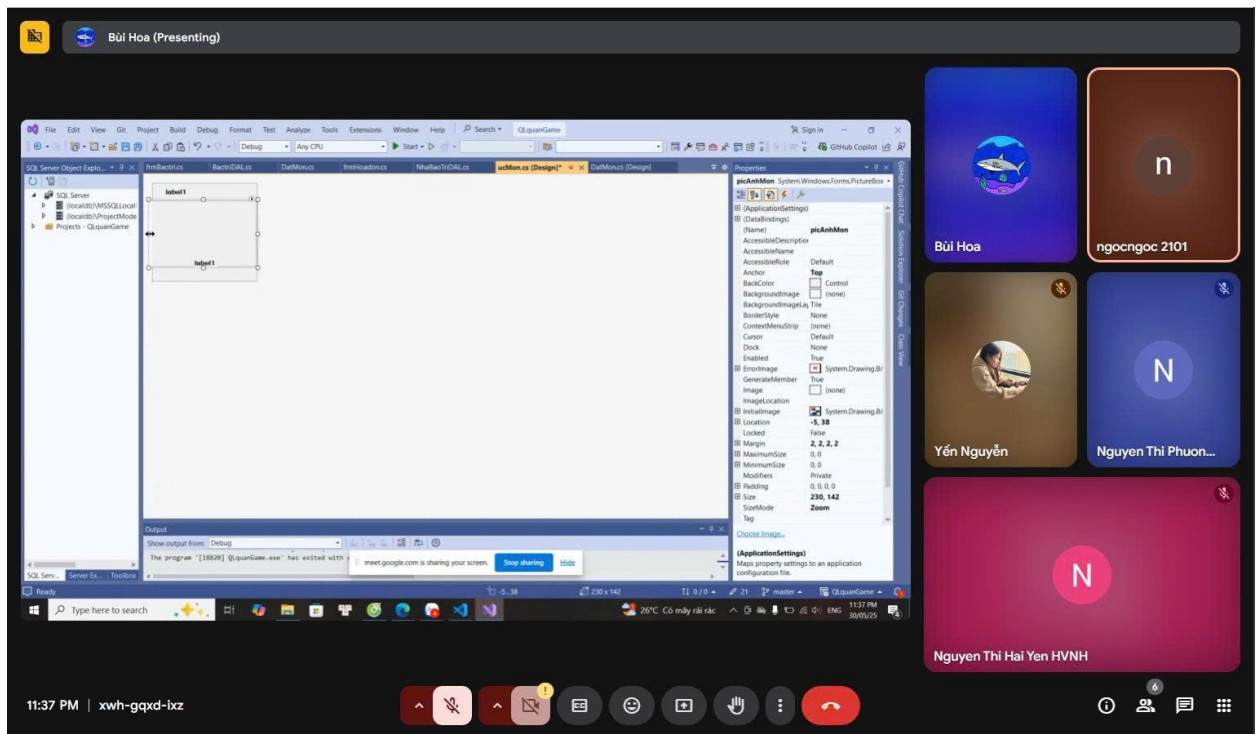
Hà Nội - 06/2025

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	MSV	Họ Và Tên	Phân Công Công Việc	Đánh Giá
1		Bùi Thị Hoa (NT)		20%
2		Nguyễn Thị Phương Anh		20%
3		Nguyễn Thị Hồng Ngọc		20%
4		Nguyễn Thị Hải Yến		20%
5		Nguyễn Thị Hải Yến		20%

MINH CHỨNG CÔNG VIỆC

- **GitHub**
- **Google Meet**



ngocngoc 2101 (Presenting)

docs.google.com/document/d/16ung32PgAyDr-Eec3QyVoupw5YATYZQ06uUd0OISL/edit?tab=T.0

STT	Tên trường	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaĐT	INT	-	Khóa chính, Khóa ngoại	Mã lần báo trị (theo kết bằng Đoàn).
2	MaMý	INT	-	Khóa chính, Khóa ngoại	Mã máy tính được báo trị (theo kết bằng Máy tính).
3	MaTT	INT	-	Khóa ngoại	Mã tính trạng (theo kết bằng Tin học).
4	MaNS	INT	-	Khóa ngoại	Mã nguyên nhân (theo kết bằng Nguyên nhân).
5	MaGP	INT	-	Khóa ngoại	Mã giải pháp áp dụng (theo kết bằng Giải pháp).

Bảng Chi tiết báo trị

Bảng lưu thông tin chi tiết về các lần báo trị máy tính.

2.3 Các Procedure sử dụng trong CSDL (tiếp)

- PROCEDURE sp_GetBinhLaiBaoTri

11:44 PM | xwh-gqxd-lxz

ngocngoc 2101

Bùi Hoa

Yến Nguyễn

Nguyễn Thị Phuong...

ngocngoc 2101 (outside Học viện Ngân Hàng) started presenting

- Google Drive

Drive

Tìm trong Drive

Drive của tôi > .NET

Loại Người Lần sửa đổi gần đây nhất

Tiến độ.xlsx

Đề cương BTL.docx

yêu cầu 2.docx

Để

Đã sử dụng 7,17 GB trong tổng số 15 GB

Mua thêm bộ nhớ

9:17 AM

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành đến ThS Trần Thị Huế – giảng viên môn Lập trình.NET đã hướng dẫn hỗ trợ nhóm trong quá trình làm bài tập lớn này. Cô đã chia sẻ những kiến thức thức chuyên sâu, bổ ích và luôn giải đáp thắc mắc, giúp đỡ chúng em nhiệt tình từ những bài học đầu tiên đến khi chúng em hoàn thành bài tập lớn này.

Trong quá trình làm bài, chúng em được mở rộng và học tập thêm những điều bổ ích và những kinh nghiệm cho bài luận sau này. Bài báo cáo tất nhiên sẽ không tránh được những hạn chế, thiếu sót. Nhóm 7 rất mong nhận được ý kiến đóng góp và nhận xét chân thành từ cô

Hà Nội, ngày 1 tháng 6 năm 2025

Nhóm 7

LỜI CAM ĐOAN

Chúng em xin cam đoan bài tập lớn “Xây dựng hệ thống quản lý cửa hàng Game điện tử” là quá trình nghiên cứu của nhóm 7 và được hướng dẫn bởi giảng viên Trần Thị Huế. Nội dung nghiên cứu là minh bạch và nhóm xin được chịu trách nhiệm trước bất kỳ sai sót nào.

Hà Nội, ngày 1 tháng 6 năm 2025

Nhóm 7

MỤC LỤC

MINH CHỨNG CÔNG VIỆC.....	ii
LỜI CẢM ƠN.....	iv
LỜI CAM ĐOAN.....	v
MỤC LỤC.....	vi
DANH MỤC HÌNH ẢNH.....	ix
DANH MỤC BẢNG.....	xii
CHƯƠNG 1: MỞ ĐẦU.....	1
1.1. PHÁT BIỂU BÀI TOÁN.....	1
1.2. LÝ DO CHỌN ĐỀ TÀI.....	2
1.3. TÍNH CẤP THIẾT CỦA ĐỀ TÀI.....	3
CHƯƠNG 2. MÔ TẢ NGHIỆP VỤ HỆ THỐNG.....	5
2.1. MÔ TẢ BÀI TOÁN.....	5
2.2. MÔ TẢ NGHIỆP VỤ HỆ THỐNG.....	7
2.2.1. Input của bài toán.....	7
2.2.2. Output của bài toán.....	8
2.3. CÁC CHỨC NĂNG CỦA HỆ THỐNG.....	9
CHƯƠNG 3: THIẾT KẾ CƠ SỞ DỮ LIỆU.....	10
3.1. THIẾT KẾ MỨC KHÁI NIỆM.....	10

3.2. THIẾT KẾ MỨC VẬT LÝ.....	10
3.3 CÁC PROCEDURE SỬ DỤNG TRONG CSDL.....	27
CHƯƠNG 4: XÂY DỰNG CHƯƠNG TRÌNH.....	44
4.1. GIỚI THIỆU VỀ KIẾN TRÚC 3 LỚP.....	44
4.1.1 Lớp DAL (Data Access Layer).....	44
4.1.2. Lớp DTO (Data Transfer Object).....	50
4.1.3. Lớp GUI (Graphical User Interface).....	52
4.2. XÂY DỰNG GIAO DIỆN FORM ĐẶT MÁY (COMPUTER).....	56
4.2.1. Yêu cầu và kịch bản xử lý của từng chức năng trên form Quản lý máy.....	56
4.2.2. Demo chức năng trên giao diện.....	57
4.2.3. Code chức năng trên tầng giao diện.....	60
4.2.4. Code trên tầng DAL.....	68
4.2.5. Code trên tầng DTO.....	73
4.3. XÂY DỰNG GIAO DIỆN FORM ĐẶT MÓN.....	74
4.3.1. Yêu cầu và kịch bản xử lý của từng chức năng form đặt món.....	74
4.3.2. Demo chức năng trên giao diện.....	75
4.3.3. Code trên GUI.....	80
4.3.4. Code trên DAL.....	96
4.3.5. Code trên tầng DTO.....	99
4.4. XÂY DỰNG GIAO DIỆN FORM HÓA ĐƠN (BILLING).....	100

4.4.1. Yêu cầu và kịch bản xử lý của từng chức năng trên form Hóa đơn.....	100
4.4.2. Demo chức năng trên giao diện.....	101
4.4.3. Code trên tầng GUI.....	105
4.3.4. Code trên tầng DAL.....	114
4.3.5. Code trên tầng DTO.....	116
4.4. XÂY DỰNG GIAO DIỆN FORM BẢO TRÌ.....	117
4.4.1. Yêu cầu và kịch bản xử lý của từng chức năng trên form Bảo trì.....	117
4.4.2. Demo chức năng trên giao diện.....	117
4.4.3 Code trên tầng GUI.....	124
4.4.4. Code trên DAL.....	133
4.4.5 Code trên DTO.....	135
KẾT LUẬN.....	137
TÀI LIỆU THAM KHẢO.....	138

DANH MỤC HÌNH ẢNH

Hình 3.1. Sơ đồ quan hệ của các bảng trong cơ sở dữ liệu.....	10
Hình 4.1. Lớp DAL	45
Hình 4.2. Lớp DTO.....	51
Hình 4.3. Lớp GUI.....	52
Hình 4.4. Giao diện Đăng nhập.....	53
Hình 4.5. Giao diện Trang chủ.....	53
Hình 4.6. Giao diện Đặt máy.....	54
Hình 4.7. Giao diện Đặt món.....	54
Hình 4.8. Giao diện Hóa đơn.....	55
Hình 4.9. Giao diện Bảo trì.....	55
Hình 4.10. Giao diện Đăng xuất.....	56
Hình 4.11. Hình. Giao diện khi ấn vào chức năng Đặt máy.....	57
Hình 4.12. Giao diện khi thực hiện Bật máy cho thuê.....	58
Hình 4.13. Giao diện thông báo bật máy thành công.....	59
Hình 4.14. Giao diện thông báo Tắt máy thành công.....	60
Hình 4.15. Giao diện màn hình đặt món.....	75
Hình 4.16. Giao diện tìm kiếm theo danh mục món ăn.....	76

Hình 4.17. Giao diện tìm kiếm theo thanh tìm kiếm.....	76
Hình 4.18. Giao diện thêm danh mục đồ ăn.....	77
Hình 4.19. Giao diện thêm món ăn.....	77
Hình 4.20. Giao diện sửa món ăn.....	78
Hình 4.21. Giao diện khi chưa chọn máy.....	79
Hình 4.22. Giao diện thêm món ăn vào danh sách các món ăn.....	79
Hình 4.23. Thêm món ăn thành công.....	80
Hình 4.24. UcMon.....	81
Hình 4.25. Giao diện hóa đơn.....	101
Hình 4.26. Giao diện tìm kiếm hóa đơn theo tên nhân viên.....	101
Hình 4.27. Giao diện lọc hóa đơn theo loại hóa đơn Chi.....	102
Hình 4.28. Giao diện lọc hóa đơn theo loại hóa đơn Thu.....	102
Hình 4.29. Giao diện thống kê hóa đơn theo khoảng.....	103
Hình 4.30. Giao diện thống kê hóa đơn theo tháng.....	103
Hình 4.31. Giao diện in hóa đơn.....	104
Hình 4.32. Giao diện hóa đơn.....	104
Hình 4.33. Giao diện Bảo trì.....	118
Hình 4.34. Giao diện chọn máy để bảo trì.....	119
Hình 4.35. Thông báo máy đang sử dụng không thể báo lỗi.....	120
Hình 4.36. Giao diện thông báo máy đang bảo trì.....	120

Hình 4.37. Giao diện chọn nhà bảo trì.....	121
Hình 4.38. Giao diện báo lỗi thành công.....	122
Hình 4.39. Giao diện hiển thị thông tin bảo trì.....	122
Hình 4.40. Giao diện thanh toán thành công.....	123
Hình 4.41. Máy trở lại trạng thái Trống (Trong) sau khi đã thanh toán thành công.....	124

DANH MỤC BẢNG

Bảng 3.1. Bảng Phòng.....	10
Bảng 3.2. Bảng Ổ cứng.....	11
Bảng 3.3. Bảng Dung lượng.....	12
Bảng 3.4. Bảng Chip.....	12
Bảng 3.5. Bảng RAM.....	12
Bảng 3.6. Bảng Tốc độ.....	13
Bảng 3.7. Bảng Ổ đĩa.....	13
Bảng 3.8. Bảng Màn hình.....	14
Bảng 3.9. Bảng Chuột.....	14
Bảng 3.10. Bảng Bàn phím.....	15
Bảng 3.11. Bảng Tai nghe.....	15
Bảng 3.12. Bảng Máy tính.....	16
Bảng 3.13. Bảng Nhân viên.....	18
Bảng 3.14. Bảng Tài khoản.....	19
Bảng 3.15. Bảng Loại món.....	20
Bảng 3.16. Bảng Món.....	20
Bảng 3.17. Bảng Hóa đơn.....	21
Bảng 3.18. Bảng Gọi món.....	22

Bảng 3.19. Bảng Gọi món.....	22
Bảng 3.20. Bảng Tình trạng.....	23
Bảng 3.21. Bảng Nguyên nhân.....	24
Bảng 3.22. Bảng Giải pháp.....	24
Bảng 3.23. Bảng Nhà bảo trì.....	25
Bảng 3.24. Bảng Bảo trì.....	25
Bảng 3.25. Bảng Chi tiết bảo trì.....	26

CHƯƠNG 1: MỞ ĐẦU

1.1. PHÁT BIỂU BÀI TOÁN

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, các quán game (cyber café) không chỉ cung cấp dịch vụ truy cập Internet mà còn tích hợp nhiều tiện ích đi kèm như phục vụ đồ ăn, thức uống và các hoạt động giải trí khác. Điều này đòi hỏi một hệ thống quản lý toàn diện để điều phối các hoạt động kinh doanh hàng ngày, bao gồm quản lý máy tính, theo dõi thời gian sử dụng của khách hàng, quản lý dịch vụ ăn uống, nhân viên, và các thông tin liên quan.

Hiện nay, nhiều quán game vẫn sử dụng phương pháp quản lý thủ công hoặc các phần mềm chưa tối ưu, dẫn đến nhiều hạn chế. Việc theo dõi trạng thái máy tính (đang hoạt động, trống, hoặc cần bảo trì) thường không chính xác, dễ gây gián đoạn dịch vụ và làm mất lòng khách hàng. Ghi chép thủ công các phiên sử dụng máy, tính toán chi phí dịch vụ, và lập hóa đơn tốn nhiều thời gian, dễ xảy ra sai sót, ảnh hưởng đến doanh thu và uy tín của quán. Ngoài ra, việc quản lý nhân viên, phân công ca làm việc, và tính toán lương thưởng mà không có hệ thống hỗ trợ cũng gây ra nhiều khó khăn cho chủ quán.

Để giải quyết các vấn đề trên, bài toán đặt ra là xây dựng một hệ thống quản lý quán game sử dụng ngôn ngữ C# trên nền tảng .NET, nhằm tự động hóa các quy trình vận hành. Hệ thống cần đáp ứng các chức năng chính sau:

- **Quản lý máy tính:** Theo dõi trạng thái máy, thời gian sử dụng, và lịch bảo trì.
- **Quản lý dịch vụ ăn uống:** Ghi nhận đơn hàng, theo dõi tồn kho, và tính toán chi phí dịch vụ.
- **Tính tiền tự động:** Tạo hóa đơn dựa trên thời gian sử dụng và dịch vụ, hỗ trợ in hóa đơn.

- **Quản lý nhân viên:** Phân công ca trực, theo dõi hiệu suất làm việc, và tính toán lương thưởng.
- **Báo cáo và thống kê:** Cung cấp báo cáo doanh thu theo ngày, tuần, tháng để hỗ trợ ra quyết định kinh doanh.

Hệ thống cần có giao diện thân thiện, dễ sử dụng, đảm bảo tính bảo mật và độ chính xác của dữ liệu, từ đó nâng cao hiệu quả quản lý và trải nghiệm khách hàng.

1.2. LÝ DO CHỌN ĐỀ TÀI

Đề tài " Xây dựng Hệ thống Quản lý cửa hàng Game điện tử sử dụng C# và .NET Framework" được lựa chọn dựa trên các lý do sau:

- **Tính thực tiễn cao:** Quán game là mô hình kinh doanh phổ biến tại Việt Nam, đặc biệt ở các đô thị lớn, nơi tích hợp nhiều dịch vụ như Internet, ăn uống, và giải trí. Một hệ thống quản lý tự động sẽ giúp giảm thiểu sai sót, tiết kiệm thời gian, và nâng cao hiệu quả vận hành, đáp ứng nhu cầu thực tế của các chủ quán.
- **Ứng dụng kiến thức môn học:** Đề tài này là cơ hội để áp dụng các kiến thức đã học trong môn Lập trình .NET, bao gồm lập trình C#, thiết kế giao diện người dùng, kết nối cơ sở dữ liệu, và xử lý logic nghiệp vụ. Việc thực hiện dự án giúp củng cố kỹ năng lập trình và làm quen với quy trình phát triển phần mềm thực tế.
- **Phù hợp với xu hướng công nghệ:** Sử dụng .NET và C# để phát triển hệ thống quản lý phản ánh xu hướng ứng dụng công nghệ hiện đại trong quản lý kinh doanh. Điều này giúp sinh viên tiếp cận các công cụ và môi trường phát triển chuyên nghiệp, chuẩn bị cho công việc sau này.
- **Đáp ứng nhu cầu cạnh tranh:** Trong bối cảnh ngành dịch vụ giải trí ngày càng cạnh tranh, các quán game cần nâng cao chất lượng dịch vụ để thu hút

và giữ chân khách hàng. Một hệ thống quản lý thông minh sẽ giúp cải thiện trải nghiệm khách hàng, từ việc đặt dịch vụ nhanh chóng đến thanh toán minh bạch.

- **Tính mở rộng và sáng tạo:** Hệ thống này có thể được phát triển thêm với các tính năng như quản lý từ xa, tích hợp thanh toán trực tuyến, hoặc hỗ trợ đa nền tảng, tạo cơ hội để nghiên cứu và ứng dụng các công nghệ mới.

Việc lựa chọn đề tài này không chỉ xuất phát từ nhu cầu thực tiễn mà còn nhằm thúc đẩy ứng dụng công nghệ thông tin vào quản lý kinh doanh, nâng cao hiệu quả và chất lượng dịch vụ, từ đó tạo ra giá trị gia tăng cho các quán game. (TS. Bùi Thị Hồng Nhung - Khoa CNTT&KTS - Học viện Ngân Hàng, 2024)

1.3. TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Trong bối cảnh hiện nay, các quán game phải quản lý đồng thời nhiều hoạt động phức tạp, từ việc theo dõi trạng thái máy tính, quản lý thời gian sử dụng, cung cấp dịch vụ ăn uống, đến quản lý nhân viên và báo cáo doanh thu. Những lý do sau đây nhấn mạnh tính cấp thiết của đề tài:

- **Hạn chế của quản lý thủ công:** Các phương pháp quản lý thủ công thường tốn nhiều thời gian, công sức, và dễ dẫn đến sai sót trong việc ghi chép thời gian sử dụng, tính toán chi phí, hoặc quản lý tồn kho. Những lỗi này không chỉ ảnh hưởng đến doanh thu mà còn làm giảm trải nghiệm của khách hàng, gây mất uy tín cho quán.
- **Nhu cầu nâng cao chất lượng dịch vụ:** Trong thị trường cạnh tranh khốc liệt, khách hàng ngày nay yêu cầu dịch vụ nhanh chóng, chuyên nghiệp, và minh bạch. Một hệ thống quản lý tự động sẽ giúp quán game đáp ứng tốt hơn các kỳ vọng này, từ việc xử lý đơn hàng nhanh chóng đến cung cấp hóa đơn chính xác, từ đó tăng sự hài lòng và lòng trung thành của khách hàng.

- **Xu hướng số hóa và hiện đại hóa:** Trong thời đại công nghiệp 4.0, việc ứng dụng công nghệ thông tin vào quản lý kinh doanh không chỉ là xu hướng mà còn là yêu cầu bắt buộc để tối ưu hóa vận hành và nâng cao hiệu quả. Hệ thống quản lý thông minh không chỉ tự động hóa các quy trình mà còn cung cấp các báo cáo phân tích dữ liệu, hỗ trợ chủ quán đưa ra quyết định kinh doanh kịp thời và chính xác.
- **Tăng cường lợi thế cạnh tranh:** Một hệ thống quản lý hiệu quả giúp các quán game nổi bật trong thị trường, cải thiện trải nghiệm khách hàng và tối ưu hóa chi phí vận hành. Điều này đặc biệt quan trọng trong bối cảnh các quán game phải cạnh tranh không chỉ với nhau mà còn với các hình thức giải trí trực tuyến khác.
- **Góp phần vào đào tạo công nghệ:** Việc phát triển hệ thống quản lý quán game không chỉ đáp ứng nhu cầu thực tế mà còn giúp sinh viên ngành công nghệ thông tin rèn luyện kỹ năng lập trình, thiết kế phần mềm, và ứng dụng công nghệ vào các vấn đề thực tiễn, từ đó chuẩn bị tốt hơn cho sự nghiệp trong tương lai.

Với những lý do trên, đề tài "Xây dựng hệ thống quản lý quán game" mang tính cấp thiết cao, không chỉ đáp ứng nhu cầu thực tiễn của các quán game mà còn phù hợp với xu hướng phát triển công nghệ và yêu cầu đào tạo trong lĩnh vực công nghệ thông tin.

CHƯƠNG 2. MÔ TẢ NGHIỆP VỤ HỆ THỐNG

2.1. MÔ TẢ BÀI TOÁN

Hệ thống quản lý cửa hàng quán game được nhóm xây dựng nhằm giải quyết bài toán tổ chức và điều hành hiệu quả các hoạt động kinh doanh thường ngày. Cụ thể, hệ thống cần đảm nhiệm các chức năng chính sau:

- **Quản lý máy trạm:** Hệ thống sẽ lưu trữ đầy đủ thông tin về từng máy, bao gồm mã máy, vị trí lắp đặt, tên gọi, trạng thái hoạt động và các mã phần cứng như CPU, RAM, ổ cứng, màn hình, chuột, card đồ họa và bàn phím. Việc này giúp việc phân bổ máy cho khách hàng và công tác bảo trì trở nên thuận tiện và có hệ thống.
- **Theo dõi bảo trì:** Mọi hoạt động bảo trì sẽ được ghi lại với các thông tin như mã bảo trì, tình trạng hiện tại, mã máy liên quan, hóa đơn đi kèm, chi phí sửa chữa, bộ phận được sửa chữa và mô tả chi tiết. Việc này giúp cửa hàng kiểm soát tốt tình trạng thiết bị và đảm bảo chất lượng hoạt động.
- **Quản lý phiên sử dụng và hóa đơn:** Đối với từng phiên sử dụng máy của khách, hệ thống sẽ ghi nhận mã phiên, mã máy, thời gian bắt đầu và kết thúc, thời lượng sử dụng, hóa đơn liên quan và tổng chi phí. Hệ thống hóa đơn sẽ lưu các thông tin như mã hóa đơn, loại hóa đơn, thời điểm lập, người lập và tổng số tiền, giúp kiểm soát tài chính minh bạch.
- **Quản lý thực phẩm và đồ uống:** Các thông tin về món ăn, thức uống như mã món, tên món, giá nhập – bán, số lượng tồn kho và phân loại món sẽ được hệ thống lưu lại đầy đủ. Điều này hỗ trợ việc kiểm soát nguyên vật liệu và phục vụ khách hàng nhanh chóng, chính xác.
- **Quản lý nhân sự:** Hệ thống sẽ ghi nhận thông tin của từng nhân viên gồm mã nhân viên, họ tên, quê quán, số điện thoại, email và thông tin tài khoản

của nhân viên. Điều này giúp theo dõi lịch làm việc của nhân viên một cách hiệu quả và hỗ trợ quản lý nhân sự khoa học hơn.

Database mà bài toán quản lý bao gồm các bảng sau:

- **Quản lý máy tính và phần cứng liên quan:**

Bảng **Maytinh**, **Dungluong**, **Ocung**, **Odia**, **Chip**, **Ram**, **Tocdo**, **Tainghe**, **Chuot**, **Manhinh** và **Banphim**: Các bảng này lưu trữ thông tin chi tiết cấu hình phần cứng của từng máy trạm trong cửa hàng. Mỗi máy tính có mã máy, tên máy, trạng thái và được liên kết đến các thành phần phần cứng thông qua mã tương ứng như: CPU (Chip), RAM, ổ cứng (Ocung), dung lượng, tốc độ RAM, màn hình, chuột, bàn phím, tai nghe,... giúp hệ thống quản lý cấu hình từng máy và hỗ trợ công tác bảo trì, phân bổ hiệu quả.

- **Quản lý bảo trì:**

Bảng **Baotri**, **ChitietBT**: Lưu thông tin liên quan đến bảo trì máy trạm, bao gồm mã bảo trì, ngày bảo trì, đơn vị thực hiện (NhaBT), hóa đơn liên quan, tình trạng lỗi, nguyên nhân và giải pháp thực hiện. Hệ thống cho phép xác định nhanh tình trạng hư hỏng và chi phí đi kèm cho từng máy.

Bảng **Tinhtrang**, **Nguyennhan**, **Giaiphap**, **NhaBT**: Cung cấp danh sách chuẩn hóa các lỗi thường gặp (tình trạng), nguyên nhân hỏng hóc, phương án sửa chữa tương ứng và thông tin các nhà bảo trì.

- **Quản lý phiên sử dụng và hóa đơn:**

Bảng **Thuemay**: Ghi nhận phiên sử dụng máy của khách hàng, gồm thời gian bắt đầu - kết thúc, mã máy, và mã hóa đơn tương ứng.

Bảng **Hoadon**: Lưu chi tiết loại hóa đơn (Thu/Chi), người lập, ngày lập và tổng tiền, phục vụ mục đích kế toán và tổng hợp chi phí.

- **Quản lý đồ ăn, thức uống:**

Bảng **Mon, LoaiMon, Goimon**: Lưu thông tin về các món ăn và thức uống được phục vụ như: mã món, tên, giá nhập – giá bán, tồn kho, ảnh minh họa và loại món. **Goimon** ghi nhận các món khách hàng gọi theo từng hóa đơn, phục vụ công tác bán hàng và tính toán doanh thu.

- Quản lý nhân viên và tài khoản:

Bảng **Nhanvien, Taikhoan**: Ghi thông tin về nhân viên như mã nhân viên, họ tên, giới tính, năm sinh, địa chỉ, số điện thoại, email và tài khoản đăng nhập tương ứng. **Taikhoan** giúp phân quyền, theo dõi hoạt động và xác thực truy cập vào hệ thống.

- Thông tin phòng máy:

Bảng **Phong**: Lưu thông tin các phòng trong cửa hàng (Phòng thường, Phòng VIP) bao gồm mã phòng, tên phòng, số lượng máy và giá thuê theo giờ. Giúp hệ thống phân loại và quản lý máy theo từng khu vực cụ thể.

2.2. MÔ TẢ NGHIỆP VỤ HỆ THỐNG

2.2.1. Input của bài toán

Thông tin máy tính: Được lưu trữ trong các bảng **Maytinh, Phong, Ocung, Dunluong, Chip, Ram, Tocdo, Odia, Manhinh, Chuot, Banphim** và **Tainghe**. Bao gồm các thông tin về từng máy trạm như: mã máy, tên máy, mã phòng, trạng thái hoạt động, cùng với các mã phần cứng như RAM, ổ cứng, tốc độ RAM, màn hình, chuột, bàn phím, tai nghe,... Các thông tin này hỗ trợ việc quản lý cấu hình, phân bổ máy và bảo trì hệ thống.

Thông tin bảo trì: Được quản lý trong các bảng **Baotri, ChitietBT, Tinhtrang, Nguyennhan, Giaiphap**, và **NhaBT**. Bao gồm mã bảo trì, ngày bảo trì, đơn vị thực hiện, ghi chú, tình trạng lỗi, nguyên nhân sự cố, giải pháp khắc phục, chi phí sửa chữa và mã

máy được bảo trì. Hệ thống này cho phép theo dõi lịch sử bảo trì và quản lý chất lượng máy móc.

Thông tin nhân viên: Lưu trữ trong các bảng **Nhanvien** và **Taikhoan**. Gồm các thông tin như mã nhân viên, họ tên, giới tính, năm sinh, địa chỉ, số điện thoại, email, cùng thông tin tài khoản đăng nhập hệ thống như tên đăng nhập, mật khẩu, ngày tạo tài khoản. Đây là cơ sở để quản lý nhân sự và phân quyền hệ thống.

Thông tin đồ ăn – thức uống: Được lưu trong các bảng **Mon**, **LoaiMon**, và **Goimon**. Bao gồm các dữ liệu như mã món, tên món, giá nhập, giá bán, số lượng tồn kho, loại món ăn (đồ ăn, đồ uống, snack), ảnh minh họa và số lượng được gọi trong từng hóa đơn. Phục vụ cho việc bán hàng và thống kê doanh thu từ dịch vụ ăn uống.

Thông tin phiên sử dụng máy: Lưu trữ tại bảng **Thuemay**, phản ánh chi tiết hoạt động thuê máy của khách hàng: mã phiên thuê, mã máy sử dụng, thời gian bắt đầu và kết thúc, mã hóa đơn liên quan. Từ đó có thể tính toán thời gian sử dụng và lập hóa đơn tương ứng.

Thông tin hóa đơn: Ghi nhận tại bảng **Hoadon**, chứa các thông tin như mã hóa đơn, loại hóa đơn (thu hoặc chi), ngày tạo, nhân viên lập phiếu, tổng số tiền. Hóa đơn có thể liên quan đến thuê máy, gọi món hoặc bảo trì, hỗ trợ công tác kế toán và thống kê chi phí.

2.2.2. Output của bài toán

Danh sách máy trạm và trạng thái hoạt động: Hệ thống có khả năng xuất danh sách tất cả các máy trạm đang hoạt động, bao gồm thông tin cấu hình phần cứng (CPU, RAM, ổ cứng, màn hình, chuột, bàn phím, tai nghe, tốc độ RAM), tình trạng sử dụng (đang thuê/trống), và vị trí đặt máy theo từng phòng.

Báo cáo bảo trì chi tiết: Xuất các báo cáo liên quan đến hoạt động bảo trì máy như: danh sách máy đã được bảo trì, lỗi gặp phải, nguyên nhân, giải pháp thực hiện, chi phí bảo trì và đơn vị bảo trì. Giúp quản lý nắm bắt nhanh tình trạng thiết bị và chi phí vận hành.

Thông tin phiên sử dụng máy: Xuất báo cáo các phiên sử dụng máy theo từng khách hàng, bao gồm thời gian sử dụng, mã máy, tổng thời gian thuê và liên kết hóa đơn thanh toán. Giúp kiểm soát hoạt động thuê máy và tính toán doanh thu.

Báo cáo doanh thu – hóa đơn: Hệ thống cho phép xuất danh sách các hóa đơn phát sinh theo thời gian, bao gồm thông tin người lập phiếu, loại hóa đơn (thu/chi), tổng số tiền, thời điểm phát sinh. Hỗ trợ tổng kết doanh thu và chi phí vận hành của cửa hàng.

Thông kê món ăn – thức uống bán ra: Thông kê số lượng các món ăn, thức uống đã được khách hàng gọi theo từng hóa đơn hoặc theo ngày/tháng, giúp quản lý kho hiệu quả và đưa ra chiến lược kinh doanh phù hợp.

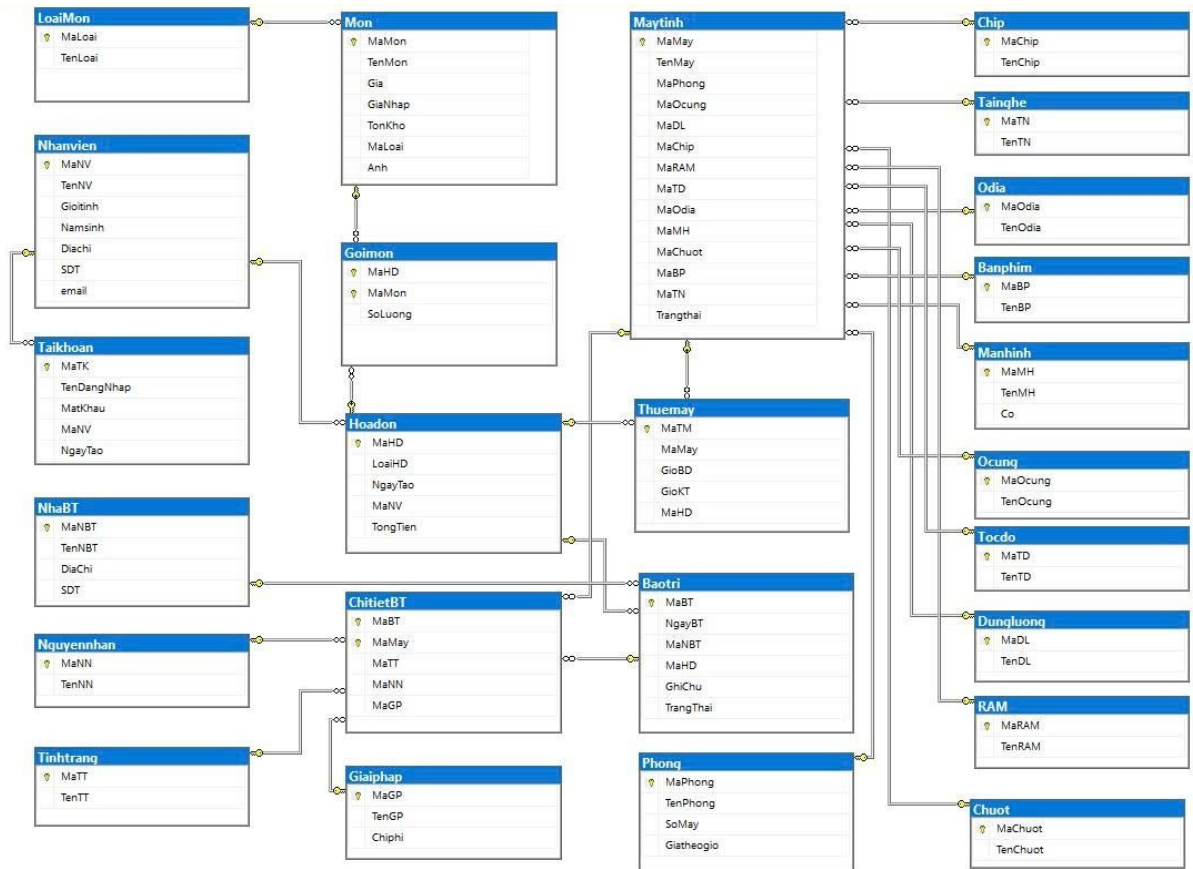
Danh sách nhân viên và hoạt động tài khoản: Xuất danh sách nhân viên hiện tại với đầy đủ thông tin cá nhân và tài khoản đăng nhập, phục vụ công tác quản lý nhân sự và phân quyền hệ thống.

2.3. CÁC CHỨC NĂNG CỦA HỆ THỐNG

- Đăng nhập
- Nhập dữ liệu từ bàn phím.
- Thêm và lưu thông tin vào cơ sở dữ liệu
- Sửa thông tin trong cơ sở dữ liệu
- Xóa thông tin trong cơ sở dữ liệu
- Tìm kiếm
- Xuất/ in hoá đơn
- Đăng xuất

CHƯƠNG 3: THIẾT KẾ CƠ SỞ DỮ LIỆU

3.1. THIẾT KẾ MỨC KHÁI NIỆM



Hình 3.1. Sơ đồ quan hệ của các bảng trong cơ sở dữ liệu

3.2. THIẾT KẾ MỨC VẬT LÝ

- **Bảng Phòng:** Bảng lưu thông tin về các phòng máy trong quán game.

Bảng 3.1. Bảng Phòng

STT	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
-----	---------	--------------	------------	------	------------

1	MaPhong	INT		Khóa chính	Mã định danh duy nhất cho mỗi phòng máy.
2	TenPhong	NVARCHAR	100		Tên phòng (ví dụ: Phòng thường, Phòng VIP).
3	SoMay	INT			Số lượng máy tính trong phòng.
4	Giatheogio	DECIMAL	(10,2)		Giá thuê máy tính theo giờ (đơn vị: VNĐ).

- **Bảng Ổ cứng: Bảng lưu thông tin về ổ cứng của các máy tính.**

Bảng 3.2. Bảng Ổ cứng

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaOcupng	INT		Khóa chính	Mã định danh duy nhất cho mỗi loại ổ cứng.
2	TenOcupng	NVARCHAR	100		Tên ổ cứng (ví dụ: Samsung 970 EVO Plus

					500GB).
--	--	--	--	--	---------

- **Bảng Dung lượng:** Bảng lưu thông tin về dung lượng của ổ cứng.

Bảng 3.3. Bảng Dung lượng

STT	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaDL	INT		Khóa chính	Mã định danh duy nhất cho dung lượng.
2	TenDL	NVARCHAR	100		Tên dung lượng (ví dụ: 500GB, 1TB).

- **Bảng Chip:** Bảng lưu thông tin về CPU của máy tính.

Bảng 3.4. Bảng Chip

STT	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaChip	INT		Khóa chính	Mã định danh duy nhất cho mỗi loại CPU.
2	TenChip	NVARCHAR	100		Tên CPU (ví dụ: Intel Core i5-13400F).

- **Bảng RAM: Bảng lưu thông tin về RAM của máy tính.**

Bảng 3.5. Bảng RAM

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaRAM	INT		Khóa chính	Mã định danh duy nhất cho mỗi loại RAM.
2	TenRAM	NVARCHAR	100		Tên RAM (ví dụ: Kingston Fury Beast 16GB DDR4).

- **Bảng Tốc độ: Bảng lưu thông tin về tốc độ RAM.**

Bảng 3.6. Bảng Tốc độ

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaTD	INT		Khóa chính	Mã định danh duy nhất cho tốc độ RAM.
2	TenTD	NVARCHA R	50		Tên tốc độ RAM (ví dụ: 3200MHz).

- **Bảng Ổ đĩa: Bảng lưu thông tin về ổ đĩa của máy tính.**

Bảng 3.7. Bảng Ổ đĩa

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaOdia	INT		Khóa chính	Mã định danh duy nhất cho mỗi loại ổ đĩa.
2	TenOdia	NVARCHAR	50		Tên ổ đĩa (ví dụ: Samsung 870 EVO 500GB).

- **Bảng Màn hình: Bảng lưu thông tin về màn hình của máy tính.**

Bảng 3.8. Bảng Màn hình

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaMH	INT		Khóa chính	Mã định danh duy nhất cho mỗi loại màn hình.
2	TenMH	NVARCHAR	50		Tên màn hình (ví dụ: LG UltraGear 27GL83A).
3	Co	NVARCHAR	20		Kích thước màn hình (ví dụ: 27 inch).

- **Bảng Chuột: Bảng lưu thông tin về chuột máy tính.**

Bảng 3.9. Bảng Chuột

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaChuot	INT		Khóa chính	Mã định danh duy nhất cho mỗi loại chuột.
2	TenChuot	NVARCHAR	100		Tên chuột (ví dụ: Logitech G203).

- **Bảng Bàn phím: Bảng lưu thông tin về bàn phím.**

Bảng 3.10. Bảng Bàn phím

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaBP	INT		Khóa chính	Mã định danh duy nhất cho mỗi loại bàn phím.
2	TenBP	NVARCHA R	100		Tên bàn phím (ví dụ: Logitech G213 Prodigy).

- **Bảng Tai nghe: Bảng lưu thông tin về tai nghe.**

Bảng 3.11. Bảng Tai nghe

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaTN	INT		Khóa chính	Mã định danh duy nhất cho mỗi loại tai nghe.
2	TenTN	NVARCHAR R	100		Tên tai nghe (ví dụ: HyperX Cloud II).

- **Bảng Máy tính: Bảng lưu thông tin cấu hình và trạng thái của các máy tính trong quán.**

Bảng 3.12. Bảng Máy tính

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaMay	INT		Khóa chính (IDENTITY)	Mã định danh duy nhất cho mỗi máy tính.
2	TenMay	NVARCHAR	100		Tên máy tính (ví dụ: PC-Thuong-01).
3	MaPhong	INT		Khóa ngoại	Mã phòng chứa máy tính (liên kết bảng Phong).

4	MaOcung	INT		Khóa ngoại	Mã ổ cứng (liên kết bảng Ocung).
5	MaDL	INT		Khóa ngoại	Mã dung lượng (liên kết bảng Dungluong).
6	MaChip	INT		Khóa ngoại	Mã CPU (liên kết bảng Chip).
7	MaRAM	INT		Khóa ngoại	Mã RAM (liên kết bảng RAM).
8	MaTD	INT		Khóa ngoại	Mã tốc độ RAM (liên kết bảng Tocdo).
9	MaOdia	INT		Khóa ngoại	Mã ổ đĩa (liên kết bảng Odia).
10	MaMH	INT		Khóa ngoại	Mã màn hình (liên kết bảng Manhinh).
11	MaChuot	INT		Khóa ngoại	Mã chuột (liên kết bảng Chuot).
12	MaBP	INT		Khóa ngoại	Mã bàn phím (liên kết

					bảng Banphim).
13	MaTN	INT		Khóa ngoại	Mã tai nghe (liên kết bảng Tainghe).
14	Trangthai	NVARCHAR	50		Trạng thái máy (ví dụ: Đang thuê, Trống).

- **Bảng Nhân viên: Bảng lưu thông tin về nhân viên của quán game.**

Bảng 3.13. Bảng Nhân viên

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaNV	INT		Khóa chính (IDENTITY)	Mã định danh duy nhất cho mỗi nhân viên.
2	TenNV	NVARCHAR	100		Tên nhân viên.
3	Gioitinh	NVARCHAR	10		Giới tính của nhân viên (Nam/Nữ).
4	Namsinh	DATE			Ngày tháng năm sinh của nhân viên.

5	Diachi	NVARCHAR	100		Địa chỉ của nhân viên.
6	SDT	VARCHAR	20		Số điện thoại liên hệ của nhân viên.
7	Email	VARCHAR	100		Địa chỉ email của nhân viên.

- **Bảng Tài khoản:** Bảng lưu thông tin tài khoản đăng nhập của nhân viên.

Bảng 3.14. Bảng Tài khoản

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaTK	INT		Khóa chính (IDENTITY)	Mã định danh duy nhất cho mỗi tài khoản.
2	TenDangNhap	VARCHAR	50		Tên đăng nhập của tài khoản.
3	MatKhau	VARCHAR	50		Mật khẩu của tài khoản (lưu dạng plaintext).

4	MaNV	INT		Khóa ngoại	Mã nhân viên sở hữu tài khoản (liên kết bảng Nhanvien).
5	NgayTao	DATE			Ngày tạo tài khoản.

- **Bảng Loại món:** Bảng lưu thông tin về loại món (đồ ăn, đồ uống, snack).

Bảng 3.15. Bảng Loại món

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaLoai	INT		Khóa chính (IDENTITY)	Mã định danh duy nhất cho mỗi loại món.
2	TenLoai	NVARCHAR	100		Tên loại món (ví dụ: Đồ ăn, Đồ uống).

- **Bảng Món:** Bảng lưu thông tin về các món phục vụ tại quán.

Bảng 3.16. Bảng Món

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
------	---------	--------------	------------	------	------------

1	MaMon	INT		Khóa chính (IDENTITY)	Mã định danh duy nhất cho mỗi món.
2	TenMon	NVARCHAR	100		Tên món (ví dụ: Mì Hào Hào, Trà sữa).
3	Gia	DECIMAL	(10,2)		Giá bán của món (đơn vị: VNĐ).
4	GiaNhap	DECIMAL	(10,2)		Giá nhập của món (đơn vị: VNĐ).
5	TonKho	INT			Số lượng tồn kho của món.
6	MaLoai	INT		Khóa ngoại	Mã loại món (liên kết bảng LoaiMon).
7	Anh	NVARCHAR	255		Đường dẫn file ảnh của món.

- **Bảng Hóa đơn: Bảng lưu thông tin về hóa đơn (thu/chi) của quán.**

Bảng 3.17. Bảng Hóa đơn

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaHD	INT		Khóa chính (IDENTITY)	Mã định danh duy nhất cho mỗi hóa đơn.
2	LoaiHD	NVARCHAR	10		Loại hóa đơn (Thu/Chi).
3	NgayTao	DATETIME			Ngày giờ tạo hóa đơn.
4	MaNV	INT		Khóa ngoại	Mã nhân viên lập hóa đơn (liên kết bảng Nhanvien).
5	TongTien	DECIMAL	(10,2)		Tổng tiền của hóa đơn (mặc định 0).

- **Bảng Gọi món:** Bảng lưu thông tin chi tiết về các món được gọi trong hóa đơn.

Bảng 3.18. Bảng Gọi món

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
---------	---------	-----------------	------------	------	------------

1	MaHD	INT		Khóa chính, Khóa ngoại	Mã hóa đơn (liên kết bảng Hoadon).
2	MaMon	INT		Khóa chính, Khóa ngoại	Mã món (liên kết bảng Mon).
3	SoLuong	INT			Số lượng món được gọi.

- **Bảng Thuê máy: Bảng lưu thông tin về các phiên thuê máy tính.**

Bảng 3.19. Bảng Gọi món

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaTM	INT		Khóa chính (IDENTITY)	Mã định danh duy nhất cho mỗi phiên thuê.
2	MaMay	INT		Khóa ngoại	Mã máy tính được thuê (liên kết bảng Maytinh).
3	GioBD	DATETIME			Thời gian bắt đầu thuê máy.
4	GioKT	DATETIME			Thời gian kết thúc thuê máy (có thể NULL).

5	MaHD	INT		Khóa ngoại	Mã hóa đơn liên quan (liên kết bảng Hoadon).
---	------	-----	--	------------	--

- **Bảng Tình trạng: Bảng lưu thông tin về tình trạng lỗi của máy tính.**

Bảng 3.20. Bảng Tình trạng

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaTT	INT		Khóa chính	Mã định danh duy nhất cho mỗi tình trạng lỗi.
2	TenTT	NVARCHAR R	100		Tên tình trạng lỗi (ví dụ: Máy không khởi động).

- **Bảng Nguyên nhân: Bảng lưu thông tin về nguyên nhân gây lỗi của máy tính.**

Bảng 3.21. Bảng Nguyên nhân

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaNN	INT		Khóa chính	Mã định danh duy nhất cho mỗi nguyên nhân.
2	TenNN	NVARCHAR	100		Tên nguyên nhân (ví dụ:

					Hồng ổ cứng).
--	--	--	--	--	---------------

- **Bảng Giải pháp: Bảng lưu thông tin về các giải pháp của các lần bảo trì máy**

Bảng 3.22. Bảng Giải pháp

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaGP	INT		Khóa chính	Mã định danh duy nhất cho mỗi giải pháp.
2	TenGP	NVARCHA R	100		Tên giải pháp (ví dụ: Thay ổ cứng mới).
3	Chiphi	DECIMAL	(10,2)		Chi phí thực hiện giải pháp (đơn vị: VNĐ).

- **Bảng Nhà bảo trì: Bảng lưu thông tin về nhà bảo trì máy tính.**

Bảng 3.23. Bảng Nhà bảo trì

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaNBT	INT		Khóa chính	Mã định danh duy nhất cho mỗi nhà bảo trì.

2	TenNBT	NVARCHAR	100		Tên nhà bảo trì (ví dụ: Công ty TNHH Sửa chữa PC).
3	DiaChi	NVARCHAR	200		Địa chỉ của nhà bảo trì.
4	SDT	NVARCHAR	20		Số điện thoại liên hệ của nhà bảo trì.

- **Bảng Bảo trì: Bảng lưu thông tin về các lần bảo trì máy tính.**

Bảng 3.24. Bảng Bảo trì

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaBT	INT		Khóa chính (IDENTITY)	Mã định danh duy nhất cho mỗi lần bảo trì.
2	NgayBT	DATE			Ngày thực hiện bảo trì (phải nhỏ hơn hoặc bằng ngày hiện tại).
3	MaNBT	INT		Khóa ngoại	Mã nhà bảo trì (liên kết bảng NhaBT).
4	MaHD	INT		Khóa ngoại	Mã hóa đơn liên quan (liên kết bảng

					Hoadon).
5	GhiChu	NVARCHAR	255		Ghi chú về lần bảo trì (ví dụ: Giải pháp áp dụng).
6	TrangThai	NVARCHAR	50		Trạng thái bảo trì (Đã hoàn tất/Chưa thanh toán).

- **Bảng Chi tiết bảo trì:** Bảng lưu thông tin chi tiết về các lần bảo trì máy tính.

Bảng 3.25. Bảng Chi tiết bảo trì

ST T	Tên cột	Kiểu dữ liệu	Kích thước	Khóa	Giải thích
1	MaBT	INT		Khóa chính, Khóa ngoại	Mã lần bảo trì (liên kết bảng Baotri).
2	MaMay	INT		Khóa chính, Khóa	Mã máy tính được bảo trì (liên kết bảng Maytinh).

				ngoại	
3	MaTT	INT		Khóa ngoại	Mã tình trạng lỗi (liên kết bảng Tinhtrang).
4	MaNN	INT		Khóa ngoại	Mã nguyên nhân lỗi (liên kết bảng Nguyennhan).
5	MaGP	INT		Khóa ngoại	Mã giải pháp áp dụng (liên kết bảng Giaiphap).

3.3 CÁC PROCEDURE SỬ DỤNG TRONG CSDL

- PROCEDURE sp_GetBillListFiltered

```
CREATE OR ALTER PROCEDURE sp_GetBillListFiltered
    @NgayBD DATETIME,
    @NgayKT DATETIME,
    @TenNV NVARCHAR(100) = NULL,
    @LoaiHD NVARCHAR(50) = NULL
AS
BEGIN
    SELECT
        h.MaHD,
        h.NgayTao,
        nv.TenNV,
        h.LoaiHD,
        ISNULL(bt.ChiPhi, 0) AS ChiPhi,
        ISNULL(gm.TongGiaMon, 0) AS GiaMon,
        ISNULL(tm.TongThueMay, 0) AS ThueMay,
        h.TongTien
    FROM Hoadon h
```

```

LEFT JOIN Nhanvien nv ON h.MaNV = nv.MaNV
LEFT JOIN (
    SELECT bt.MaHD, SUM(gp.ChiPhi) AS ChiPhi
    FROM Baotri bt
    JOIN ChitietBT cbt ON bt.MaBT = cbt.MaBT
    JOIN Giaiphap gp ON cbt.MaGP = gp.MaGP
    GROUP BY bt.MaHD
) bt ON h.MaHD = bt.MaHD
LEFT JOIN (
    SELECT g.MaHD, SUM(g.SoLuong * m.Gia) AS TongGiaMon
    FROM Goimon g
    JOIN Mon m ON g.MaMon = m.MaMon
    GROUP BY g.MaHD
) gm ON h.MaHD = gm.MaHD
LEFT JOIN (
    SELECT tm.MaHD,
        SUM(DATEDIFF(MINUTE, tm.GioBD, ISNULL(tm.GioKT, GETDATE())))
/ 60.0 * p.Giatheogio) AS TongThueMay
FROM Thuemay tm
JOIN Maytinh mt ON tm.MaMay = mt.MaMay
JOIN Phong p ON mt.MaPhong = p.MaPhong
GROUP BY tm.MaHD
) tm ON h.MaHD = tm.MaHD
WHERE h.NgayTao BETWEEN @NgayBD AND @NgayKT
    AND (@TenNV IS NULL OR nv.TenNV LIKE '%' + @TenNV + '%')
    AND (@LoaiHD IS NULL OR h.LoaiHD = @LoaiHD)
END
go

```

Thủ tục này nhận vào các tham số đầu vào: @NgayBD, @NgayKT kiểu DATETIME; @TenNV, @LoaiHD kiểu NVARCHAR(100). Khi thủ tục được gọi, nó thực hiện truy vấn để lấy danh sách hóa đơn (Hoadon) có ngày tạo nằm trong khoảng từ @NgayBD đến @NgayKT. Thủ tục có thể lọc thêm theo tên nhân viên (@TenNV) và loại hóa đơn (@LoaiHD) nếu được cung cấp. Kết quả trả về bao gồm: mã hóa đơn, ngày tạo, tên nhân viên, loại hóa đơn, chi phí bảo trì, giá món, thuế thuê máy và tổng tiền.

- PROCEDURE sp_GetAllPhong

```

CREATE PROCEDURE sp_GetAllPhong
AS

```

```
BEGIN
    SELECT MaPhong, TenPhong, SoMay, Giatheogio
    FROM Phong
END
go
```

Thủ tục này không nhận tham số đầu vào. Khi được gọi, thủ tục thực hiện truy vấn để trả về tất cả các bản ghi trong bảng Phong. Kết quả gồm các cột: MaPhong, TenPhong, SoMay, Giatheogio.

- PROCEDURE sp_UpdateTongTien

```
CREATE PROCEDURE sp_UpdateTongTien
    @MaHD INT,
    @Amount DECIMAL(18,2)
AS
BEGIN
    UPDATE Hoadon
    SET TongTien = TongTien + @Amount
    WHERE MaHD = @MaHD
END
go
```

Thủ tục này nhận hai tham số đầu vào: @MaHD kiểu INT, @Amount kiểu DECIMAL(18,2). Khi thủ tục được gọi, nó sẽ thực hiện lệnh cập nhật (UPDATE) trên bảng Hoadon, tăng tổng tiền (TongTien) của hóa đơn có mã là @MaHD thêm một giá trị bằng @Amount.

- PROCEDURE sp_CalculateTotalByMaHD

```
CREATE PROCEDURE sp_CalculateTotalByMaHD
    @MaHD INT,
    @TongTien DECIMAL(18,2) OUTPUT
AS
BEGIN
    SELECT @TongTien = ISNULL(SUM(g.SoLuong * m.Gia), 0)
    FROM Goimon g
    JOIN Mon m ON g.MaMon = m.MaMon
    WHERE g.MaHD = @MaHD
END
```

```
go
```

Thủ tục này nhận một tham số đầu vào là @MaHD (INT) và một biến output @TongTien (DECIMAL). Khi thủ tục được gọi, nó sẽ tính tổng tiền các món trong hóa đơn tương ứng bằng cách nhân số lượng với giá món, sau đó gán kết quả vào biến @TongTien.

- PROCEDURE sp_GetGoiMonByMaHD

```
CREATE PROCEDURE sp_GetGoiMonByMaHD
    @MaHD INT
AS
BEGIN
    SELECT g.MaMon, m.TenMon, g.SoLuong, m.Gia, (g.SoLuong * m.Gia) AS
    ThanhTien
    FROM Goimon g
    JOIN Mon m ON g.MaMon = m.MaMon
    WHERE g.MaHD = @MaHD
END
go
```

Thủ tục này nhận @MaHD là mã hóa đơn và trả về danh sách các món ăn trong hóa đơn, gồm mã món, tên món, số lượng, đơn giá và thành tiền (tính bằng số lượng nhân giá).

- PROCEDURE sp_AddOrUpdateGoiMon

```
CREATE PROCEDURE sp_AddOrUpdateGoiMon
    @MaHD INT,
    @MaMon INT,
    @SoLuong INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    IF EXISTS (SELECT 1 FROM Goimon WHERE MaHD = @MaHD AND MaMon = @MaMon)
        UPDATE Goimon SET SoLuong = SoLuong + @SoLuong WHERE MaHD = @MaHD
    AND MaMon = @MaMon
    ELSE
```

```

        INSERT INTO Goimon (MaHD, MaMon, SoLuong) VALUES (@MaHD, @MaMon,
@SoLuong)
        COMMIT;
END
go

```

Thủ tục này nhận mã hóa đơn, mã món và số lượng. Nếu món ăn đã tồn tại trong hóa đơn thì cập nhật cộng thêm số lượng, nếu chưa thì chèn mới một dòng vào bảng Goimon.

- PROCEDURE sp_CapNhatMon:

```

CREATE PROCEDURE sp_CapNhatMon
    @MaMon INT,
    @TenMon NVARCHAR(100),
    @Gia DECIMAL(18,2),
    @GiaNhap DECIMAL(18,2),
    @TonKho INT,
    @MaLoai INT,
    @Anh NVARCHAR(255)
AS
BEGIN
    UPDATE Mon
    SET TenMon = @TenMon, Gia = @Gia, GiaNhap = @GiaNhap, TonKho =
@TonKho, MaLoai = @MaLoai, Anh = @Anh
    WHERE MaMon = @MaMon
END
go

```

Thủ tục này cập nhật thông tin một món ăn dựa trên mã món. Các trường được cập nhật gồm tên món, giá bán, giá nhập, số lượng tồn kho, mã loại và ảnh minh họa.

- PROCEDURE sp_XoaMon:

```

CREATE PROCEDURE sp_XoaMon
    @MaMon INT,
    @Result INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM Goimon WHERE MaMon = @MaMon)

```



```

        SET @Result = -1 -- Món đang được sử dụng
    ELSE
    BEGIN
        DELETE FROM Mon WHERE MaMon = @MaMon
        SET @Result = @@ROWCOUNT
    END
END
go

```

Thủ tục này nhận @MaMon là mã món và một biến output @Result. Nếu món đang được sử dụng trong bảng gọi món thì không xóa và trả -1, nếu không thì thực hiện xóa và trả về số dòng bị xóa qua @Result.

- PROCEDURE sp_GetMaHDFromThuemay:

```

CREATE PROCEDURE sp_GetMaHDFromThuemay
    @MaMay INT,
    @MaPhong INT,
    @MaHD INT OUTPUT
AS
BEGIN
    SELECT TOP 1 @MaHD = t.MaHD
    FROM Thuemay t
    JOIN Maytinh m ON t.MaMay = m.MaMay
    WHERE t.MaMay = @MaMay
    AND m.MaPhong = @MaPhong
    AND m.Trangthai = 'Dang thue'
    AND t.GioKT IS NULL
    ORDER BY t.GioBD DESC
END
go

```

Thủ tục này lấy mã hóa đơn @MaHD tương ứng với máy tính đang được thuê (theo mã máy và mã phòng) với điều kiện máy có trạng thái "Đang thuê" và chưa kết thúc giờ thuê.

- PROCEDURE sp_CapNhatTongTienHoaDon:

```

CREATE PROCEDURE sp_CapNhatTongTienHoaDon
    @MaHD INT,
    @TongTien DECIMAL(18,2)

```

```

AS
BEGIN
    UPDATE Hoadon
    SET TongTien = ISNULL(TongTien, 0) + @TongTien
    WHERE MaHD = @MaHD
END
go

```

Thủ tục này nhận @MaHD và @TongTien, sau đó cộng thêm số tiền vào tổng tiền của hóa đơn tương ứng trong bảng Hoadon.

- PROCEDURE sp_GetMaintainListForMachine

```

CREATE PROCEDURE sp_GetMaintainListForMachine
    @MaMay INT
AS
BEGIN
    SELECT m.TenMay,
           t.TenTT AS BoPhan,
           n.TenNN AS NguyenNhan,
           g.TenGP AS GiaiPhap,
           b.GhiChu,
           b.MaBT,
           g.Chiphi,
           b.Trangthai AS TrangthaiBaotri
    FROM dbo.Maytinh m
    JOIN dbo.ChitietBT c ON m.MaMay = c.MaMay
    JOIN dbo.Baotri b ON c.MaBT = b.MaBT
    JOIN dbo.Tinhtrang t ON c.MaTT = t.MaTT
    JOIN dbo.Nguyennhan n ON c.MaNN = n.MaNN
    JOIN dbo.Giaiphap g ON c.MaGP = g.MaGP
    WHERE m.MaMay = @MaMay AND m.Trangthai = N'Bao loi';
END
GO

```

Thủ tục này dùng để lấy danh sách các lần bảo trì của một máy tính cụ thể theo @MaMay. Nó truy vấn thông tin từ nhiều bảng liên quan gồm: tên máy, bộ phận lỗi, nguyên nhân, giải pháp, chi phí, ghi chú và trạng thái bảo trì. Điều kiện lọc thêm là máy đang có trạng thái "Bảo lỗi".

- PROCEDURE sp_CreateBaotri

```
CREATE OR ALTER PROCEDURE sp_CreateBaotri
    @MaNV INT,
    @MaMay INT,
    @MaTT INT,
    @MaNN INT,
    @MaGP INT,
    @GhiChu NVARCHAR(MAX),
    @ChiPhi DECIMAL(18,2),
    @MaNBT INT,
    @MaBT INT OUTPUT,
    @MaHD INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        -- Tạo hóa đơn
        INSERT INTO dbo.HoaDon (LoaiHD, NgayTao, MaNV, TongTien)
        VALUES ('Chi', GETDATE(), @MaNV, 0);
        SET @MaHD = SCOPE_IDENTITY();

        -- Cập nhật trạng thái máy
        UPDATE dbo.Maytinh
        SET Trangthai = N'Bao loi'
        WHERE MaMay = @MaMay;

        -- Tạo bản ghi bảo trì
        INSERT INTO dbo.Baotri (NgayBT, MaNBT, MaHD, GhiChu, Trangthai)
        VALUES (CAST(GETDATE() AS DATE), @MaNBT, @MaHD, @GhiChu, N'Chưa
        thanh toán');
        SET @MaBT = SCOPE_IDENTITY();

        -- Tạo chi tiết bảo trì
        INSERT INTO dbo.ChitietBT (MaBT, MaMay, MaTT, MaNN, MaGP)
        VALUES (@MaBT, @MaMay, @MaTT, @MaNN, @MaGP);

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
```

```

        DECLARE @ErrorSeverity INT = ERROR_SEVERITY();
        DECLARE @ErrorState INT = ERROR_STATE();
        RAISERROR (@ErrorMessage, @ErrorSeverity, @ErrorState);
    END CATCH;
END
GO

```

Thủ tục này thực hiện việc tạo mới một phiếu bảo trì gồm các bước: Tạo mới hóa đơn chi (loại "Chi") cho bảo trì. Cập nhật trạng thái máy thành "Báo lỗi". Tạo bản ghi bảo trì (gắn với nhân viên tiếp nhận, ghi chú, hóa đơn...). Tạo bản ghi chi tiết bảo trì với mã máy, tình trạng, nguyên nhân và giải pháp; Tất cả thực hiện trong giao dịch để đảm bảo tính toàn vẹn.

- PROCEDURE sp_ThanhToanBaotri

```

CREATE OR ALTER PROCEDURE sp_ThanhToanBaotri
    @MaNV INT,
    @MaBTList NVARCHAR(MAX), -- Danh sách MaBT cách nhau bởi dấu phẩy
    @MaHD INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        DECLARE @MaBTTable TABLE (MaBT INT);
        INSERT INTO @MaBTTable (MaBT)
        SELECT TRY_CAST(value AS INT)
        FROM STRING_SPLIT(@MaBTList, ',')
        WHERE TRY_CAST(value AS INT) IS NOT NULL;

        SELECT TOP 1 @MaHD = b.MaHD
        FROM dbo.Baotri b
        JOIN @MaBTTable mt ON b.MaBT = mt.MaBT;

        IF @MaHD IS NULL
        BEGIN
            RAISERROR (N'Không tìm thấy mã hóa đơn cho các bản ghi bảo trì.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        RETURN;
    END CATCH
END

```

```

END
-- Tính tổng chi phí
DECLARE @TotalCost DECIMAL(18,2);
SELECT @TotalCost = SUM(g.ChiPhi)
FROM @MaBTTable mt
JOIN dbo.Baotri b ON mt.MaBT = b.MaBT
JOIN dbo.ChitietBT c ON b.MaBT = c.MaBT
JOIN dbo.Giaiphap g ON c.MaGP = g.MaGP
WHERE b.Trangthai != N'Đã hoàn tất';

IF @TotalCost IS NULL
    SET @TotalCost = 0;

-- Cập nhật tổng tiền hóa đơn
UPDATE dbo.HoaDon
SET TongTien = @TotalCost
WHERE MaHD = @MaHD;
-- Cập nhật trạng thái bảo trì
UPDATE b
SET b.Trangthai = N'Đã hoàn tất'
FROM dbo.Baotri b
JOIN @MaBTTable mt ON b.MaBT = mt.MaBT
WHERE b.Trangthai != N'Đã hoàn tất'
-- Cập nhật trạng thái máy
UPDATE m
SET m.Trangthai = CASE
    WHEN EXISTS (
        SELECT 1
        FROM dbo.ChitietBT c
        JOIN dbo.Baotri b ON c.MaBT = b.MaBT
        WHERE c.MaMay = m.MaMay AND b.Trangthai != N'Đã hoàn tất'
    ) THEN N'Bao loi'
    ELSE N'Trong'
END
FROM dbo.Maytinh m
WHERE m.MaMay IN (
    SELECT DISTINCT c.MaMay
    FROM @MaBTTable mt
    JOIN dbo.ChitietBT c ON mt.MaBT = c.MaBT
);

COMMIT TRANSACTION;

```

```

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrorSeverity INT = ERROR_SEVERITY();
    DECLARE @ErrorState INT = ERROR_STATE();
    RAISERROR (@ErrorMessage, @ErrorSeverity, @ErrorState);
END CATCH
END;
go

```

Thủ tục này thực hiện thanh toán cho một hoặc nhiều bản ghi bảo trì (danh sách truyền vào dạng chuỗi @MaBTList):

- Tạo bảng tạm để lưu danh sách MaBT.
- Tính tổng chi phí từ các giải pháp liên quan.
- Cập nhật tổng tiền vào hóa đơn.
- Cập nhật trạng thái của các bản ghi bảo trì thành "Đã hoàn tất".
- Cập nhật trạng thái máy: nếu không còn lỗi thì chuyển thành "Trống", ngược lại vẫn là "Báo lỗi".

- PROCEDURE sp_StartThueMay

```

CREATE PROCEDURE sp_StartThueMay
    @MaMay INT,
    @MaHD INT,
    @MaTM INT OUTPUT
AS
BEGIN
    INSERT INTO ThueMay (MaMay, GioBD, GioKT, MaHD)
    VALUES (@MaMay, GETDATE(), NULL, @MaHD)
    SET @MaTM = SCOPE_IDENTITY()
END
go

```

Thủ tục này dùng để bắt đầu một phiên thuê máy. Nó thêm một dòng vào bảng ThueMay với GioBD là thời gian hiện tại và gán @MaTM là mã thuê máy mới sinh.

- PROCEDURE sp_EndThueMay

```
CREATE OR ALTER PROCEDURE sp_EndThueMay @MaTM INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    DECLARE @MaMay INT;

    -- Kiểm tra phiên thuê tồn tại và chưa kết thúc
    IF NOT EXISTS (SELECT 1 FROM ThueMay WHERE MaTM = @MaTM AND GioKT IS
NULL)
    BEGIN
        RAISERROR(N'Phiên thuê không tồn tại hoặc đã kết thúc.', 16, 1);
        RETURN;
    END

    -- Lưu MaMay vào biến
    SELECT @MaMay = MaMay FROM ThueMay WHERE MaTM = @MaTM;

    -- Giao dịch
    BEGIN TRANSACTION;

    -- Cập nhật thời gian kết thúc
    UPDATE ThueMay
    SET GioKT = GETDATE()
    WHERE MaTM = @MaTM;

    -- Cập nhật trạng thái máy
    UPDATE MayTinh
    SET Trangthai = N'Trong'
    WHERE MaMay = @MaMay;

    COMMIT TRANSACTION;
END
GO
```

Thủ tục này kết thúc phiên thuê máy theo mã @MaTM: Kiểm tra xem phiên thuê còn hoạt động không (GioKT NULL). Nếu hợp lệ thì cập nhật GioKT, đồng thời cập nhật trạng thái máy về "Trống".

- PROCEDURE sp_CreateHoaDon

```
CREATE PROCEDURE sp_CreateHoaDon
    @MaNV INT,
    @LoaiHD NVARCHAR(10),
    @MaHD INT OUTPUT
AS
BEGIN
    INSERT INTO Hoadon (NgayTao, MaNV, LoaiHD, TongTien)
    VALUES (GETDATE(), @MaNV, @LoaiHD, 0)
    SET @MaHD = SCOPE_IDENTITY()
END
go
```

Thủ tục này dùng để tạo mới một hóa đơn, gồm ngày tạo, mã nhân viên và loại hóa đơn (thu hoặc chi), ban đầu tổng tiền là 0. Mã hóa đơn mới được trả về qua @MaHD.

CHƯƠNG 4: XÂY DỰNG CHƯƠNG TRÌNH

4.1. GIỚI THIỆU VỀ KIẾN TRÚC 3 LỚP

Chương trình quản lý cửa hàng được xây dựng theo mô hình 3 lớp, nhằm tách biệt các chức năng khác nhau để tăng tính modul, dễ bảo trì và mở rộng. Mô hình này bao gồm ba lớp chính:

Lớp DAL (Data Access Layer): Chịu trách nhiệm tương tác với cơ sở dữ liệu, thực hiện các thao tác truy vấn và quản lý dữ liệu.

Lớp DTO (Data Transfer Object): Dùng để truyền dữ liệu giữa các lớp trong ứng dụng một cách hiệu quả.

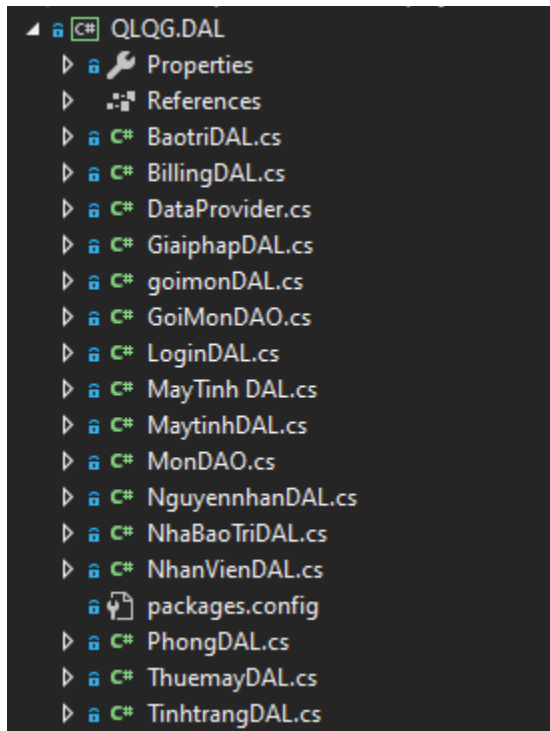
Lớp GUI (Graphical User Interface): Là giao diện người dùng, nơi người dùng thực hiện các thao tác như đặt máy, thanh toán, quản lý bảo trì, v.v. (Microsoft, 2025)

Dưới đây là chi tiết chức năng và nhiệm vụ của từng lớp:

4.1.1 Lớp DAL (Data Access Layer)

Lớp DAL là tầng trung gian giữa ứng dụng và cơ sở dữ liệu, đảm bảo rằng tất cả các thao tác liên quan đến dữ liệu được thực hiện một cách tập trung và hiệu quả. Nhiệm vụ chính của lớp này bao gồm:

- Kết nối cơ sở dữ liệu.
- Thực hiện các truy vấn (query).
- Quản lý thao tác dữ liệu.



Hình 4.1. Lớp DAL

- Code kết nối với cơ sở dữ liệu:

```
namespace QLQG.DAL
{
    public class DataProvider
    {
        private static DataProvider instance;
        private readonly string connectionString =
System.Configuration.ConfigurationManager.ConnectionStrings["QLQuanGameCon
nection"].ConnectionString;

        public static DataProvider Instance
        {
            get
            {
                if (instance == null)
                {
                    instance = new DataProvider();
                }
                return instance;
            }
        }
    }
}
```

```

    }
    private set => instance = value;
}

private DataProvider() { }

public SqlConnection GetConnection()
{
    return new SqlConnection(connectionString);
}

public DataTable ExecuteQuery(string query, (string, object)[]
parameters, CommandType commandType = CommandType.Text)
{
    try
    {
        using (SqlConnection connection = new
SqlConnection(connectionString))
        {
            connection.Open();
            using (SqlCommand command = new SqlCommand(query,
connection))
            {
                command.CommandType = commandType;
                if (parameters != null)
                {
                    foreach (var (name, value) in parameters)
                    {
                        Console.WriteLine($"ExecuteQuery: Thêm
tham số {name} = {value}");
                        command.Parameters.AddWithValue(name,
value ?? DBNull.Value);
                    }
                }

                using (SqlDataAdapter adapter = new
SqlDataAdapter(command))
                {
                    DataTable dt = new DataTable();
                    adapter.Fill(dt);
                    return dt;
                }
            }
        }
    }
}

```

```

    }
    }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Lỗi trong ExecuteQuery: {ex.Message}\n{ex.StackTrace}");
        throw new Exception($"Lỗi khi thực thi truy vấn: {query}",
ex);
    }
}

public int ExecuteNonQuery(string query, (string, object)[]
parameters, CommandType commandType = CommandType.Text)
{
    try
    {
        using (SqlConnection connection = new
SqlConnection(connectionString))
        {
            connection.Open();
            using (SqlCommand command = new SqlCommand(query,
connection))
            {
                command.CommandType = commandType;
                if (parameters != null)
                {
                    foreach (var (name, value) in parameters)
                    {
                        Console.WriteLine($"ExecuteNonQuery: Thêm
tham số {name} = {value}");
                        command.Parameters.AddWithValue(name,
value ?? DBNull.Value);
                    }
                }
                return command.ExecuteNonQuery();
            }
        }
    }
    catch (Exception ex)
    {

```

```

        Console.WriteLine($"Lỗi trong ExecuteNonQuery:
{ex.Message}\n{ex.StackTrace}");
        throw new Exception($"Lỗi khi thực thi truy vấn: {query}",
ex);
    }
}

public object ExecuteScalar(string query, params (string, object)
[] parameters)
{
    try
    {
        using (SqlConnection connection = new
SqlConnection(connectionString))
        {
            connection.Open();
            using (SqlCommand command = new SqlCommand(query,
connection))
            {
                command.CommandType = CommandType.Text;
                if (parameters != null)
                {
                    foreach (var (name, value) in parameters)
                    {
                        Console.WriteLine($"ExecuteScalar: Thêm
tham số {name} = {value}");
                        command.Parameters.AddWithValue(name,
value ?? DBNull.Value);
                    }
                }
                return command.ExecuteScalar();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Lỗi trong ExecuteScalar: {ex.Message}\
n{ex.StackTrace}");
        throw new Exception($"Lỗi khi thực thi truy vấn: {query}",
ex);
    }
}

```

```

        public void ExecuteStoredProcedure(string storedProcedureName, out
Dictionary<string, object> outputParams, params (string, object)[]
parameters)
        {
            try
            {
                using (SqlConnection connection = new
SqlConnection(connectionString))
                {
                    connection.Open();
                    using (SqlCommand command = new
SqlCommand(storedProcedureName, connection))
                    {
                        command.CommandType = CommandType.StoredProcedure;
                        Dictionary<string, object> outputValues = new
Dictionary<string, object>();

                        if (parameters != null)
                        {
                            foreach (var (name, value) in parameters)
                            {
                                SqlParameter param = new
SqlParameter(name, value ?? DBNull.Value);
                                if (value == null)
                                {
                                    param.Direction =
ParameterDirection.Output;
                                    param.SqlDbType = SqlDbType.Int;
                                }

                                Console.WriteLine($"ExecuteStoredProcedure: Thêm tham số {name} =
{value}");
                                command.Parameters.Add(param);
                            }
                        }

                        command.ExecuteNonQuery();

                        foreach (SqlParameter param in command.Parameters)
                        {

```

```

        if (param.Direction ==
ParameterDirection.Output)
        {
            outputValues[param.ParameterName] =
param.Value;
        }

        outputParams = outputValues;
    }
}
catch (Exception ex)
{
    Console.WriteLine($"Lỗi trong ExecuteStoredProcedure:
{ex.Message}\n{ex.StackTrace}");
    throw new Exception($"Lỗi khi thực thi stored procedure:
{storedProcedureName}", ex);
}
}
}
}

```

Kết luận:

Lớp DAL đóng vai trò quan trọng trong việc tách biệt logic truy cập dữ liệu khỏi các lớp khác, giúp chương trình dễ dàng chuyển đổi sang cơ sở dữ liệu khác (chỉ cần thay đổi chuỗi kết nối) mà không ảnh hưởng đến các lớp khác. (IBM, 2025)

4.1.2. Lớp DTO (Data Transfer Object)

Lớp DTO là một lớp trung gian được thiết kế để truyền dữ liệu giữa các tầng của ứng dụng (DAL, GUI, và các tầng logic khác nếu có). Nhiệm vụ chính của lớp này bao gồm:

- Truyền dữ liệu giữa các lớp:

DTO chứa các đối tượng đại diện cho các thực thể trong cơ sở dữ liệu (như Maytinh, Hoadon, Baotri) dưới dạng các thuộc tính (properties).

Ví dụ, một lớp DTO cho bảng Maytinh có thể bao gồm các thuộc tính như MaMay, TenMay, và Trangthai, được sử dụng để truyền dữ liệu từ DAL lên GUI hoặc ngược lại.

- Không chứa logic xử lý:

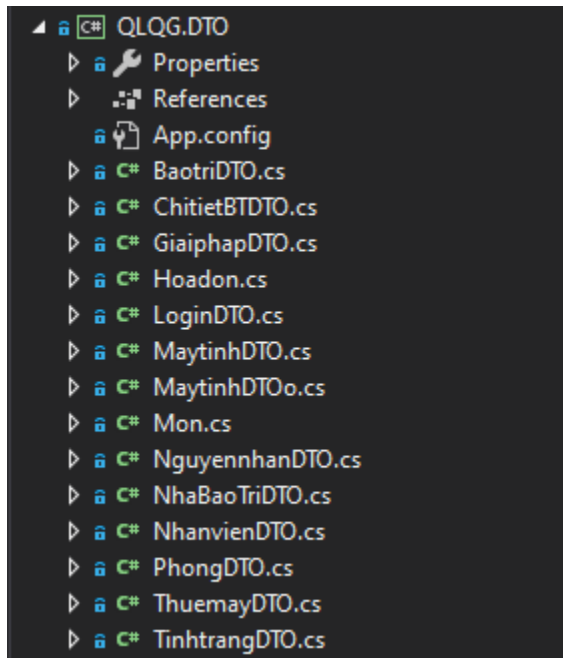
DTO chỉ đóng vai trò lưu trữ dữ liệu, không thực hiện bất kỳ logic nghiệp vụ nào (như tính toán hay kiểm tra). Logic xử lý được để lại cho các lớp khác (ví dụ: lớp DAL hoặc lớp Business Logic nếu có).

Điều này giúp giảm độ phức tạp của DTO và tăng tính tái sử dụng.

- Tối ưu hóa truyền dữ liệu:

DTO giúp giảm số lần truy cập cơ sở dữ liệu bằng cách đóng gói toàn bộ dữ liệu cần thiết vào một đối tượng, thay vì truyền từng giá trị riêng lẻ.

Điều này đặc biệt hữu ích khi truyền dữ liệu qua mạng hoặc giữa các tầng trong ứng dụng phân tán. (IBM, 2025)

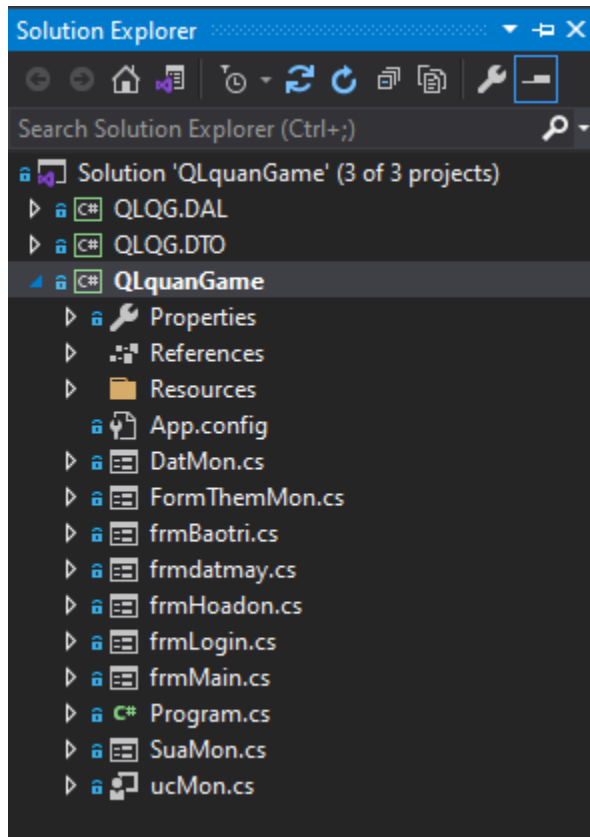


Hình 4.2. Lớp DTO

4.1.3. Lớp GUI (Graphical User Interface)

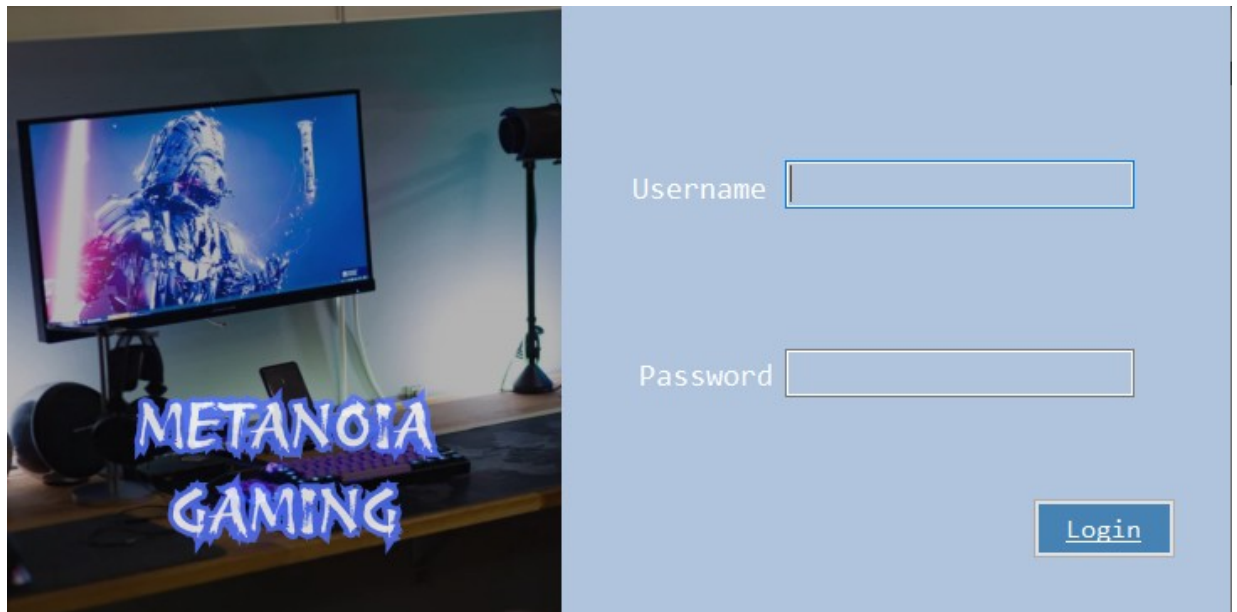
Lớp GUI là tầng giao diện người dùng, nơi người dùng tương tác trực tiếp với chương trình để thực hiện các chức năng quản lý cửa hàng. Nhiệm vụ chính của lớp này bao gồm:

- Cung cấp giao diện tương tác.
- Hỗ trợ người dùng thực hiện các chức năng.
- Hiển thị và cập nhật dữ liệu.
- Xử lý sự kiện người dùng. (IBM, 2025)

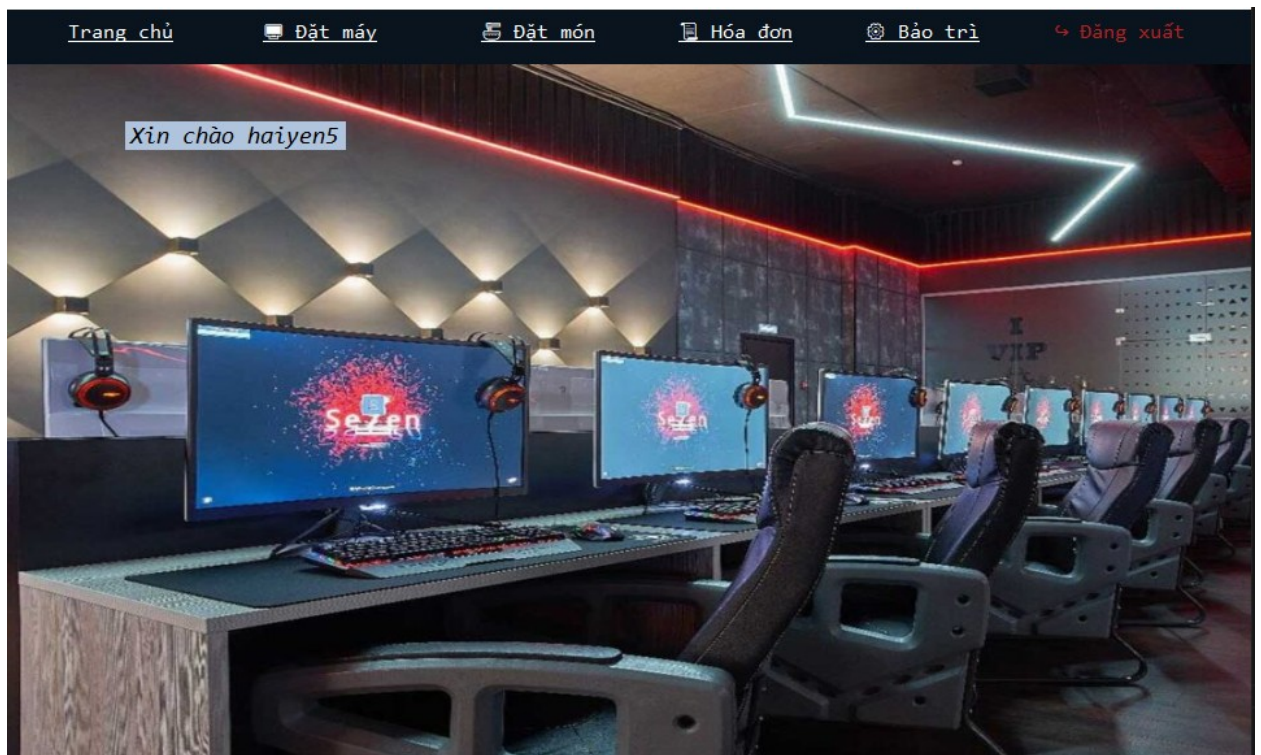


Hình 4.3. Lớp GUI

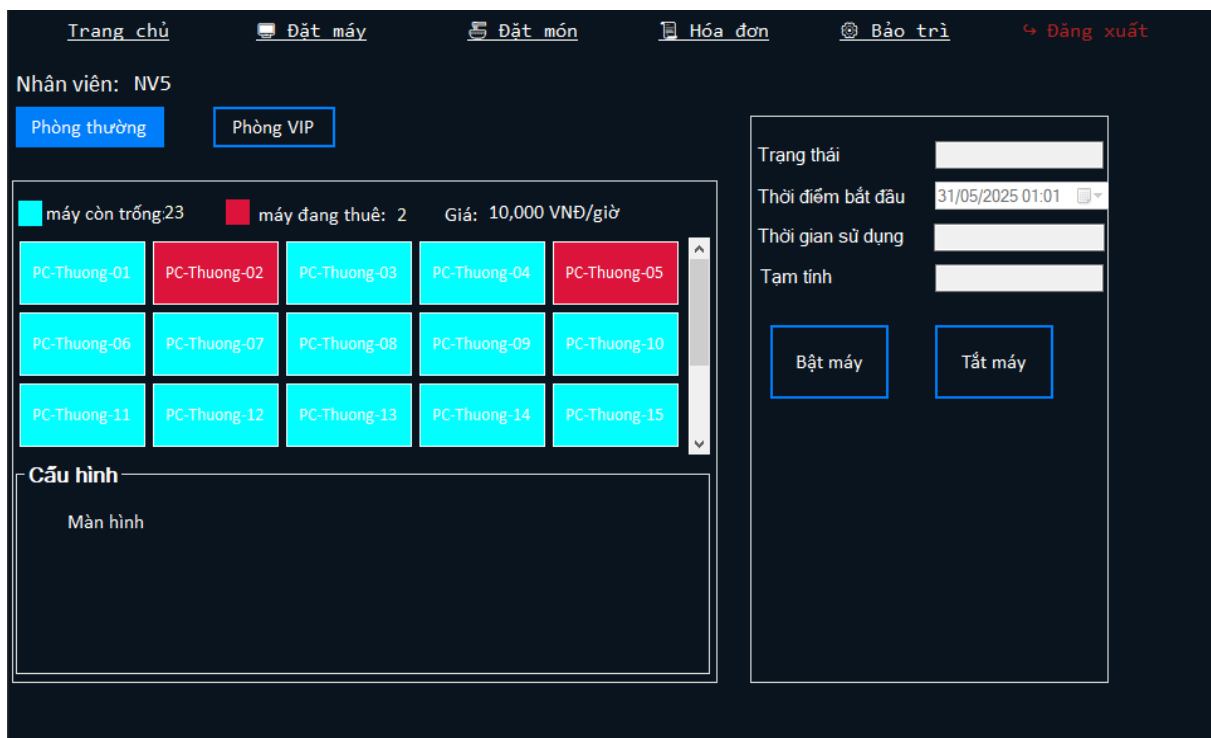
- Giao diện của chương trình quản lý quán game:



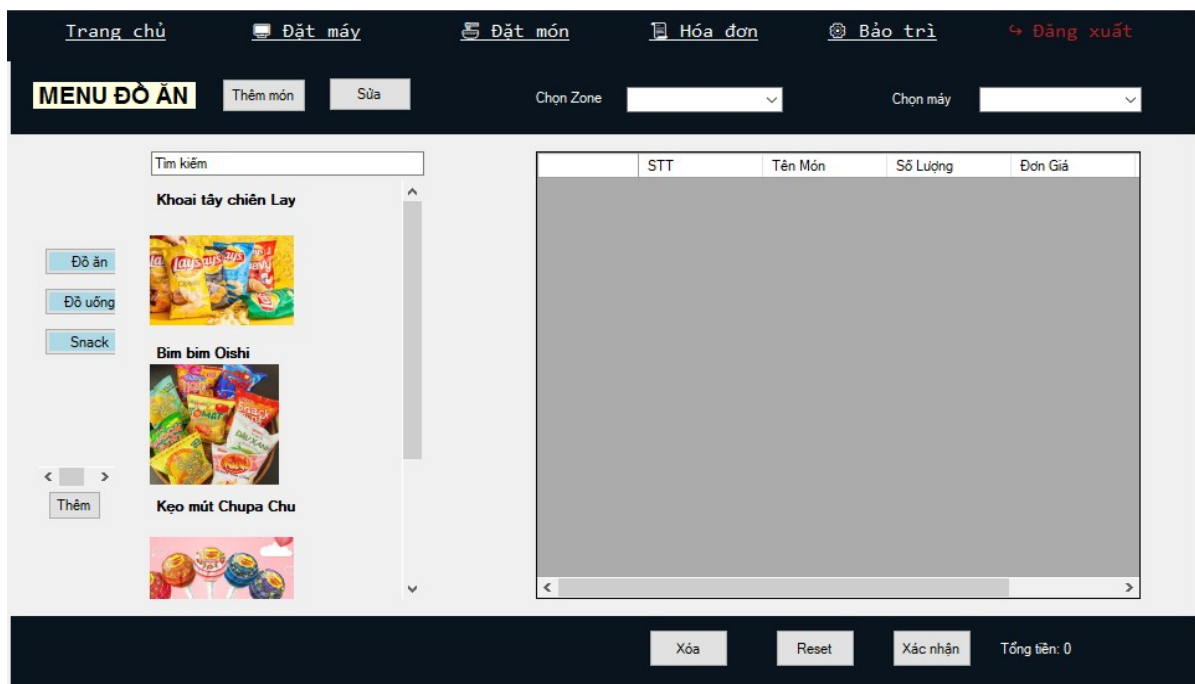
Hình 4.4. Giao diện Đăng nhập



Hình 4.5. Giao diện Trang chủ



Hình 4.6. Giao diện Đặt máy



Hình 4.7. Giao diện Đặt món

[Trang chủ](#)
[Đặt máy](#)
[Đặt món](#)
[Hóa đơn](#)
[Bảo trì](#)
[Đăng xuất](#)

Lịch sử thanh toán

Bộ lọc

Nhân viên

Thu

Chi

Reset

Mã hóa đơn	Ngày thanh toán	Nhân viên	Loại hóa đơn	Giá món ăn	Chi phí bảo trì	Giá thuê máy
1	15/05/2025 10:00	Bùi Thị Hoa	Thu	110.000,00 đ	0,00 đ	3.771.000,00 đ
2	16/05/2025 15:30	Nguyen Thi ...	Thu	95.000,00 đ	0,00 đ	3.476.000,00 đ
3	17/05/2025 09:00	Nguyen Thi ...	Chi	0,00 đ	0,00 đ	0,00 đ
4	18/05/2025 20:00	Nguyen Thi ...	Thu	80.000,00 đ	0,00 đ	8.813.000,00 đ
5	19/05/2025 14:00	Nguyen Thi ...	Chi	0,00 đ	0,00 đ	0,00 đ
32	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	400.000,00 đ	0,00 đ
36	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
38	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
42	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
44	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
48	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.800.000,00 đ	0,00 đ
51	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
55	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
58	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ

● Thống kê theo khoảng

Từ ngày

01/05/2025 00:00

Đến

31/05/2025 01:06

Tổng thu

13.224.166,65 đ

Tổng chi

10.400.000,00 đ

Lợi nhuận

2.824.166,65 đ

● Thống kê theo tháng

Tháng

05/2025

Lợi nhuận

0

Tháng

04/2025

Lợi nhuận

0

In hóa đơn

Hình 4.8. Giao diện Hóa đơn

[Trang chủ](#)
[Đặt máy](#)
[Đặt món](#)
[Hóa đơn](#)
[Bảo trì](#)
[Đăng xuất](#)

Nhân viên: Nguyen Thi Hai Yen

Bảo trì

máy còn trống: 27

máy đang sử dụng: 3

máy báo lỗi: 0

PC-Thuong-01	PC-Thuong-02	PC-Thuong-03	PC-Thuong-04	PC-Thuong-05
PC-Thuong-06	PC-Thuong-07	PC-Thuong-08	PC-Thuong-09	PC-Thuong-10
PC-Thuong-11	PC-Thuong-12	PC-Thuong-13	PC-Thuong-14	PC-Thuong-15
PC-Thuong-16	PC-Thuong-17	PC-Thuong-18	PC-Thuong-19	PC-Thuong-20
PC-Thuong-21	PC-Thuong-22	PC-Thuong-23	PC-Thuong-24	PC-Thuong-25
PC-VIP-01	PC-VIP-02	PC-VIP-03	PC-VIP-04	PC-VIP-05

Trạng thái máy

Tình trạng máy

Giải pháp

Nhà Bảo trì

Báo lỗi

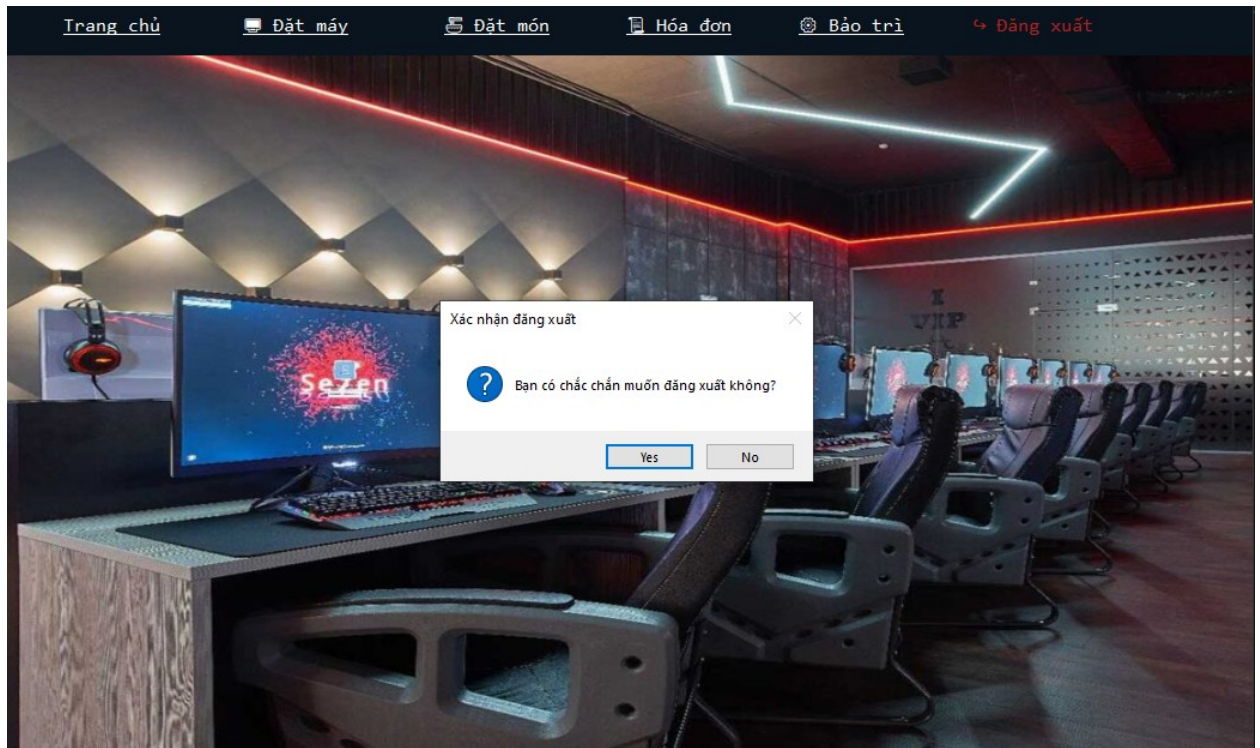
Danh sách bảo trì

Bộ phận	Mô tả	Chi phí	Trạng
---------	-------	---------	-------

Tổng chi phí bảo trì

Thanh toán

Hình 4.9. Giao diện Bảo trì



Hình 4.10. Giao diện Đăng xuất

4.2. XÂY DỰNG GIAO DIỆN FORM ĐẶT MÁY (COMPUTER)

4.2.1. Yêu cầu và kịch bản xử lý của từng chức năng trên form Quản lý máy

Khi khách đến thuê máy, màn hình điều khiển cho phép chọn mã phòng, sau đó hiển thị danh sách tất cả các máy: Sau khi người dùng chọn form Đặt máy, phía panel trên cùng sẽ hiện ra các phòng của quán. Người dùng sẽ ấn chọn vào phòng theo yêu cầu của khách, và giao diện sẽ hiển thị cấu hình của phòng, giá máy của phòng, các máy của phòng đó bao gồm cả trạng thái của máy: offline đối với máy chưa có khách sử dụng, online đối với máy đang được sử dụng. Số máy offline, số máy online. Những máy online thì không thể bật.

Số máy trong bảng Phòng được tự động tính khi thêm 1 máy vào trong phòng đó: Sau khi ấn vào nút bật máy cho khách, trạng thái của máy đó chuyển từ offline sang online, máy còn trống sẽ trừ đi 1, máy đang sử dụng sẽ cộng thêm 1.

Giờ vào và giờ ra được tính tự động trong hệ thống khi khách bắt đầu thuê và trả máy: Sau khi ấn vào nút bật máy, giờ vào sẽ là thời gian ngay lúc ấn nút bật, giờ ra sẽ là thời gian khi ấn nút tắt máy. Giờ ra trừ giờ vào sẽ được tổng thời gian sử dụng. Từ đó tính được số tiền khách phải trả đối với việc sử dụng máy.

Xem thông tin chi tiết của từng máy: Khi ấn vào từng máy, panel phía bên phải sẽ hiển thị thông tin chi tiết của máy đó, bao gồm tên máy, trạng thái máy, thời điểm bắt đầu, thời gian sử dụng, tiền tạm tính đối với việc sử dụng máy.

Tắt máy cho khách hàng: Khi khách không sử dụng máy nữa và có nhu cầu thanh toán, người dùng ấn chọn vào máy của khách, giao diện sẽ hiển thị tổng tiền tạm tính của hóa đơn đặt máy và khi người dùng ấn nút tắt máy thì thông tin hoá đơn tạm tính sẽ được lưu vào bảng hoá đơn cùng với danh sách gọi món mà người dùng đã sử dụng. Khi đó máy đó sẽ trở về trạng thái offline, mọi dịch vụ sử dụng được làm mới.

4.2.2. Demo chức năng trên giao diện

Giao diện khi ấn vào chức năng Đặt máy

Form1

Trang chủ Đặt máy Đặt món Hóa đơn Bảo trì Đăng xuất

Nhân viên: NV1

Phòng thường Phòng VIP

máy còn trống: máy đang thuê: Giá:

Cấu hình

Màn hình

Trạng thái

Thời điểm bắt đầu 25/05/2024 19:10

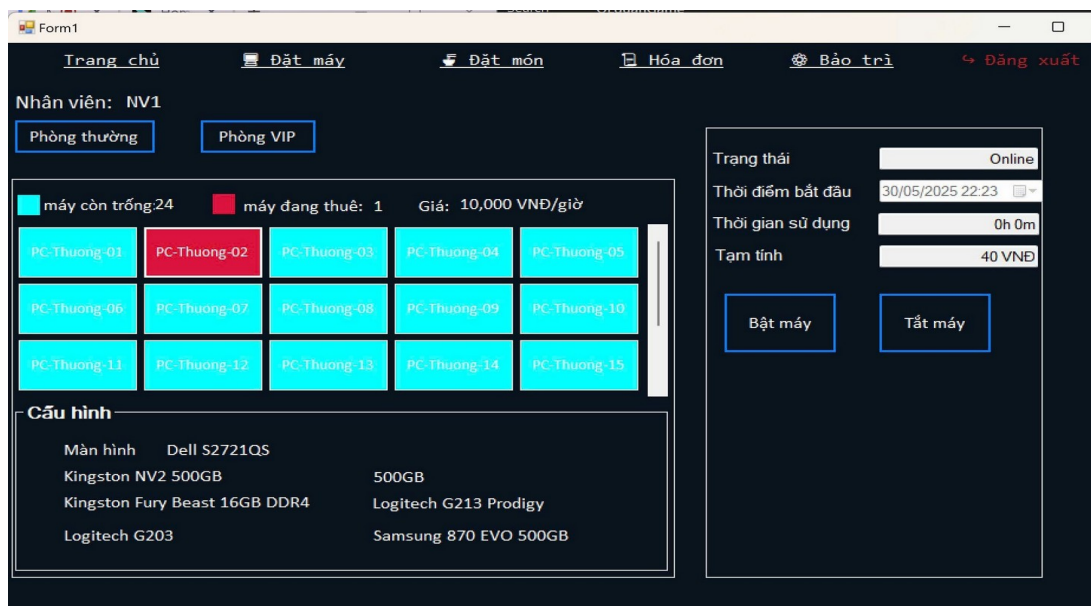
Thời gian sử dụng

Tạm tính

Bật máy Tắt máy

Hình 4.11. Hình. Giao diện khi ấn vào chức năng Đặt máy

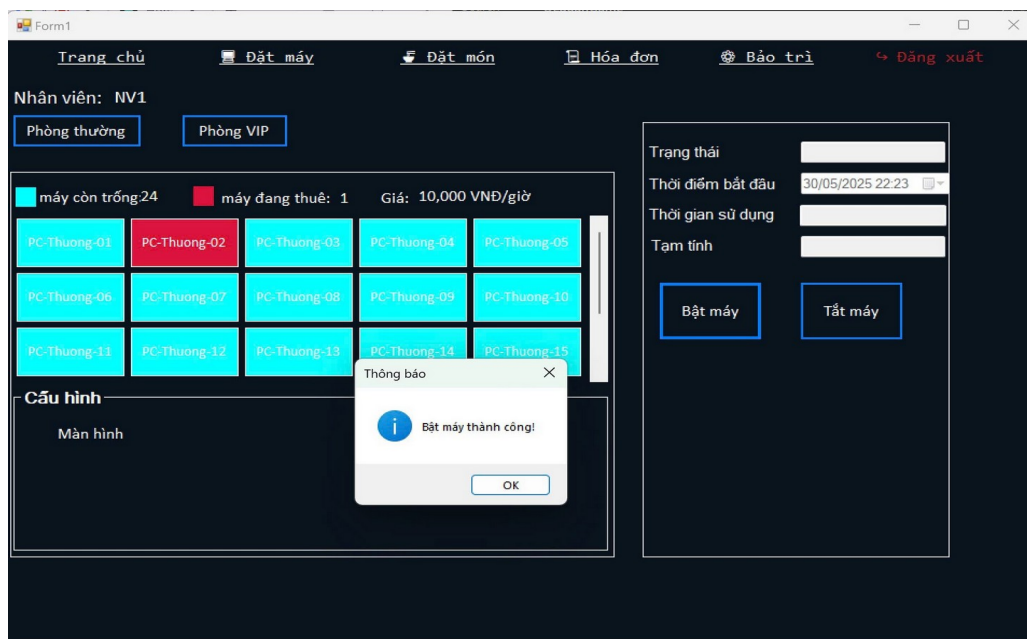
Khi ấn chọn phòng Thường. Giao diện hiển thị phòng thường bao gồm tất cả 25 máy cùng với giá và cấu hình của phòng thường (có thanh cuộn để cuộn xem được tất cả 25 máy). Hiện tại giao diện hiển thị 1 máy đang bật (màu đỏ) còn lại 24 máy chưa bật (màu xanh).



Hình 4.12. Giao diện khi thực hiện *Bật máy cho thuê*

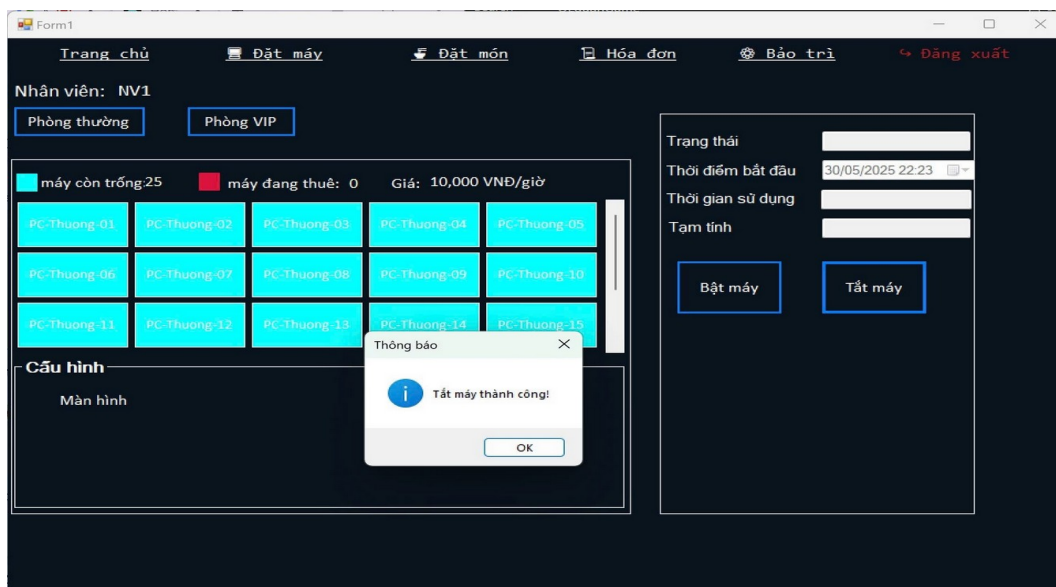
Khi ấn vào 1 máy bất kì (PC- Thuong-02) phía panel bên phải hiển thị chi tiết của Máy PC- Thuong-02, bao gồm trạng thái, thời điểm bắt đầu, thời gian sử dụng, tiền tạm tính theo giá phòng. Còn panel bên dưới sẽ hiển thị thông chi tiết về máy đó bao gồm cấu hình, bàn phím, tai nghe,...

Khi chọn 1 máy offline bất kỳ ấn nút bật máy thì giao diện hiển thị thông báo bật máy thành công, máy chuyển sang trạng thái online và màu của máy chuyển sang màu đỏ (ban đầu là xanh), khác biệt so với máy có trạng thái offline.



Hình 4.13. Giao diện thông báo bật máy thành công

Khi khách có nhu cầu thanh toán và không sử dụng máy nữa, người dùng chọn máy rồi ấn vào nút tắt máy để lưu lại toàn bộ hóa đơn thuê máy. Máy sau khi tắt máy sẽ thông báo tắt máy thành công và chuyển về trạng thái offline. Hóa đơn sẽ được lưu lại trong cơ sở dữ liệu của hệ thống.



Hình 4.14. Giao diện thông báo Tắt máy thành công

4.2.3. Code chức năng trên tầng giao diện

- Hiển thị thông tin từng phòng và máy của phòng

Hàm LoadRooms() có nhiệm vụ tải danh sách các phòng máy hiện có trong hệ thống và hiển thị chúng thông qua hai nút bấm (Button) tương ứng với mỗi phòng. Hàm gọi đến phương thức GetAllPhong() từ tầng DAL để lấy danh sách các đối tượng phòng (phongList) dưới dạng List<PhongDTO>. Sau khi lấy được danh sách phòng, hệ thống kiểm tra nếu danh sách không tồn tại hoặc có ít hơn 2 phòng, sẽ hiện thông báo lỗi và không tiếp tục xử lý.

Nếu danh sách hợp lệ: Gán mã phòng (MaPhong) vào thuộc tính Tag của btnZone1 và btnZone2, dùng để xác định phòng khi click. Gán tên phòng (TenPhong) làm nội dung hiển thị (Text) cho mỗi Button. Gán sự kiện Click của cả hai Button về cùng một sự kiện xử lý chung RoomButton_Click.

Quá trình thực thi được đặt trong khối try-catch để bắt lỗi và hiển thị thông báo nếu xảy ra lỗi trong quá trình tải dữ liệu.

```

private void LoadRooms()
{
    try
    {
        phongList = phongDAL.GetAllPhong();
        if (phongList == null || phongList.Count < 2)
        {
            MessageBox.Show("Không tìm thấy đủ phòng để hiển thị!",
"Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
        btnZone1.Tag = phongList[0].MaPhong;
        btnZone1.Text = phongList[0].TenPhong;
        btnZone1.Click += RoomButton_Click;

        btnZone2.Tag = phongList[1].MaPhong;
        btnZone2.Text = phongList[1].TenPhong;
        btnZone2.Click += RoomButton_Click;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Lỗi khi tải danh sách phòng: {ex.Message}",
"Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

Hàm RoomButton_Click(object sender, EventArgs e) được gọi khi người dùng nhấn vào một nút đại diện cho phòng máy (được tạo trong hàm LoadRooms). Mục đích của hàm là hiển thị các máy trong phòng được chọn, cùng với thông tin trạng thái và giá thuê.

```

private void RoomButton_Click(object sender, EventArgs e)
{
    try
    {
        Button clickedButton = sender as Button;
        if (clickedButton?.Tag == null)
        {
            MessageBox.Show("Nút phòng không hợp lệ!", "Lỗi",
MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
}

```

```

    }
    selectedRoomId = Convert.ToInt32(clickedButton.Tag);

    currentMachines = mayTinhDAL.GetMayTinhByPhong(selectedRoomId)
?? new List<MayTinhDTO>();
    DisplayMachines();

    PhongDTO selectedPhong = phongList.Find(p => p.MaPhong ==
selectedRoomId);
    txtPrice.Text = selectedPhong != null ?
"${selectedPhong.Giatheogio:N0} VNĐ/giờ" : "0 VNĐ/giờ";

    int offlineCount = currentMachines.Count(m =>
m.Trangthai.Trim().Equals("Trong", StringComparison.OrdinalIgnoreCase));
    int onlineCount = currentMachines.Count(m =>
m.Trangthai.Trim().Equals("Dang thue", StringComparison.OrdinalIgnoreCase));
    txtAvailable.Text = offlineCount.ToString();
    txtUsing.Text = onlineCount.ToString();

    ClearMachineDetails();
}
catch (Exception ex)
{
    MessageBox.Show($"Lỗi khi tải phòng: {ex.Message}", "Lỗi",
MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}

```

Hàm DisplayMachines() dùng để hiển thị danh sách các máy trạm thuộc phòng đã chọn lên giao diện thông qua FlowLayoutPanel. Trước tiên, hàm kiểm tra xem flpCom đã được khởi tạo hay chưa; nếu chưa, hiển thị thông báo lỗi và kết thúc hàm. Sau đó, toàn bộ các Button cũ trong flpCom sẽ được xóa để chuẩn bị hiển thị dữ liệu mới. Với mỗi máy trong danh sách currentMachines, hệ thống sẽ tạo một Button mới, hiển thị tên máy và gán đối tượng máy vào thuộc tính Tag. Button được thiết lập kích thước, màu nền và font chữ. Màu sắc được xác định theo trạng thái: nếu máy đang trống thì màu xanh da trời, nếu đang thuê thì màu đỏ, còn lại hiển thị màu xám. Mỗi Button cũng được gán sự kiện

Click để xử lý khi người dùng chọn máy. Cuối cùng, Button được thêm vào FlowLayoutPanel để hiển thị lên giao diện.

```
private void DisplayMachines()
{
    if (flpCom == null)
    {
        MessageBox.Show("Lỗi: flpCom không được khởi tạo!", "Lỗi",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }
    flpCom.Controls.Clear();
    foreach (var mayTinh in currentMachines ??
    Enumerable.Empty<MayTinhDTO>())
    {
        var machineButton = new Button
        {
            Text = mayTinh.TenMay,
            Tag = mayTinh,
            Width = 100,
            Height = 50,
            BackColor = mayTinh.Trangthai == "Trong" ? Color.Cyan :
mayTinh.Trangthai == "Dang thue" ? Color.Crimson : Color.Gray,
            ForeColor = Color.White,
            Font = new Font("Calibri", 10F),
            FlatStyle = FlatStyle.Flat
        };
        machineButton.Click += MachineButton_Click;
        flpCom.Controls.Add(machineButton);
    }
}
```

- **Hiển thị thông tin chi tiết của từng máy**

Hàm DisplayMachineDetails(MayTinhDTO mayTinh) được sử dụng để hiển thị chi tiết cấu hình và thông tin sử dụng của một máy trạm lên giao diện. Nếu đối tượng máy được truyền vào là null, hàm sẽ gọi ClearMachineDetails() để xóa thông tin và kết thúc. Ngược lại, hệ thống sẽ hiển thị trạng thái hoạt động của máy (Offline nếu "Trong",

Online nếu "Đang thuê") cùng các thông tin phần cứng như ổ cứng, dung lượng, RAM, ổ đĩa, màn hình, chuột, bàn phím thông qua các TextBox tương ứng. Nếu máy đang được thuê, hệ thống tiếp tục truy xuất thông tin phiên thuê hiện tại từ thuêMayDAL, bao gồm thời gian bắt đầu, tính thời lượng đã sử dụng và hiển thị số giờ – phút đã dùng. Dựa vào đơn giá theo giờ của phòng, hệ thống sẽ tính và hiển thị số tiền tạm tính. Nếu máy chưa thuê hoặc không có dữ liệu thuê, thời gian và số tiền tạm tính sẽ mặc định là 0. Tất cả các thao tác được đặt trong khối try-catch để đảm bảo an toàn và thông báo lỗi nếu có sự cố xảy ra khi xử lý dữ liệu.

```
private void DisplayMachineDetails(MayTinhDTO mayTinh)
{
    try
    {
        if (mayTinh == null)
        {
            ClearMachineDetails();
            return;
        }

        txtTT.Text = mayTinh.Trangthai == "Trong" ? "Offline" :
"Online";

        txtOcung.Text = mayTinh.TenOcung;
        txtDL.Text = mayTinh.TenDL;
        txtRam.Text = mayTinh.TenRAM;
        txtOdia.Text = mayTinh.TenOdia;
        txtManhinh.Text = mayTinh.TenMH;
        txtChuot.Text = mayTinh.TenChuot;
        txtBP.Text = mayTinh.TenBP;

        if (mayTinh.Trangthai == "Đang thuê")
        {
            ThueMayDTO thueMay =
thueMayDAL.GetCurrentThueMay(mayTinh.MaMay);
            if (thueMay != null)
            {
                tpStime.Value = thueMay.GioBD;
                TimeSpan duration = DateTime.Now - thueMay.GioBD;
                txtTime.Text = $"{duration.Hours}h {duration.Minutes}m";
            }
        }
    }
}
```

```

        PhongDTO phong = phongList.Find(p => p.MaPhong ==
mayTinh.MaPhong);
        txtTamtinh.Text = phong != null ?
$"{{{(decimal)duration.TotalHours * phong.Giatheogio}:N0} VNĐ" : "0 VNĐ";
    }
    else
    {
        tpStime.Value = DateTime.Now;
        txtTime.Text = "0h 0m";
        txtTamtinh.Text = "0 VNĐ";
    }
}
else
{
    tpStime.Value = DateTime.Now;
    txtTime.Text = "0h 0m";
    txtTamtinh.Text = "0 VNĐ";
}
}
catch (Exception ex)
{
    MessageBox.Show($"Lỗi khi hiển thị chi tiết máy: {ex.Message}",
"Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}
}

```

- **Bật máy cho khách**

Hàm btnBatmay_Click được gọi khi người dùng nhấn vào nút bật máy. Trước tiên, hàm kiểm tra xem đã chọn máy hay chưa; nếu chưa, hệ thống hiển thị cảnh báo yêu cầu chọn máy. Tiếp theo, nếu máy đang trong trạng thái "Dang thue" (đang được sử dụng) hoặc "Bao loi" (đang bảo trì), hệ thống sẽ thông báo và không cho phép bật máy. Nếu máy hợp lệ, hệ thống sẽ tạo một hóa đơn mới bằng cách gọi billingDAL.CreateHoaDon, sử dụng mã nhân viên hiện tại. Sau đó, khởi tạo phiên thuê máy qua thueMayDAL.StartThueMay và cập nhật trạng thái máy sang "Dang thue" trong cơ sở dữ liệu bằng mayTinhDAL.UpdateTrangThaiMay. Cuối cùng, hàm gọi lại RoomButton_Click để cập nhật lại danh sách máy và hiển thị thông báo "Bật máy thành

công". Mọi xử lý đều nằm trong khối try-catch để đảm bảo nếu có lỗi phát sinh sẽ được bắt và hiển thị thông báo phù hợp.

```
private void btnBatmay_Click(object sender, EventArgs e)
{
    try
    {
        if (selectedMachine == null)
        {
            MessageBox.Show("Vui lòng chọn một máy!", "Thông báo",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        if (selectedMachine.Trangthai == "Dang thue")
        {
            MessageBox.Show("Máy đang được sử dụng!", "Thông báo",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        if (selectedMachine.Trangthai == "Bao loi")
        {
            MessageBox.Show("Máy đang được bảo trì, không thể mở!",
            "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        int maHD = billingDAL.CreateHoaDon(currentMaNV);
        thueMayDAL.StartThueMay(selectedMachine.MaMay, maHD);
        mayTinhDAL.UpdateTrangThaiMay(selectedMachine.MaMay, "Dang
thue");

        RoomButton_Click(GetCurrentRoomButton(), null);
        MessageBox.Show("Bật máy thành công!", "Thông báo",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Lỗi khi bật máy: {ex.Message}", "Lỗi",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

```
}  
}
```

- Tắt máy cho khách

Hàm `btnTatmay_Click` được gọi khi người dùng nhấn nút tắt máy. Trước tiên, hệ thống kiểm tra xem đã chọn máy chưa (`selectedMachine`), nếu chưa thì hiển thị thông báo yêu cầu chọn máy. Sau đó, kiểm tra trạng thái của máy: nếu máy không đang trong trạng thái "Đang thuê" thì thông báo và dừng xử lý. Tiếp theo, hàm kiểm tra thông tin phiên thuê hiện tại bằng cách lấy dữ liệu từ `thueMayDAL.GetCurrentThueMay`. Nếu không tìm thấy thông tin thuê máy, hệ thống báo lỗi. Trong trường hợp hợp lệ, hàm gọi `EndThueMay` để kết thúc phiên thuê, đồng thời cập nhật trạng thái máy về mặc định thông qua stored procedure. Sau đó, hệ thống tính toán tổng số tiền thuê máy dựa trên thời gian sử dụng và đơn giá theo giờ của phòng, rồi cập nhật vào hóa đơn bằng `billingDAL.UpdateTongTien`. Cuối cùng, hàm gọi lại `RoomButton_Click` để làm mới giao diện, đồng thời hiển thị thông báo "Tắt máy thành công!". Mọi thao tác đều nằm trong khối try-catch để đảm bảo chương trình an toàn khi có lỗi xảy ra.

```
private void btnTatmay_Click(object sender, EventArgs e)
{
    ThueMayDTO thueMay =
thueMayDAL.GetCurrentThueMay(selectedMachine.MaMay);
    try
    {
        if (selectedMachine == null) { MessageBox.Show("Vui lòng chọn
một máy!", "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning); return; }

        if (selectedMachine.Trangthai != "Đang thuê")
        {
            MessageBox.Show("Máy không đang được sử dụng!", "Thông báo",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        if (thueMay == null)
```

```

        {
            MessageBox.Show("Không tìm thấy thông tin thuê máy. Vui lòng kiểm tra dữ liệu!", "Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        thueMayDAL.EndThueMay(thueMay.MaTM); // sp_EndThueMay đã cập nhật trạng thái máy

        PhongDTO phong = phongList.Find(p => p.MaPhong == selectedMachine.MaPhong);
        if (phong != null)
        {
            TimeSpan duration = DateTime.Now - thueMay.GioBD;
            decimal tongTien = (decimal)duration.TotalHours * phong.Giatheogio;
            billingDAL.UpdateTongTien(thueMay.MaHD, tongTien);
        }

        RoomButton_Click(GetCurrentRoomButton(), null);
        MessageBox.Show("Tắt máy thành công!", "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Lỗi khi tắt máy (MaTM = {thueMay?.MaTM}): {ex.Message}", "Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

4.2.4. Code trên tầng DAL

Hàm GetAllPhong() thuộc lớp PhongDAL có nhiệm vụ truy xuất toàn bộ danh sách phòng máy từ cơ sở dữ liệu. Hàm thực thi stored procedure sp_GetAllPhong thông qua DataProvider.Instance.ExecuteQuery, và kết quả trả về là một DataTable. Mỗi dòng dữ liệu trong bảng này được ánh xạ thành một đối tượng PhongDTO với các thuộc tính gồm: MaPhong, TenPhong, SoMay, và Giatheogio. Các đối tượng này sau đó được thêm vào danh sách phongList và trả về cho tầng giao diện sử dụng. Trong quá trình xử lý,

hàm có cơ chế kiểm tra dữ liệu null trước khi gán để tránh lỗi khi chuyển đổi kiểu. Nếu có lỗi xảy ra trong quá trình truy vấn hoặc xử lý dữ liệu, các khối try-catch sẽ bắt lỗi SQL hoặc lỗi hệ thống và ném ngoại lệ với thông báo rõ ràng. Đây là hàm nền tảng để hiển thị danh sách phòng trên giao diện và phục vụ các thao tác khác liên quan đến quản lý phòng.

- Hiển thị danh sách máy theo phòng

```
public List<PhongDTO> GetAllPhong()
{
    List<PhongDTO> phongList = new List<PhongDTO>();
    try
    {
        DataTable dt =
        DataProvider.Instance.ExecuteQuery("sp_GetAllPhong", null,
        CommandType.StoredProcedure);
        foreach (DataRow row in dt.Rows)
        {
            PhongDTO phong = new PhongDTO
            {
                MaPhong = row["MaPhong"] != DBNull.Value ?
                Convert.ToInt32(row["MaPhong"]) : 0,
                TenPhong = row["TenPhong"]?.ToString() ?? "",
                SoMay = row["SoMay"] != DBNull.Value ?
                Convert.ToInt32(row["SoMay"]) : 0,
                Giatheogio = row["Giatheogio"] != DBNull.Value ?
                Convert.ToDecimal(row["Giatheogio"]) : 0
            };
            phongList.Add(phong);
        }
        return phongList;
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine($"Lỗi SQL trong PhongDAL.GetAllPhong:
        {sqlEx.Message}\n{sqlEx.StackTrace}");
        throw new Exception("Không thể lấy danh sách phòng", sqlEx);
    }
    catch (Exception ex)
    {
    }
```

```

        Console.WriteLine($"Lỗi trong PhongDAL.GetAllPhong:
{ex.Message}\n{ex.StackTrace}");
        throw new Exception("Không thể lấy danh sách phòng", ex);
    }
}

```

Hàm GetMayTinhByPhong nhận vào mã phòng và gọi một thủ tục lưu trữ trong cơ sở dữ liệu để lấy danh sách các máy tính thuộc phòng đó, sau đó chuyển từng dòng dữ liệu thành đối tượng MayTinhDTO và trả về danh sách các đối tượng này. Phương thức thứ hai là UpdateTrangThaiMay, dùng để cập nhật trạng thái của một máy tính cụ thể dựa vào mã máy, bằng cách thực thi một câu lệnh SQL cập nhật trực tiếp vào bảng Maytinh. Phương thức này không trả về kết quả mà chỉ thực hiện thay đổi trên cơ sở dữ liệu.

```

public List<MayTinhDTO> GetMayTinhByPhong(int maPhong)
{
    List<MayTinhDTO> list = new List<MayTinhDTO>();
    var parameters = new (string, object)[]
    {
        ("@MaPhong", maPhong)
    };

    DataTable dt =
DataProvider.Instance.ExecuteQuery("sp_GetMayTinhByPhong", parameters,
CommandType.StoredProcedure);
    foreach (DataRow row in dt.Rows)
    {
        list.Add(new MayTinhDTO(row));
    }
    return list;
}

public void UpdateTrangThaiMay(int maMay, string trangThai)
{
    DataProvider.Instance.ExecuteNonQuery(
        "UPDATE Maytinh SET Trangthai = @TrangThai WHERE MaMay =
@MaMay",
        new (string, object)[] { ("@TrangThai", trangThai), ("@MaMay",
maMay) }
    );
}

```

```
}
```

Phương thức StartThueMay nhận vào mã máy (maMay) và mã hóa đơn (maHD), gọi thủ tục lưu trữ sp_StartThueMay trong cơ sở dữ liệu để bắt đầu phiên thuê máy. Trong đó có một tham số đầu ra (@MaTM) dự kiến nhận mã phiên thuê mới nhưng chưa được xử lý trong code.

Phương thức EndThueMay nhận mã phiên thuê máy (maTM), gọi thủ tục sp_EndThueMay để kết thúc phiên thuê tương ứng. Nếu có lỗi xảy ra trong quá trình gọi, phương thức sẽ bắt và ném lại ngoại lệ kèm thông báo chi tiết.

```
public void StartThueMay(int maMay, int maHD)
{
    Dictionary<string, object> outputParams;
    DataProvider.Instance.ExecuteStoredProcedure(
        "sp_StartThueMay",
        out outputParams,
        ("@MaMay", maMay),
        ("@MaHD", maHD),
        ("@MaTM", null) // OUTPUT
    );
}

public void EndThueMay(int maTM)
{
    try
    {
        DataProvider.Instance.ExecuteNonQuery(
            "sp_EndThueMay",
            new (string, object)[] { ("@MaTM", maTM) },
            CommandType.StoredProcedure
        );
    }
    catch (Exception ex)
    {
        throw new Exception($"Lỗi khi kết thúc phiên thuê máy (MaTM = {maTM}): {ex.Message}", ex);
    }
}
```

Phương thức LoadBillList để lấy danh sách hóa đơn theo khoảng thời gian bắt đầu (ngayBD), kết thúc (ngayKT), và tùy chọn lọc thêm theo tên nhân viên (tenNV) hoặc loại hóa đơn (loaiHD). Phương thức gọi thủ tục lưu trữ sp_GetBillListFiltered với các tham số truyền vào, nhận về một bảng dữ liệu (DataTable). Nếu không có kết quả, nó trả về danh sách rỗng và ghi log thông báo không tìm thấy hóa đơn. Ngược lại, nó duyệt từng dòng dữ liệu, khởi tạo đối tượng Hoadon rồi thêm vào danh sách kết quả. Phương thức cũng xử lý ngoại lệ, ghi lại lỗi chi tiết nếu có sự cố khi truy vấn hoặc xử lý dữ liệu, rồi ném lại ngoại lệ tổng quát kèm thông báo phù hợp.

4.2.5. Code trên tầng DTO

Đối tượng Phong: lớp PhongDTO đại diện cho phòng máy tính có các thuộc tính như mã phòng (MaPhong), tên phòng (TenPhong), số lượng máy (SoMay) và giá theo giờ (Giatheogio), được định nghĩa với các thuộc tính có thể đọc và ghi (get; set;)

```
public class PhongDTO
{
    public int MaPhong { get; set; }
    public string TenPhong { get; set; }
    public int SoMay { get; set; }
    public decimal Giatheogio { get; set; }
}
```

Lớp ThueMayDTO đại diện cho một phiên thuê máy, với các thuộc tính chính bao gồm: mã phiên thuê (MaTM), mã máy tính (MaMay), thời gian bắt đầu thuê (GioBD), thời gian kết thúc thuê (GioKT, có thể null nếu chưa kết thúc), và mã hóa đơn liên quan (MaHD). Các thuộc tính này có getter và setter để dễ dàng truy cập và cập nhật dữ liệu trong quá trình quản lý thuê máy.

```
public class ThueMayDTO {
    public int MaTM { get; set; }
    public int MaMay { get; set; }
    public DateTime GioBD { get; set; }
    public DateTime? GioKT { get; set; }
    public int MaHD { get; set; } }
```

}

4.3. XÂY DỰNG GIAO DIỆN FORM ĐẶT MÓN

4.3.1. Yêu cầu và kịch bản xử lý của từng chức năng form đặt món

Mô tả chức năng của màn hình điều khiển Form DatMon trong ứng dụng QLQuanGame

Màn hình form DatMon trong ứng dụng quản lý quán game (QLQuanGame) được thiết kế để quản lý và hiển thị danh sách các món ăn, đồ uống, và các dịch vụ đi kèm tại quán game.

Giao diện của form DatMon thân thiện với người dùng, cho phép dễ dàng theo dõi danh sách món ăn. Mỗi món được hiển thị kèm theo hình ảnh minh họa, tên món, và giá bán, giúp người dùng nhanh chóng nắm bắt thông tin cơ bản.

Các danh mục món ăn được tổ chức dưới dạng các nút bấm, giúp người dùng dễ dàng chọn lọc và xem danh sách món thuộc một danh mục cụ thể (ví dụ: đồ ăn, đồ uống, snack).

Tính năng tìm kiếm được tích hợp với một thanh tìm kiếm, cho phép người dùng nhập tên món để nhanh chóng tìm thấy các món ăn cần thiết.

Người quản lý có thể thêm mới danh mục món ăn, giúp cập nhật và mở rộng danh sách danh mục theo nhu cầu kinh doanh của quán.

Hệ thống hỗ trợ các thao tác như thêm, chỉnh sửa, và xóa món ăn, giúp người quản lý dễ dàng duy trì và cập nhật dữ liệu món ăn trong cơ sở dữ liệu.

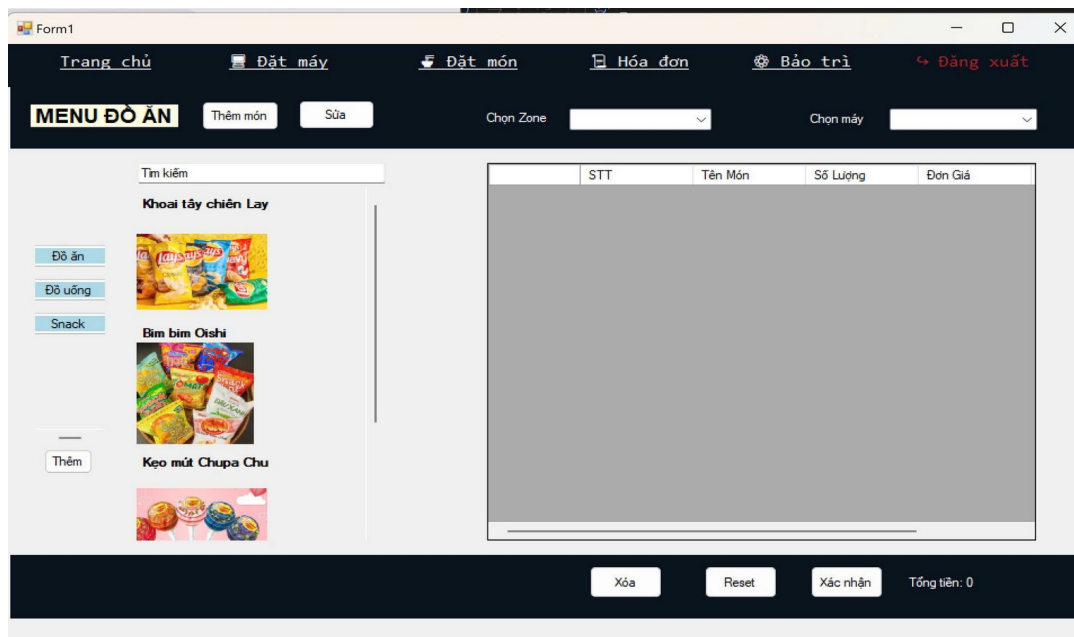
Người dùng có thể chọn khu vực (phòng máy) và máy tính cụ thể để xem danh sách các món ăn đã được khách hàng tại máy đó gọi.

Bảng dgvGiohang hiển thị chi tiết các món ăn đã chọn, bao gồm thông tin về số lượng, đơn giá, và tổng chi phí của từng món, giúp người dùng theo dõi rõ ràng các đơn hàng.

Tổng số tiền của các món ăn đã chọn được tự động tính toán và hiển thị, hỗ trợ người dùng kiểm tra chi phí trước khi xác nhận.

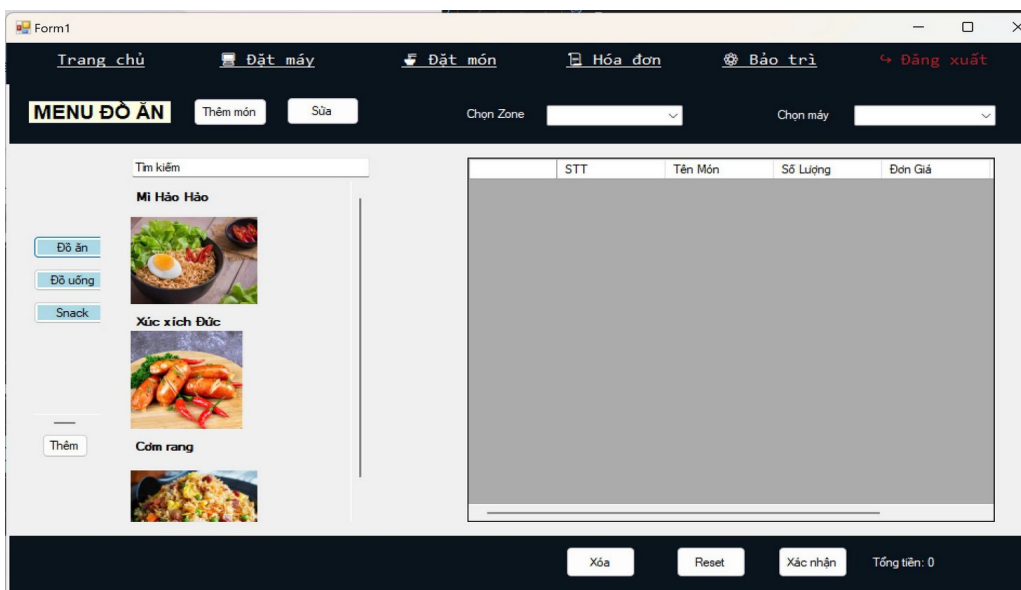
Sau khi xác nhận đơn hàng, thông tin các món ăn đã chọn sẽ được lưu vào cơ sở dữ liệu, phục vụ cho việc tính tiền và ghi nhận lịch sử đơn hàng, mang lại trải nghiệm quản lý và thanh toán thuận tiện, hiệu quả cho cả nhân viên và khách hàng.

4.3.2. Demo chức năng trên giao diện



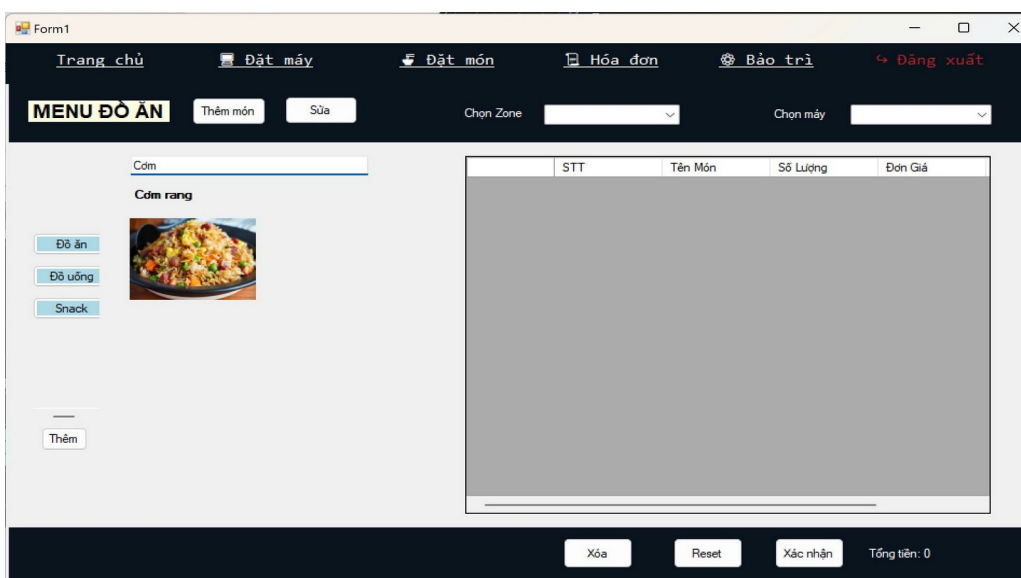
Hình 4.15. Giao diện màn hình đặt món

Giao diện hiển thị Menu các món ăn và các chức năng bao gồm: thêm danh mục món ăn, thêm món ăn mới vào menu, sửa món ăn, chọn phòng và máy, tìm kiếm, DataGridView để hiển thị các món ăn đã chọn, thêm hoặc xóa món ăn cho máy đang dùng.



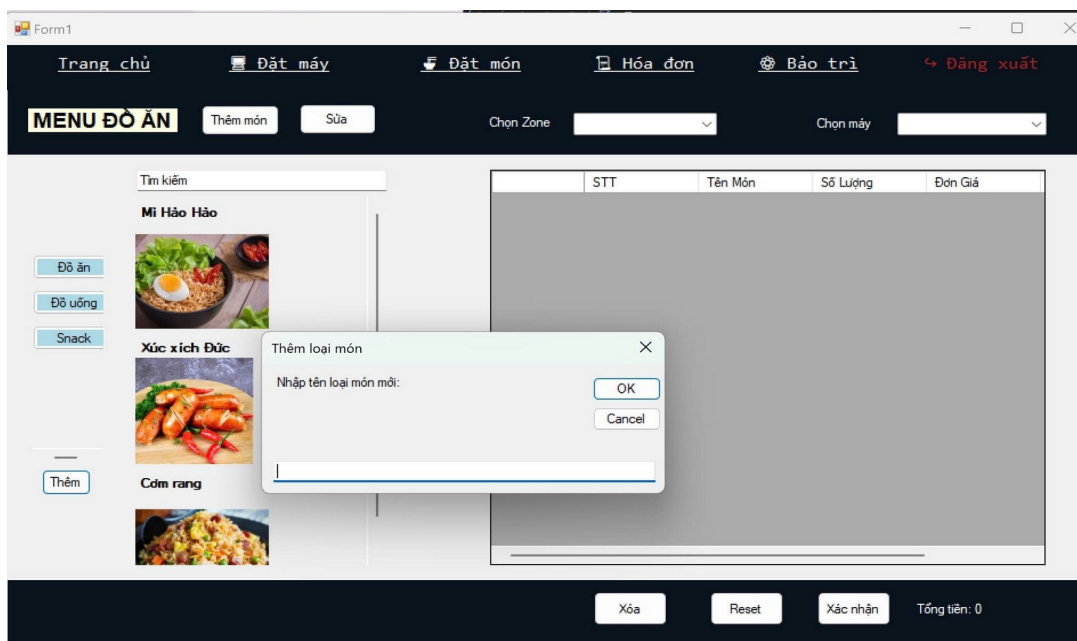
Hình 4.16. Giao diện tìm kiếm theo danh mục món ăn

Danh sách các món ăn được lọc để chỉ hiển thị các món có danh mục là “Đồ ăn”



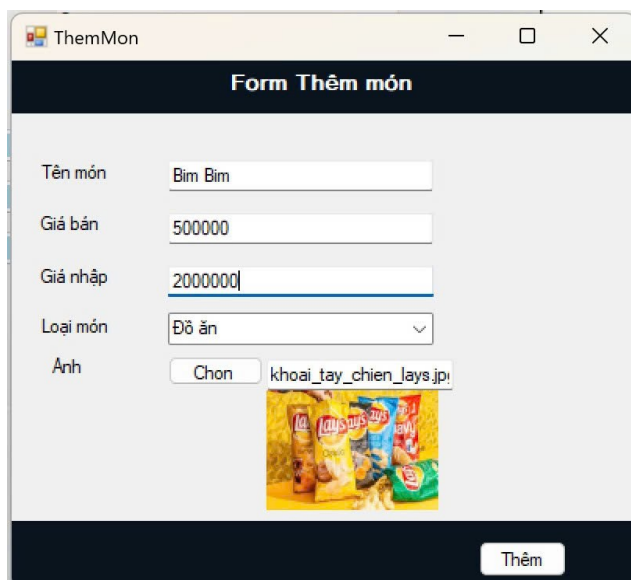
Hình 4.17. Giao diện tìm kiếm theo thanh tìm kiếm

Danh sách các món ăn được lọc để chỉ hiển thị các món có tên là “Cơm”



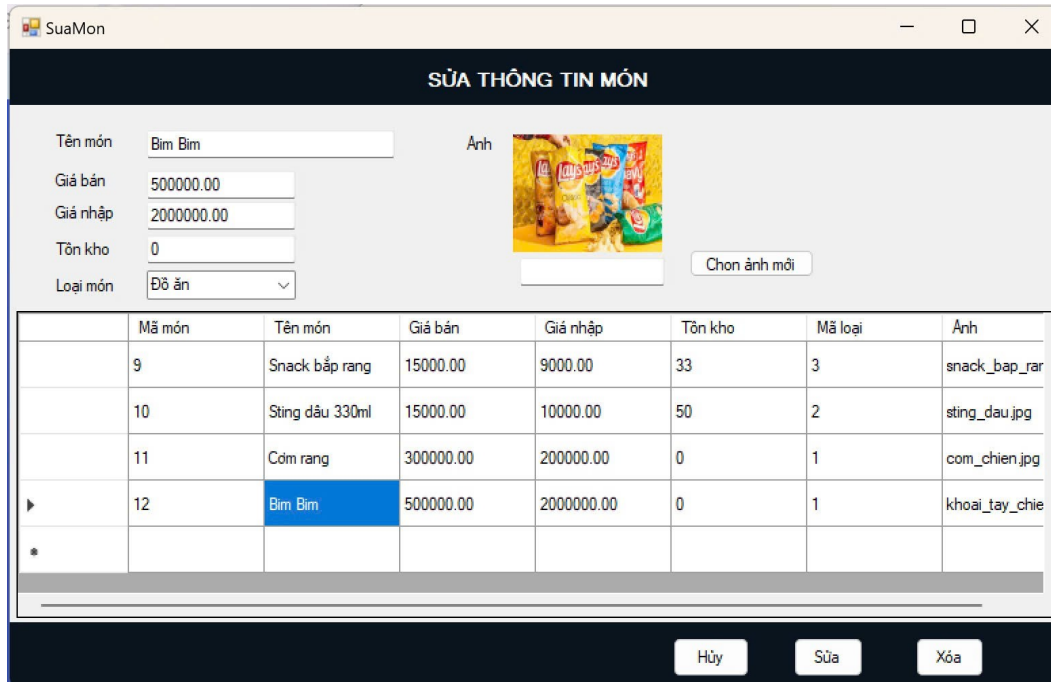
Hình 4.18. Giao diện thêm danh mục đồ ăn

Người dùng nhập tên danh mục đồ ăn mới và ấn OK, hệ thống sẽ kiểm tra trùng loại nếu hợp lý, tên danh mục sẽ được thêm vào cơ sở dữ liệu và hiển thị trên giao diện.



Hình 4.19. Giao diện thêm món ăn

Người dùng nhập đầy đủ thông tin đồ ăn mới bao gồm chọn ảnh, tên, giá bán, giá nhập, loại món của món ăn và ấn thêm, hệ thống sẽ kiểm tra, nếu hợp lý, món ăn sẽ được thêm vào cơ sở dữ liệu và hiển thị trên giao diện.



SuaMon

SỬA THÔNG TIN MÓN

Tên món: Ảnh: 

Giá bán:

Giá nhập:

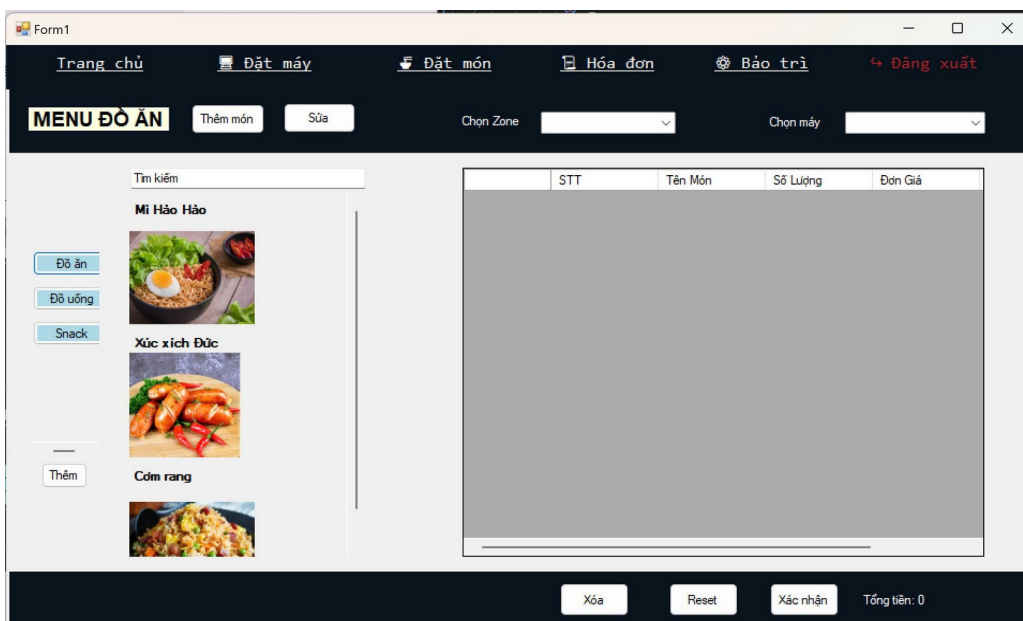
Tồn kho:

Loại món:

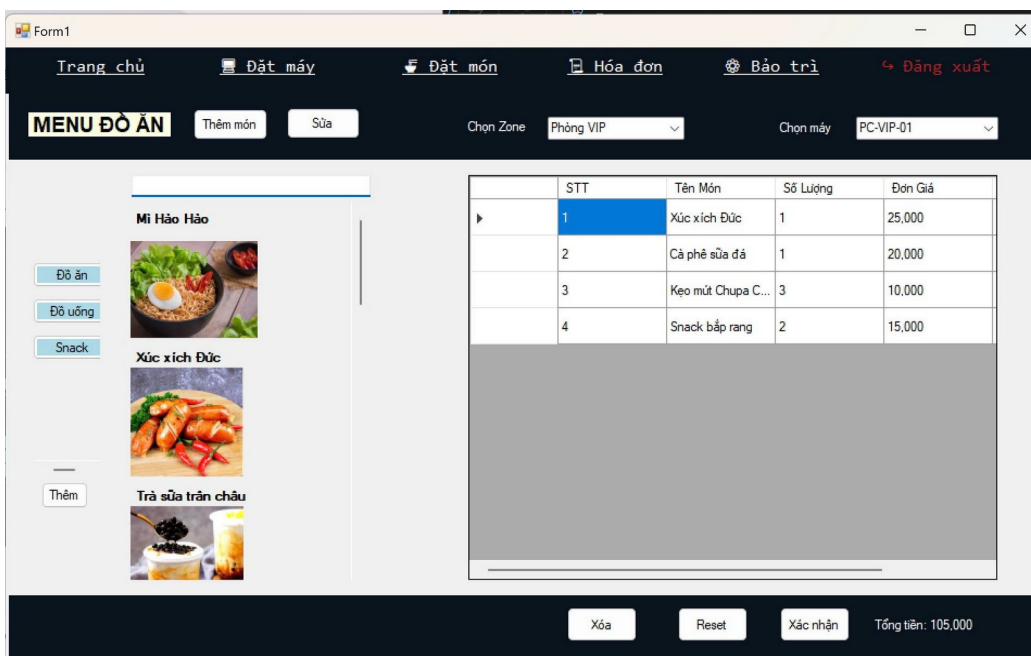
	Mã món	Tên món	Giá bán	Giá nhập	Tồn kho	Mã loại	Ảnh
	9	Snack bắp rang	15000.00	9000.00	33	3	snack_bap_rar
	10	Sting dầu 330ml	15000.00	10000.00	50	2	sting_dau.jpg
	11	Cơm rang	300000.00	200000.00	0	1	com_chien.jpg
▶	12	Bim Bim	500000.00	2000000.00	0	1	khoai_tay_chie
*							

Hình 4.20. Giao diện sửa món ăn

Hiển thị danh sách các món ăn, khi người dùng muốn sửa hoặc xóa món ăn, chọn vào dòng của món ăn đó, thông tin món ăn sẽ hiển thị để cho người dùng thay đổi, hệ thống kiểm tra, nếu hợp lý, món ăn sẽ được sửa hoặc xóa.



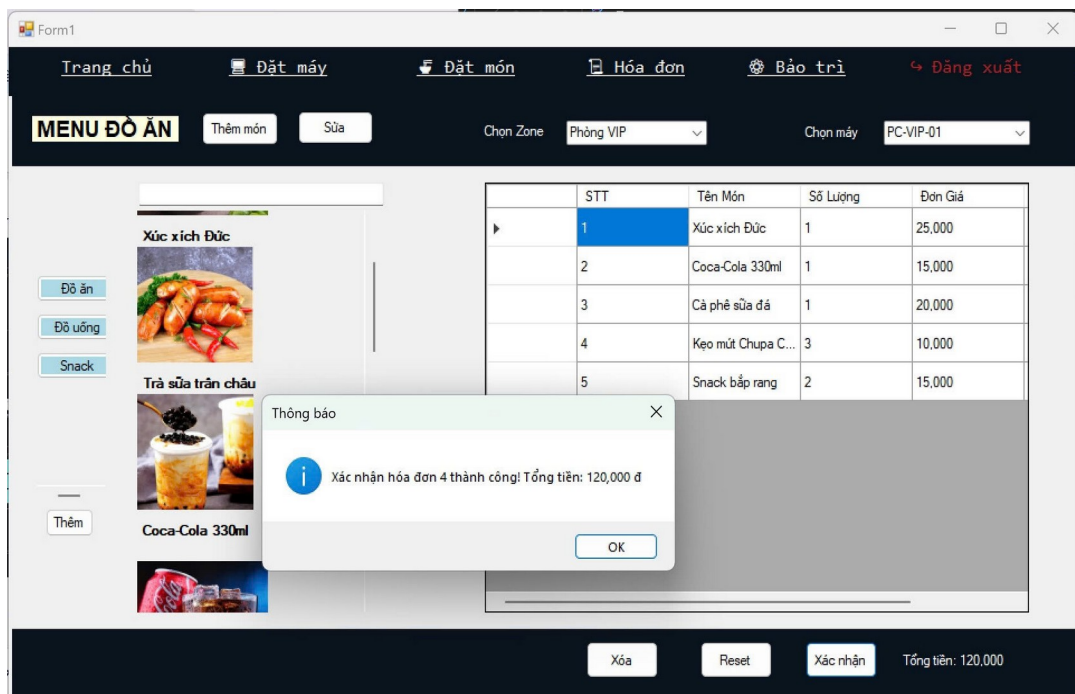
Hình 4.21. Giao diện khi chưa chọn máy



Hình 4.22. Giao diện thêm món ăn vào danh sách các món ăn

Sau khi chọn máy, bảng sẽ hiển thị những món ăn mà máy đã đặt trước đó nếu có, người dùng có thể thêm món ăn bằng cách chọn món ăn trong menu, hệ thống sẽ ghi

nhận và thêm vào trong bảng, nút reset được sử dụng khi người dùng chọn sai món hoặc muốn thay đổi, nút xóa để xóa 1 món bất kì



Hình 4.23. Thêm món ăn thành công

Xác nhận các món ăn người dùng thêm vào máy, hệ thống kiểm tra, nếu hợp lý, các món ăn sẽ được thêm, xóa vào hóa đơn của máy đó.

4.3.3. Code trên GUI

- Hiển thị Menu món ăn

Mỗi món ăn trong menu là một ucMon - điều khiển người dùng (User Control), được sử dụng để hiển thị thông tin về các món ăn trong menu giúp lấy dữ liệu từ cơ sở dữ liệu và hiển thị dưới dạng hình ảnh và thông tin liên quan như tên, giá.



Hình 4.24. UcMon

Hàm LoadMenu() hiển thị toàn bộ món ăn lên FlowLayoutPanel (flpMonAn). Gọi MonDAO.LayTatCaMon() để lấy danh sách món từ cơ sở dữ liệu. Xóa các điều khiển cũ, tạo ucMon cho mỗi món (mã, tên, giá, ảnh). Gắn sự kiện Click để xử lý chọn món. Hiển thị lỗi nếu truy vấn thất bại.

```
private void LoadMenu()
{
    try
    {
        List<Mon> monList = MonDAO.Instance.LayTatCaMon();
        flpMonAn.Controls.Clear();

        foreach (var mon in monList)
        {
            ucMon item = new ucMon(mon.MaMon, mon.TenMon, mon.Gia,
mon.HinhAnh);
            item.Click += UcMon_Click;
            flpMonAn.Controls.Add(item);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Lỗi khi tải menu: {ex.Message}", "Lỗi",
MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

```
}
```

- Thêm đồ ăn cho máy được chọn

Khi sự kiện `UcMon_Click` xảy ra, hệ thống sẽ kiểm tra xem người dùng đã chọn phòng (`cboZone`) và máy (`cboChonmay`) hay chưa. Sau đó, thông tin món ăn (`maMon`, `tenMon`, `gia`) được lấy từ user control `ucMon`. Hệ thống thực hiện truy vấn để lấy mã hóa đơn (`maHD`) của máy đang thuê và gọi hàm `AddMonToGrid` để thêm món vào giỏ hàng (`orderTable`). Nếu món ăn đã tồn tại trong giỏ, số lượng sẽ được tăng lên; nếu chưa có, một hàng mới sẽ được thêm vào và cập nhật số thứ tự (STT). Cuối cùng, hệ thống làm mới `dgvGioHang` và gọi `UpdateTotal` để cập nhật tổng tiền đơn hàng.

```
private void UcMon_Click(object sender, EventArgs e)
{
    // Kiểm tra đã chọn phòng và máy chưa
    if (cboZone.SelectedIndex == -1 || cboChonmay.SelectedIndex == -1 ||
        cboChonmay.SelectedValue == null || cboZone.SelectedValue == null)
    {
        MessageBox.Show("Vui lòng chọn máy và phòng trước.", "Thông báo",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        Console.WriteLine("UcMon_Click: Chưa chọn phòng hoặc máy");
        return;
    }

    ucMon uc = sender as ucMon;
    if (uc == null)
    {
        Console.WriteLine("UcMon_Click: ucMon không hợp lệ");
        return;
    }

    int maMon = uc.MaMon;
    string tenMon = uc.TenMon;
    decimal gia = uc.Gia;

    int maMay, maPhong;
    try
    {
        maMay = Convert.ToInt32(cboChonmay.SelectedValue);
```



```

        maPhong = Convert.ToInt32(cboZone.SelectedValue);
        Console.WriteLine($"UcMon_Click: Đã chọn máy MaMay = {maMay},
phòng MaPhong = {maPhong}");
    }
    catch (Exception ex)
    {
        MessageBox.Show("Máy hoặc phòng không hợp lệ, vui lòng chọn
lại.", "Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
        Console.WriteLine($"UcMon_Click: Lỗi chuyển đổi
cboChonmay.SelectedValue hoặc cboZone.SelectedValue: {ex.Message}");
        return;
    }

    try
    {
        // Kiểm tra hóa đơn hiện tại từ Thuemay
        if (!maHoaDonTheoMay.ContainsKey(maMay))
        {
            string query = @"
SELECT TOP 1 t.MaHD
FROM Thuemay t
JOIN Maytinh m ON t.MaMay = m.MaMay
WHERE t.MaMay = @maMay
AND m.MaPhong = @maPhong
AND m.Trangthai = 'Dang thue'
AND t.GioKT IS NULL
ORDER BY t.GioBD DESC";
            object result = DataProvider.Instance.ExecuteScalar(query,
new[] { ("@maMay", (object)maMay), ("@maPhong", (object)maPhong) });
            if (result == null || result == DBNull.Value)
            {
                MessageBox.Show("Máy này không đang thuê hoặc thông tin
phòng/máy không khớp, không thể gọi món.", "Thông báo", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
                Console.WriteLine($"UcMon_Click: Không tìm thấy MaHD cho
MaMay = {maMay}, MaPhong = {maPhong}");
                return;
            }
            maHoaDonTheoMay[maMay] = Convert.ToInt32(result);
        }

        // Gọi AddMonToGrid để thêm món

```

```

        AddMonToGrid(maMon, tenMon, gia);
        Console.WriteLine($"UcMon_Click: Đã gọi AddMonToGrid cho món
{tenMon} (MaMon: {maMon})");
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Lỗi khi thêm món: {ex.Message}", "Lỗi",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
        Console.WriteLine($"Lỗi trong UcMon_Click: {ex.Message}\n
{ex.StackTrace}");
    }
}

```

- Xóa món

Khi người dùng thực hiện thao tác xóa, hệ thống sẽ kiểm tra xem đã chọn một hàng trong dgvGioHang hay chưa. Nếu có, hộp thoại xác nhận sẽ được hiển thị để đảm bảo người dùng thực sự muốn xóa món. Nếu người dùng đồng ý, hệ thống sẽ xóa hàng được chọn khỏi DataGridView và gọi hàm UpdateTotal để cập nhật lại tổng tiền.

```

private void btnXoamon_Click(object sender, EventArgs e)
{
    if (dgvGioHang.CurrentRow != null)
    {
        if (MessageBox.Show("Bạn có chắc chắn muốn xóa món này?", "Xác nhận
xóa", MessageBoxButtons.YesNo, MessageBoxIcon.Question) == DialogResult.Yes)
        {
            dgvGioHang.Rows.RemoveAt(dgvGioHang.CurrentRow.Index);
            UpdateTotal();
        }
    }
    else
    {
        MessageBox.Show("Vui lòng chọn một món để xóa!", "Thông báo",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}

```

Hệ thống kiểm tra xem người dùng đã chọn phòng và máy chưa, sau đó lấy mã hóa đơn (maHD) bằng GetCurrentMaHD. Các món cũ trong bảng GoiMon liên quan đến

maHD sẽ bị xóa. Với mỗi món trong orderTable, hệ thống lấy mã món, kiểm tra tồn kho, rồi gọi stored procedure sp_XacNhanGoiMon để thêm món mới. Sau đó cập nhật tồn kho, tổng tiền hóa đơn và làm mới danh sách món đã gọi. Cuối cùng, hiển thị thông báo xác nhận thành công hoặc báo lỗi nếu có

```
private void btnXacnhan_Click(object sender, EventArgs e)
{
    if (cboChonmay.SelectedIndex == -1 || cboZone.SelectedIndex == -1)
    {
        MessageBox.Show("Vui lòng chọn máy và phòng trước khi xác nhận.", "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    int maMay = Convert.ToInt32(cboChonmay.SelectedValue);
    int maPhong = Convert.ToInt32(cboZone.SelectedValue);

    try
    {
        int? maHD = GetCurrentMaHD(maMay, maPhong);
        if (!maHD.HasValue)
        {
            MessageBox.Show("Không có hóa đơn thuê máy nào cho máy và phòng này.", "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        // XÓA CÁC MÓN CŨ TRƯỚC
        DataProvider.Instance.ExecuteNonQuery("DELETE FROM GoiMon WHERE MaHD = @maHD", new[] { ("@maHD", (object)maHD.Value) });

        foreach (DataRow row in orderTable.Rows)
        {
            string tenMon = row["TenMon"].ToString();
            int soLuong = Convert.ToInt32(row["SoLuong"]);

            int maMon =
            Convert.ToInt32(DataProvider.Instance.ExecuteScalar("SELECT MaMon FROM Mon WHERE TenMon = @tenMon", new[] { ("@tenMon", (object)tenMon) }));
            if (maMon == 0)
```

```

        {
            MessageBox.Show($"Không tìm thấy món {tenMon} trong cơ sở dữ liệu!", "Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        int tonKho =
(int)DataProvider.Instance.ExecuteScalar("SELECT TonKho FROM Mon WHERE MaMon = @MaMon", new[] { ("@MaMon", (object)maMon) });
        if (soLuong > tonKho)
        {
            MessageBox.Show($"Không đủ tồn kho cho món {tenMon}!", "Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        using (SqlConnection conn =
DataProvider.Instance.GetConnection())
        {
            conn.Open();
            using (SqlCommand cmd = new
SqlCommand("sp_XacNhanGoiMon", conn))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@MaHD", maHD.Value);
                cmd.Parameters.AddWithValue("@MaMon", maMon);
                cmd.Parameters.AddWithValue("@SoLuong", soLuong);
                SqlParameter resultParam = new
SqlParameter("@Result", SqlDbType.Int) { Direction =
ParameterDirection.Output };
                cmd.Parameters.Add(resultParam);

                cmd.ExecuteNonQuery();
                if (Convert.ToInt32(resultParam.Value) != 1)
                {
                    MessageBox.Show($"Lỗi khi thêm món {tenMon} vào hóa đơn!", "Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
                    return;
                }
            }

            // Cập nhật tồn kho

```

```

DataProvider.Instance.ExecuteNonQuery("UPDATE Mon SET TonKho = TonKho - @SoLuong
WHERE MaMon = @MaMon",
                                new[] { ("@SoLuong", soLuong), ("@MaMon",
(object)maMon) });
        }
    }

    decimal tongTien =
GoiMonDAO.Instance.CalculateTotalByMaHD(maHD.Value);

    // Cập nhật tổng tiền vào bảng hóa đơn
    DataProvider.Instance.ExecuteNonQuery(
        "UPDATE HoaDon SET TongTien = @TongTien WHERE MaHD = @MaHD",
        new[] { ("@TongTien", tongTien), ("@MaHD",
(object)maHD.Value) }
    );

    // Làm mới lại hiển thị
    LoadGoiMonTheoHoaDon(maHD.Value);

    MessageBox.Show($"Xác nhận hóa đơn {maHD.Value} thành công! Tổng
tiền: {tongTien:N0} đ", "Thông báo", MessageBoxButtons.OK,
MessageBoxIcon.Information);
}
catch (Exception ex)
{
    MessageBox.Show($"Lỗi khi xác nhận hóa đơn: {ex.Message}",
"Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}

```

Hàm UpdateTotal được gọi trong các hàm trên để cập nhật tổng tiền hiển thị trên giao diện.

```

private void UpdateTotal()
{
    decimal total = 0;
    foreach (DataRow row in orderTable.Rows)
    {
        total += Convert.ToDecimal(row["ThanhTien"]);
    }
}

```

```

        lblTongtien.Text = $"Tổng tiền: {total:N0}";
        lblthanhtien.Text = total.ToString("N0");
    }

```

Hàm UpdateTotal được gọi trong các hàm trên để cập nhật tổng tiền hiển thị trên giao diện.

- Thêm món ăn mới

Chức năng thêm món ăn mới trong FormThemMon cho phép người dùng nhập thông tin món ăn (tên, giá bán, giá nhập, danh mục, ảnh) và lưu vào cơ sở dữ liệu. Khi form tải, danh sách danh mục được lấy từ bảng LoaiMon để hiển thị trong ComboBox. Nút "Chọn ảnh" mở hộp thoại để chọn file ảnh, sao chép vào thư mục Resources và hiển thị trên giao diện. Nút "Xác nhận" kiểm tra dữ liệu đầu vào, gọi stored procedure sp_ThemMon để thêm món vào bảng Mon, và đóng form nếu thành công. Xử lý lỗi được thực hiện để thông báo khi dữ liệu không hợp lệ hoặc có vấn đề với cơ sở dữ liệu.

```

private void btnXacnhanthem_Click(object sender, EventArgs e)
{
    string tenMon = txtTenMon.Text.Trim();
    if (string.IsNullOrEmpty(tenMon))
    {
        MessageBox.Show("Tên món không được để trống!", "Thông báo",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (!decimal.TryParse(txtGiaBan.Text, out decimal gia) || gia < 0)
    {
        MessageBox.Show("Giá bán không hợp lệ!", "Thông báo",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (!decimal.TryParse(txtGiaNhap.Text, out decimal giaNhap) ||
    giaNhap < 0)
    {

```

```

        MessageBox.Show("Giá nhập không hợp lệ!", "Thông báo",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (cbbLoaiMon.SelectedValue == null)
    {
        MessageBox.Show("Vui lòng chọn loại món!", "Thông báo",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    int maLoai = Convert.ToInt32(cbbLoaiMon.SelectedValue);
    string anh = txtAnh.Text.Trim();

    try
    {
        using (SqlConnection conn =
        DataProvider.Instance.GetConnection())
        {
            conn.Open();
            using (SqlCommand cmd = new SqlCommand("sp_ThemMon", conn))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@TenMon", tenMon);
                cmd.Parameters.AddWithValue("@Gia", gia);
                cmd.Parameters.AddWithValue("@GiaNhap", giaNhap);
                cmd.Parameters.AddWithValue("@TonKho", 0);
                cmd.Parameters.AddWithValue("@MaLoai", maLoai);
                cmd.Parameters.AddWithValue("@Anh", anh);
                SqlParameter maMonParam = new SqlParameter("@MaMon",
                SqlDbType.Int) { Direction = ParameterDirection.Output };
                cmd.Parameters.Add(maMonParam);

                cmd.ExecuteNonQuery();
                MessageBox.Show($"Thêm món mới thành công! Mã món:
                {maMonParam.Value}", "Thông báo", MessageBoxButtons.OK,
                MessageBoxIcon.Information);
                this.Close();
            }
        }
    }
}

```

```

        catch (Exception ex)
        {
            MessageBox.Show($"Lỗi khi thêm món: {ex.Message}", "Lỗi",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}

```

- Tìm kiếm trên thanh tìm kiếm

Chức năng tìm kiếm món ăn trong DatMon cho phép người dùng nhập từ khóa vào ô txtTimkiem để lọc danh sách món ăn hiển thị trên FlowLayoutPanel. Khi từ khóa thay đổi, sự kiện txtTimkiem_TextChanged kích hoạt, xóa danh sách món hiện tại và gọi MonDAO.TimKiemMon để lấy món phù hợp. Kết quả được hiển thị dưới dạng các ucMon, mỗi món có thể nhấp để thêm vào giỏ hàng. Hàm xử lý lỗi đảm bảo thông báo nếu truy vấn cơ sở dữ liệu thất bại. Chức năng này giúp người dùng nhanh chóng tìm món ăn theo tên hoặc đặc điểm.

```

private void txtTimkiem_TextChanged(object sender, EventArgs e)
{
    try
    {
        string keyword = txtTimkiem.Text.Trim();
        flpMonAn.Controls.Clear();
        List<Mon> dsMon = MonDAO.Instance.TimKiemMon(keyword);
        foreach (var mon in dsMon)
        {
            ucMon item = new ucMon(mon.MaMon, mon.TenMon, mon.Gia,
                mon.HinhAnh);
            item.Click += UcMon_Click;
            flpMonAn.Controls.Add(item);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Lỗi khi tìm kiếm món: {ex.Message}", "Lỗi",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```


4.3.4. Code trên DAL

- GoiMonDAO.AddOrUpdateGoiMon

```
public bool AddOrUpdateGoiMon(int maHD, int maMon)
{
    try
    {
        using (SqlConnection conn =
DataProvider.Instance.GetConnection())
        {
            conn.Open();
            using (SqlCommand cmd = new
SqlCommand("sp_AddOrUpdateGoiMon", conn))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@MaHD", maHD);
                cmd.Parameters.AddWithValue("@MaMon", maMon);
                cmd.Parameters.AddWithValue("@SoLuong", 1);
                int rowsAffected = cmd.ExecuteNonQuery();
                return rowsAffected > 0;
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Lỗi trong GoiMonDAO.AddOrUpdateGoiMon:
{ex.Message}\n{ex.StackTrace}");
        throw new Exception("Không thể thêm hoặc cập nhật món gọi", ex);
    }
}
```

Phương thức này thêm một món ăn (maMon) vào hóa đơn (maHD) hoặc cập nhật số lượng nếu món đã tồn tại trong hóa đơn. Nó gọi stored procedure sp_AddOrUpdateGoiMon và rất quan trọng để form đặt món cho phép người dùng thêm hoặc chỉnh sửa món.

- GoiMonDAO.CalculateTotalByMaHD

```
public decimal CalculateTotalByMaHD(int maHD)
{
```

```

        try
        {
            using (SqlConnection conn =
DataProvider.Instance.GetConnection())
            {
                conn.Open();
                using (SqlCommand cmd = new
SqlCommand("sp_CalculateTotalByMaHD", conn))
                {
                    cmd.CommandType = CommandType.StoredProcedure;
                    cmd.Parameters.AddWithValue("@MaHD", maHD);
                    SqlParameter outputParam = new SqlParameter("@TongTien",
SqlDbType.Decimal) { Direction = ParameterDirection.Output };
                    cmd.Parameters.Add(outputParam);

                    cmd.ExecuteNonQuery();
                    return outputParam.Value != null && outputParam.Value !=
DBNull.Value ? Convert.ToDecimal(outputParam.Value) : 0;
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Lỗi trong GoiMonDAO.CalculateTotalByMaHD:
{ex.Message}");
            throw new Exception("Không thể tính tổng tiền hóa đơn", ex);
        }
    }
}

```

Phương thức này tính tổng tiền của một hóa đơn (maHD) bằng cách gọi stored procedure sp_CalculateTotalByMaHD. Nó quan trọng để hiển thị tổng giá trị các món trong form đặt món.

4.3.5. Code trên tầng DTO

- Hàm LayTatCaMon()

```

public Mon(int id, string tenMon, decimal gia, string hìnhAnh, int maLoai)
{
    MaMon = id;
    TenMon = tenMon;
}

```

```

        Gia = gia;
        HinhAnh = hinhAnh;
        MaLoai = maLoai;
    }

    public Mon(DataRow row)
    {
        MaMon = row["MaMon"] != DBNull.Value ? Convert.ToInt32(row["MaMon"])
: 0;

        TenMon = row["TenMon"]?.ToString() ?? "";
        Gia = row["Gia"] != DBNull.Value ? Convert.ToDecimal(row["Gia"]) :
0;

        HinhAnh = row["Anh"]?.ToString() ?? "";
        MaLoai = row["MaLoai"] != DBNull.Value ?
Convert.ToInt32(row["MaLoai"]) : 0;
    }

```

Đây là lớp Mon dùng để biểu diễn một món ăn trong ứng dụng. Lớp có hai constructor. Constructor thứ nhất nhận trực tiếp các tham số: mã món, tên món, giá, hình ảnh và mã loại. Constructor thứ hai khởi tạo đối tượng từ một DataRow, dùng để lấy dữ liệu từ cơ sở dữ liệu, đồng thời xử lý kiểm tra giá trị null.

4.4. XÂY DỰNG GIAO DIỆN FORM HÓA ĐƠN (BILLING)

4.4.1. Yêu cầu và kịch bản xử lý của từng chức năng trên form Hóa đơn

Xây dựng chức năng báo cáo tình hình kinh doanh của cửa hàng theo các khoảng thời gian khác nhau, thể hiện rõ sự khác biệt giữa doanh thu và chi phí. Cụ thể, hệ thống hiển thị danh sách toàn bộ lịch sử giao dịch, phân loại thành hóa đơn thu và hóa đơn chi. Người dùng có khả năng lọc danh sách này theo một khoảng thời gian tùy chọn, đồng thời giao diện sẽ tự động tính toán và hiển thị tổng doanh thu, tổng chi phí và lợi nhuận trong khoảng thời gian đó. Thêm vào đó, người dùng có thể so sánh lợi nhuận giữa hai tháng bất kỳ.

Ngoài ra, hệ thống cung cấp chức năng tìm kiếm thông tin lịch sử hóa đơn một cách linh hoạt dựa trên các tiêu chí hợp lệ như tên nhân viên tạo hóa đơn hoặc loại hóa đơn (thu/chi).

Cuối cùng, hệ thống cho phép người dùng in hóa đơn thanh toán cho khách hàng sau khi họ sử dụng dịch vụ.

4.4.2. Demo chức năng trên giao diện

The screenshot displays the 'Hóa đơn' (Invoice) section of a web application. The top navigation bar includes links for 'Trang chủ', 'Đặt máy', 'Đặt món', 'Hóa đơn', 'Bảo trì', and 'Đăng xuất'. The main content area is titled 'Lịch sử thanh toán' (Transaction History) and features a search filter for 'Nhân viên' (Staff) and buttons for 'Thu' (Revenue) and 'Chi' (Expense), along with a 'Reset' button. Below the filter is a table listing transactions with columns for 'Mã hóa đơn' (Invoice Code), 'Ngày thanh toán' (Payment Date), 'Nhân viên' (Staff), 'Loại hóa đơn' (Invoice Type), 'Giá món ăn' (Food Price), 'Chi phí bảo trì' (Maintenance Cost), and 'Giá thuê máy' (Machine Rental Price). The table shows several entries, including those for 'Bùi Thị Hoa' and 'Nguyễn Thị...'. To the right of the table, there are two summary panels: 'Tổng kê theo khoảng' (Summary by Period) and 'Tổng kê theo tháng' (Summary by Month). The 'Tổng kê theo khoảng' panel shows a date range from '01/05/2025 00:00' to '31/05/2025 01:51' with a total of '13.224.166,65 đ'. The 'Tổng kê theo tháng' panel shows a date range from '05/2025' to '04/2025' with a total of '0'. At the bottom right, there is a button labeled 'In hóa đơn' (Print Invoice).

Mã hóa đơn	Ngày thanh toán	Nhân viên	Loại hóa đơn	Giá món ăn	Chi phí bảo trì	Giá thuê máy
1	15/05/2025 10:00	Bùi Thị Hoa	Thu	110.000,00 đ	0,00 đ	3.778.500,00 đ
2	16/05/2025 15:30	Nguyễn Thị...	Thu	95.000,00 đ	0,00 đ	3.483.500,00 đ
3	17/05/2025 09:00	Nguyễn Thị...	Chi	0,00 đ	0,00 đ	0,00 đ
4	18/05/2025 20:00	Nguyễn Thị...	Thu	80.000,00 đ	0,00 đ	8.835.500,00 đ
5	19/05/2025 14:00	Nguyễn Thị...	Chi	0,00 đ	0,00 đ	0,00 đ
32	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	400.000,00 đ	0,00 đ
36	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
38	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
42	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
44	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
48	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.800.000,00 đ	0,00 đ
51	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
55	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
58	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ

Hình 4.25. Giao diện hóa đơn

Khi người dùng chọn mục "Hóa đơn", giao diện sẽ hiển thị danh sách các giao dịch đã thực hiện của quán. Ở phía bên phải màn hình sẽ có một khu vực (panel) chứa hai chức năng thống kê của form hóa đơn.

[Trang chủ](#)
[Đặt máy](#)
[Đặt món](#)
[Hóa đơn](#)
[Bảo trì](#)
[Đăng xuất](#)

Lịch sử thanh toán

Bộ lọc

Nhân viên

Bùi Thị Hoa

☐ Thu
☒ Chi

Reset

Mã hóa đơn	Ngày thanh toán	Nhân viên	Loại hóa đơn	Giá món ăn	Chi phí bảo trì	Giá thuê máy
1	15/05/2025 10:00	Bùi Thị Hoa	Thu	110.000,00 đ	0,00 đ	3.778.666,66 đ
32	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	400.000,00 đ	0,00 đ
36	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
38	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
42	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
44	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
48	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.800.000,00 đ	0,00 đ
51	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
55	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
58	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ

• Thống kê theo khoảng

Từ ngày

01/05/2025 00:00

Đến

31/05/2025 01:51

Tổng thu

3.248.833,33 đ

Tổng chi

6.000.000,00 đ

Lợi nhuận

-2.751.166,67 đ

• Thống kê theo tháng

Tháng

05/2025

Lợi nhuận

0

Tháng

04/2025

Lợi nhuận

0

In hóa đơn

Hình 4.26. Giao diện tìm kiếm hóa đơn theo tên nhân viên

Khi nhập tên nhân viên vào ô tìm kiếm, những hóa đơn do nhân viên tạo sẽ hiển thị. Các hóa đơn khác sẽ bị ẩn đi.

[Trang chủ](#)
[Đặt máy](#)
[Đặt món](#)
[Hóa đơn](#)
[Bảo trì](#)
[Đăng xuất](#)

Lịch sử thanh toán

Bộ lọc

Nhân viên

☐ Thu
☒ Chi

Reset

Mã hóa đơn	Ngày thanh toán	Nhân viên	Loại hóa đơn	Giá món ăn	Chi phí bảo trì	Giá thuê máy
3	17/05/2025 09:00	Nguyen Thi ...	Chi	0,00 đ	0,00 đ	0,00 đ
5	19/05/2025 14:00	Nguyen Thi ...	Chi	0,00 đ	0,00 đ	0,00 đ
32	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	400.000,00 đ	0,00 đ
36	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
38	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
42	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
44	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
48	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.800.000,00 đ	0,00 đ
51	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
55	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
58	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ

• Thống kê theo khoảng

Từ ngày

01/05/2025 00:00

Đến

31/05/2025 01:51

Tổng thu

0,00 đ

Tổng chi

10.400.000,00 đ

Lợi nhuận

-10.400.000,00 đ

• Thống kê theo tháng

Tháng

05/2025

Lợi nhuận

0

Tháng

04/2025

Lợi nhuận

0

In hóa đơn

Hình 4.27. Giao diện lọc hóa đơn theo loại hóa đơn Chi

Khi ấn vào nút Chi, những hóa đơn có loại hóa đơn là Chi (chi phí bảo trì) sẽ hiển thị trên giao diện. Các hóa đơn có loại hóa đơn là Thu sẽ bị ẩn đi.

Lịch sử thanh toán

Bộ lọc: Nhân viên ☒ Thu ☐ Chi

Mã hóa đơn	Ngày thanh toán	Nhân viên	Loại hóa đơn	Giá món ăn	Chi phí bảo trì	Giá thuê máy
1	15/05/2025 10:00	Bùi Thị Hoa	Thu	110.000,00 đ	0,00 đ	3.779.166,66 đ
2	16/05/2025 15:30	Nguyễn Thị ...	Thu	95.000,00 đ	0,00 đ	3.484.166,66 đ
4	18/05/2025 20:00	Nguyễn Thị ...	Thu	80.000,00 đ	0,00 đ	8.837.499,98 đ

• Thống kê theo khoảng

Từ ngày: 01/05/2025 00:00
Đến: 31/05/2025 01:51

Tổng thu: 13.224.166,65 đ
Tổng chi: 0,00 đ
Lợi nhuận: 13.224.166,65 đ

• Thống kê theo tháng

Tháng: 05/2025
Lợi nhuận: 0

Tháng: 04/2025
Lợi nhuận: 0

Hình 4.28. Giao diện lọc hóa đơn theo loại hóa đơn Thu

Khi ấn vào nút Thu, những hóa đơn có loại hóa đơn Thu sẽ hiển thị trên giao diện. Các hóa đơn có loại hóa đơn Chi sẽ bị ẩn đi.

Lịch sử thanh toán

Bộ lọc: Nhân viên ☐ Thu ☒ Chi

Mã hóa đơn	Ngày thanh toán	Nhân viên	Loại hóa đơn	Giá món ăn	Chi phí bảo trì	Giá thuê máy
5	19/05/2025 14:00	Nguyễn Thị ...	Chi	0,00 đ	0,00 đ	0,00 đ
32	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	400.000,00 đ	0,00 đ
36	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
38	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
42	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
44	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
48	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.800.000,00 đ	0,00 đ
51	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
55	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
58	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ

• Thống kê theo khoảng

Từ ngày: 19/05/2025 00:00
Đến: 31/05/2025 01:51

Tổng thu: 0,00 đ
Tổng chi: 8.000.000,00 đ
Lợi nhuận: -8.000.000,00 đ

• Thống kê theo tháng

Tháng: 05/2025
Lợi nhuận: 0

Tháng: 04/2025
Lợi nhuận: 0

Hình 4.29. Giao diện thống kê hóa đơn theo khoảng

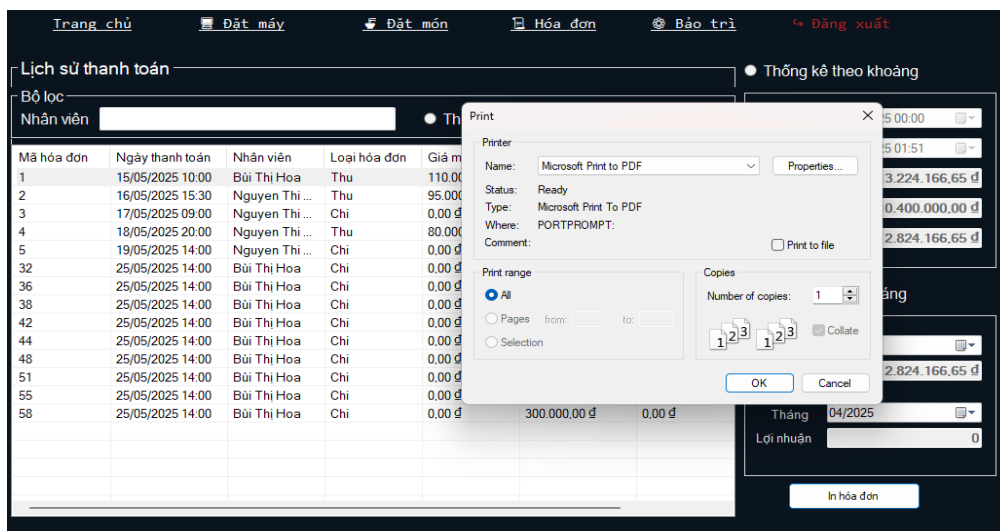
Khi người dùng chọn chức năng "Tổng kê theo khoảng" và chỉ định khoảng thời gian từ ngày (ví dụ 19/05/2025 đến 31/05/2025), giao diện sẽ hiển thị danh sách các giao dịch diễn ra trong khoảng thời gian này, đồng thời hiển thị tổng doanh thu, tổng chi phí và lợi nhuận tương ứng.

The screenshot displays a software interface with a dark theme. At the top, there are navigation tabs: "Trang chủ", "Đặt máy", "Đặt món", "Hóa đơn", "Bảo trì", and "Đăng xuất". The main section is titled "Lịch sử thanh toán" (Transaction History). It includes a filter section with "Bộ lọc:" and "Nhân viên" (Employee) with a dropdown menu, and radio buttons for "Thu" (Revenue) and "Chi" (Expense), along with a "Reset" button. Below this is a table with the following columns: "Mã hóa đơn" (Invoice Code), "Ngày thanh toán" (Payment Date), "Nhân viên" (Employee), "Loại hóa đơn" (Invoice Type), "Giá món ăn" (Food Item Price), "Chi phí bảo trì" (Maintenance Cost), and "Giá thuê máy" (Machine Rental Price). The table lists several transactions, including those from 15/05/2025 and 25/05/2025. To the right of the table, there are two summary sections. The first, "Tổng kê theo khoảng" (Summary by Range), shows a date range from 19/05/2025 00:00 to 31/05/2025 01:51, with a total revenue of 13,224,166.65 đ, total expense of 10,400,000.00 đ, and a profit of 2,824,166.65 đ. The second, "Tổng kê theo tháng" (Summary by Month), shows the month of 05/2025 with a profit of 2,824,166.65 đ. At the bottom right, there is a button labeled "In hóa đơn" (Print Invoice).

Mã hóa đơn	Ngày thanh toán	Nhân viên	Loại hóa đơn	Giá món ăn	Chi phí bảo trì	Giá thuê máy
1	15/05/2025 10:00	Bùi Thị Hoa	Thu	110.000,00 đ	0,00 đ	3.779.333,33 đ
2	16/05/2025 15:30	Nguyen Thi ...	Thu	95.000,00 đ	0,00 đ	3.484.333,33 đ
3	17/05/2025 09:00	Nguyen Thi ...	Chi	0,00 đ	0,00 đ	0,00 đ
4	18/05/2025 20:00	Nguyen Thi ...	Thu	80.000,00 đ	0,00 đ	8.837.999,99 đ
5	19/05/2025 14:00	Nguyen Thi ...	Chi	0,00 đ	0,00 đ	0,00 đ
32	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	400.000,00 đ	0,00 đ
36	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
38	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
42	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	200.000,00 đ	0,00 đ
44	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ
48	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.800.000,00 đ	0,00 đ
51	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
55	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	1.500.000,00 đ	0,00 đ
58	25/05/2025 14:00	Bùi Thị Hoa	Chi	0,00 đ	300.000,00 đ	0,00 đ

Hình 4.30. Giao diện thống kê hóa đơn theo tháng

Người dùng chọn tháng cần thống kê, các giao dịch trong tháng đó được hiển thị trên danh sách đồng thời hiển thị lợi nhuận tương ứng. Người dùng có thể chọn đồng thời 2 tháng để so sánh lợi nhuận.



Hình 4.31. Giao diện in hóa đơn

Khi chọn chức năng in hóa đơn, nếu chưa chọn hóa đơn sẽ hiển thị thông báo “Vui lòng chọn một hóa đơn”. Sau khi chọn hóa đơn, chọn nút In hóa đơn, giao diện sẽ hiển thị ô để người dùng chọn kiểu mình muốn, ấn nút OK để thực hiện quá trình in, Cancel để hủy.



Hình 4.32. Giao diện hóa đơn

Sau khi in hóa đơn, hóa đơn được in sẽ hiển thị theo định dạng được cài đặt sẵn bao gồm: Tên hóa đơn, Mã hóa đơn, Ngày tạo, Nhân viên, Loại hóa đơn, Danh sách món, Danh sách thuê máy, Giá thuê máy, Giá món ăn, Chi phí bảo trì, Tổng tiền.

4.4.3. Code trên tầng GUI

- Hiển thị danh sách lịch sử giao dịch

```
void ShowBill(DateTime startDate, DateTime endDate)
{
    try
    {
        lvBill.Items.Clear();
        string searchName = txtNhanvien.Text.Trim();
        string billingType = rbThu.Checked ? "Thu" : rbChi.Checked ?
"Chi" : null;

        if (!rbKhoang.Checked && rbHai.Checked &&
lastSelectedDateTimePicker != null)
        {
            startDate = new
DateTime(lastSelectedDateTimePicker.Value.Year,
lastSelectedDateTimePicker.Value.Month, 1);
            endDate = startDate.AddMonths(1).AddSeconds(-1);
        }

        currentBillList = BillingDAL.Instance.LoadBillList(startDate,
endDate, searchName, billingType);
        decimal income = 0;
        decimal outcome = 0;

        lvBill.Columns.Clear();
        lvBill.View = View.Details;
        lvBill.FullRowSelect = true;
        lvBill.GridLines = true;

        lvBill.Columns.Add("Mã hóa đơn", 100);
        lvBill.Columns.Add("Ngày thanh toán", 120);
        lvBill.Columns.Add("Nhân viên", 100);
        lvBill.Columns.Add("Loại hóa đơn", 100);
        lvBill.Columns.Add("Giá món ăn", 100);
        lvBill.Columns.Add("Chi phí bảo trì", 120);
        lvBill.Columns.Add("Giá thuê máy", 100);
        lvBill.Columns.Add("Tổng tiền", 120);

        foreach (var bill in currentBillList)
```

```

        {
            ListViewItem item = new ListViewItem(bill.MaHD.ToString());
            item.SubItems.Add(bill.NgayTao.ToString("dd/MM/yyyy
HH:mm"));

            item.SubItems.Add(bill.TenNV);
            item.SubItems.Add(bill.LoaiHD);
            item.SubItems.Add(bill.GiaMon.ToString("C2", ct));
            item.SubItems.Add(bill.ChiPhi.ToString("C2", ct));
            item.SubItems.Add(bill.ThueMay.ToString("C2", ct));
            item.SubItems.Add(bill.TongTien.ToString("C2", ct));

            if (bill.LoaiHD == "Thu")
                income += bill.TongTien;
            else
                outcome += bill.TongTien;

            lvBill.Items.Add(item);
        }

        inc = income;
        outc = outcome;
        profit = income - outcome;
        txtIn.Text = inc.ToString("C2", ct);
        txtOut.Text = outc.ToString("C2", ct);
        txtProfit.Text = profit.ToString("C2", ct);

        if (rbHai.Checked && lastSelectedDateTimePicker != null)
        {
            if (lastSelectedDateTimePicker == tpTs)
                txtProfitMonth.Text = profit.ToString("C2", ct);
            else if (lastSelectedDateTimePicker == tpTe)
                txtProfitOtherMonth.Text = profit.ToString("C2", ct);
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Lỗi trong ShowBill: {ex.Message}\n{ex.StackTrace}");
        MessageBox.Show("Không thể tải danh sách hóa đơn.", "Lỗi",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
        currentBillList = new List<Hoadon>();
        lvBill.Items.Clear();
    }
}

```

```
}  
}
```

Hàm ShowBill có nhiệm vụ lấy dữ liệu hóa đơn từ tầng DAL thông qua hàm loadBillList và hiển thị danh sách này lên ListView. Hàm này nhận vào bốn tham số để lọc dữ liệu, bao gồm: ngaybd (ngày bắt đầu), ngaykt (ngày kết thúc), search (từ khóa tìm kiếm theo tên nhân viên) và billingType (kiểu hóa đơn, ví dụ: thu hoặc chi). Kết quả trả về của loadBillList (một danh sách các đối tượng Billing) sẽ được gán và hiển thị trên giao diện ListView.

- In hóa đơn

```
private void PrintDocument_PrintPage(object sender, PrintPageEventArgs e)
{
    if (lvBill.SelectedItems.Count == 0 || currentBillList == null ||
currentBillList.Count == 0)
        return;

    int selectedIndex = lvBill.SelectedIndices[0];
    if (selectedIndex < 0 || selectedIndex >= currentBillList.Count)
        return;

    Hoadon bill = currentBillList[selectedIndex];
    Font titleFont = new Font("Arial", 16, FontStyle.Bold);
    Font regularFont = new Font("Arial", 12);
    Font boldFont = new Font("Arial", 12, FontStyle.Bold);
    float yPos = 50;
    float pageWidth = e.MarginBounds.Width;
    CultureInfo ct = new CultureInfo("vi-VN"); // Định dạng VNĐ

    // Tiêu đề
    string title = "Hóa Đơn Metanoia Gaming";
    float xPos = (pageWidth - e.Graphics.MeasureString(title,
titleFont).Width) / 2;
    e.Graphics.DrawString(title, titleFont, Brushes.Black, xPos, yPos);
    yPos += titleFont.GetHeight() + 20;

    // Thông tin hóa đơn
    e.Graphics.DrawString($"Mã hóa đơn: {bill.MaHD}", regularFont,
Brushes.Black, 50, yPos);
```

```

        yPos += regularFont.GetHeight();
        e.Graphics.DrawString($"Ngày tạo: {bill.NgayTao:dd/MM/yyyy HH:mm}",
regularFont, Brushes.Black, 50, yPos);
        yPos += regularFont.GetHeight();
        e.Graphics.DrawString($"Nhân viên: {bill.TenNV}", regularFont,
Brushes.Black, 50, yPos);
        yPos += regularFont.GetHeight();
        e.Graphics.DrawString($"Loại hóa đơn: {bill.LoaiHD}", regularFont,
Brushes.Black, 50, yPos);
        yPos += regularFont.GetHeight() + 10;

        // Chi tiết món gọi
        e.Graphics.DrawString("Danh sách món:", boldFont, Brushes.Black, 50,
yPos);
        yPos += boldFont.GetHeight() + 5;
        try
        {
            DataTable dt = GoiMonDAO.Instance.GetGoiMonByMaHD(bill.MaHD);
            if (dt.Rows.Count == 0)
            {
                e.Graphics.DrawString("Không có món gọi.", regularFont,
Brushes.Black, 70, yPos);
                yPos += regularFont.GetHeight();
            }
            else
            {
                foreach (DataRow row in dt.Rows)
                {
                    string tenMon = row["TenMon"]?.ToString() ?? "";
                    int soLuong = row["SoLuong"] != DBNull.Value ?
Convert.ToInt32(row["SoLuong"]) : 0;
                    decimal thanhTien = row["ThanhTien"] != DBNull.Value ?
Convert.ToDecimal(row["ThanhTien"]) : 0;
                    e.Graphics.DrawString($"{tenMon} x {soLuong}:
{thanhTien.ToString("C2", ct)}", regularFont, Brushes.Black, 70, yPos);
                    yPos += regularFont.GetHeight();
                }
            }
        }
        catch (Exception ex)
        {

```

```

        Console.WriteLine($"Lỗi khi in món gọi (MaHD={bill.MaHD}):
{ex.Message}\n{ex.StackTrace}");
        e.Graphics.DrawString("Không thể tải danh sách món.",
regularFont, Brushes.Black, 70, yPos);
        yPos += regularFont.GetHeight();
    }

    // Chi tiết thuê máy
    yPos += 10;
    e.Graphics.DrawString("Danh sách thuê máy:", boldFont,
Brushes.Black, 50, yPos);
    yPos += boldFont.GetHeight() + 5;
    try
    {
        using (SqlConnection conn =
DataProvider.Instance.GetConnection())
        {
            conn.Open();
            SqlCommand cmd = new SqlCommand(@"
SELECT
    mt.TenMay,
    tm.GioBD,
    tm.GioKT,
    p.Giatheogio,
    DATEDIFF(MINUTE, tm.GioBD, ISNULL(tm.GioKT, GETDATE())) /
60.0 * p.Giatheogio AS ThanhTien
FROM Thuemay tm
JOIN Maytinh mt ON tm.MaMay = mt.MaMay
JOIN Phong p ON mt.MaPhong = p.MaPhong
WHERE tm.MaHD = @MaHD", conn);
            cmd.Parameters.AddWithValue("@MaHD", bill.MaHD);
            using (SqlDataReader dr = cmd.ExecuteReader())
            {
                if (!dr.HasRows)
                {
                    e.Graphics.DrawString("Không có thuê máy.",
regularFont, Brushes.Black, 70, yPos);
                    yPos += regularFont.GetHeight();
                }
                else
                {
                    while (dr.Read())

```

```

        {
            string tenMay = dr["TenMay"]?.ToString() ?? "";
            string gioBD =
Convert.ToDateTime(dr["GioBD"]).ToString("dd/MM/yyyy HH:mm");
            string gioKT = dr["GioKT"] != DBNull.Value
                ?
Convert.ToDateTime(dr["GioKT"]).ToString("dd/MM/yyyy HH:mm")
                : "Đang thuê";
            decimal thanhTien = dr["ThanhTien"] !=
DBNull.Value
                ? Convert.ToDecimal(dr["ThanhTien"])
                : 0;
            e.Graphics.DrawString($"{tenMay} ({gioBD} -
{gioKT}): {thanhTien.ToString("C2", ct)}", regularFont, Brushes.Black, 70,
yPos);
            yPos += regularFont.GetHeight();
        }
    }
}
}
catch (Exception ex)
{
    Console.WriteLine($"Lỗi khi in thuê máy (MaHD={bill.MaHD}):
{ex.Message}\n{ex.StackTrace}");
    e.Graphics.DrawString("Không thể tải danh sách thuê máy.",
regularFont, Brushes.Black, 70, yPos);
    yPos += regularFont.GetHeight();
}

// Tổng hợp
yPos += 10;
e.Graphics.DrawString($"Giá thuê máy: {bill.ThueMay.ToString("C2",
ct)}", regularFont, Brushes.Black, 50, yPos);
yPos += regularFont.GetHeight();
e.Graphics.DrawString($"Giá món ăn: {bill.GiaMon.ToString("C2",
ct)}", regularFont, Brushes.Black, 50, yPos);
yPos += regularFont.GetHeight();
e.Graphics.DrawString($"Chi phí bảo trì: {bill.ChiPhi.ToString("C2",
ct)}", regularFont, Brushes.Black, 50, yPos);
yPos += regularFont.GetHeight();

```

```

        e.Graphics.DrawString($"Tổng tiền: {bill.TongTien.ToString("C2",
ct)}}", boldFont, Brushes.Black, 50, yPos);
        yPos += boldFont.GetHeight() + 20;

        // Lời cảm ơn
        xPos = (pageWidth - e.Graphics.MeasureString("Cảm ơn quý khách!",
new Font("Arial", 12, FontStyle.Italic)).Width) / 2;
        e.Graphics.DrawString("Cảm ơn quý khách!", new Font("Arial", 12,
FontStyle.Regular), Brushes.Black, xPos, yPos);
    }

```

Hàm PrintDocument_PrintPage có nhiệm vụ vẽ toàn bộ hóa đơn chi tiết ra trang in, bao gồm thông tin chung, món gọi, máy thuê và tổng hợp chi phí. Đây là hàm cốt lõi để xuất hóa đơn ra giấy trong phần mềm Metanoia Gaming.

```

private void btnIn_Click(object sender, EventArgs e)
{
    if (lvBill.SelectedItems.Count > 0)
    {
        if (currentBillList == null || currentBillList.Count == 0)
        {
            MessageBox.Show("Danh sách hóa đơn trống!", "Lỗi",
MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        if (lvBill.Items.Count != currentBillList.Count)
        {
            MessageBox.Show("Dữ liệu không đồng bộ!", "Lỗi",
MessageBoxButtons.OK, MessageBoxIcon.Error);
            ShowBill(tpBd.Value, tpKt.Value);
            return;
        }

        int selectedIndex = lvBill.SelectedIndex;
        if (selectedIndex < 0 || selectedIndex >= currentBillList.Count)
        {
            MessageBox.Show("Chỉ số hóa đơn không hợp lệ!", "Lỗi",
MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
}

```

```

        PrintDialog printDialog = new PrintDialog();
        printDialog.Document = printDocument;
        if (printDialog.ShowDialog() == DialogResult.OK)
        {
            printDocument.Print();
        }
    }
    else
    {
        MessageBox.Show("Vui lòng chọn một hóa đơn để in!", "Thông báo",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}

```

Khi người dùng nhấp vào nút in hóa đơn, phương thức btnIn_Click sẽ được thực thi. Đầu tiên, nó sẽ kiểm tra xem người dùng đã chọn một hóa đơn nào từ DataGridView hay chưa. Nếu có hóa đơn được chọn, phương thức sẽ tiến hành truy xuất thông tin chi tiết của hóa đơn đó cùng với danh sách các món ăn có trong hóa đơn từ cơ sở dữ liệu. Cuối cùng, một tài liệu in sẽ được tạo ra, cho phép người dùng in hóa đơn này.

```

void ApplyDateFilter()
{
    if (lastSelectedDateTimePicker != null)
    {
        DateTime startDate = new
        DateTime(lastSelectedDateTimePicker.Value.Year,
        lastSelectedDateTimePicker.Value.Month, 1);
        DateTime endDate = startDate.AddMonths(1).AddSeconds(-1);
        ShowBill(startDate, endDate);
    }
}

```

Phương thức ApplyDateFilter được khởi tạo để lấy dữ liệu hóa đơn trong thời hạn nhất định và hiển thị lên giao diện. Để lọc và hiển thị danh sách hóa đơn theo tháng được chọn cuối cùng, hãy kiểm tra nếu lastSelectedDateTimePicker khác null, sau đó lấy ngày

đầu tháng từ `lastSelectedDateTimePicker.Value` và tính ngày cuối tháng bằng cách cộng một tháng rồi trừ một giây, cuối cùng gọi `ShowBill(startDate, endDate)`.

```
void ResetValue()
{
    txtNhanvien.Text = "";
    rbThu.Checked = false;
    rbChi.Checked = false;
}
```

Hàm `ResetValue()` có chức năng khôi phục trạng thái ban đầu của các bộ lọc trên giao diện. Cụ thể, nó sẽ xóa toàn bộ nội dung trong ô nhập liệu tìm kiếm nhân viên (`txtNhanvien.Text = ""`;) và đồng thời bỏ chọn cả hai tùy chọn radio button lọc theo kiểu hóa đơn thu và chi (`rbThu.Checked = false`; `rbChi.Checked = false`;) . Mục đích của hàm này là cung cấp cho người dùng một cách nhanh chóng để loại bỏ các tiêu chí lọc hiện tại, trả giao diện về trạng thái hiển thị mặc định, tạo điều kiện thuận lợi cho việc thực hiện các tìm kiếm hoặc thống kê mới.

```
void LoadDate()
{
    DateTime today = DateTime.Now;
    tpBd.Value = new DateTime(today.Year, today.Month, 1);
    tpKt.Value = today;
    tpTs.Value = new DateTime(today.Year, today.Month, 1);
    tpTe.Value = today.AddMonths(-1);
}
```

Phương thức `LoadDate()` được thiết kế để khởi tạo các giá trị mặc định cho các bộ chọn thời gian trên giao diện. Nó lấy thời điểm hiện tại của hệ thống và sử dụng thông tin này để đặt ngày bắt đầu của khoảng thời gian hiện tại là ngày đầu tiên của tháng hiện tại cho bộ chọn `tpBd`, và ngày kết thúc là ngày hiện tại cho `tpKt`. Tương tự, cho mục đích so sánh, nó đặt ngày bắt đầu của khoảng thời gian so sánh (`tpTs`) là ngày đầu tiên của tháng hiện tại, và ngày kết thúc (`tpTe`) là ngày hiện tại của tháng trước. Như vậy, khi giao diện được tải, người dùng sẽ thấy các bộ chọn thời gian đã được thiết lập sẵn với các khoảng thời gian mặc định liên quan đến tháng hiện tại và tháng trước.

4.3.4. Code trên tầng DAL

- Hàm LoadBillist

```
public List<Hoadon> LoadBillList(DateTime ngayBD, DateTime ngayKT, string tenNV = null, string loaiHD = null)
{
    try
    {
        Console.WriteLine($"Gọi sp_GetBillListFiltered: NgayBD={ngayBD}, NgayKT={ngayKT}, TenNV={tenNV ?? "NULL"}, LoaiHD={loaiHD ?? "NULL"}");

        var parameters = new (string, object)[]
        {
            ("@NgayBD", ngayBD),
            ("@NgayKT", ngayKT),
            ("@TenNV", (object)tenNV ?? DBNull.Value),
            ("@LoaiHD", (object)loaiHD ?? DBNull.Value)
        };

        DataTable dt =
        DataProvider.Instance.ExecuteQuery("sp_GetBillListFiltered", parameters,
        CommandType.StoredProcedure);

        List<Hoadon> danhSach = new List<Hoadon>();

        if (dt == null || dt.Rows.Count == 0)
        {
            Console.WriteLine("Không tìm thấy hóa đơn.");
            return danhSach;
        }

        foreach (DataRow row in dt.Rows)
        {
            danhSach.Add(new Hoadon(row));
        }
        return danhSach;
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine($"Lỗi SQL trong BillingDAL.LoadBillList: {sqlEx.Message}\n{sqlEx.StackTrace}");
    }
}
```

```

        throw new Exception("Không thể tải danh sách hóa đơn", sqlEx);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Lỗi trong BillingDAL.LoadBillList:
{ex.Message}\n{ex.StackTrace}");
        throw new Exception("Không thể tải danh sách hóa đơn", ex);
    }
}

```

Truy vấn cơ sở dữ liệu để lấy toàn bộ hóa đơn được lập trong khoảng thời gian chỉ định, sau đó ánh xạ kết quả truy vấn thành danh sách các đối tượng Billing phục vụ cho việc xử lý hoặc hiển thị trong hệ thống.

- Hàm UpdateTongTien

```

public void UpdateTongTien(int maHD, decimal amount)
{
    try
    {
        var parameters = new (string, object)[]
        {
            ("@MaHD", maHD),
            ("@Amount", amount)
        };

        DataProvider.Instance.ExecuteNonQuery("sp_UpdateTongTien",
parameters, CommandType.StoredProcedure);

    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine($"Lỗi SQL trong BillingDAL.UpdateTongTien:
{sqlEx.Message}\n{sqlEx.StackTrace}");
        throw new Exception("Không thể cập nhật tổng tiền", sqlEx);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Lỗi trong BillingDAL.UpdateTongTien:
{ex.Message}\n{ex.StackTrace}");
        throw new Exception("Không thể cập nhật tổng tiền", ex);
    }
}

```

```
}  
}
```

Hàm UpdateTongTien(int maHD, decimal amount) nằm trong lớp BillingDAL của tầng Data Access Layer (DAL). Chức năng chính của hàm này là cập nhật giá trị của cột TongTien (tổng tiền) cho một hóa đơn cụ thể trong cơ sở dữ liệu. Hóa đơn cần cập nhật được xác định thông qua tham số maHD (mã hóa đơn), và giá trị tổng tiền mới được truyền vào qua tham số amount.

4.3.5. Code trên tầng DTO

```
public int MaHD { get; set; }  
public DateTime NgayTao { get; set; }  
public string TenNV { get; set; }  
public string LoaiHD { get; set; }  
public decimal ChiPhi { get; set; }  
public decimal GiaMon { get; set; }  
public decimal ThueMay { get; set; }  
public decimal TongTien { get; set; }
```

Mô tả một hóa đơn trong hệ thống, bao gồm các thuộc tính như MaHD, NgayTao, TenNV, LoaiHD, ChiPhi, GiaMon, ThueMay, TongTien. Các thuộc tính có get và set tương ứng để truy cập và thiết lập các giá trị của hóa đơn.

```
public Hoadon(DataRow row)  
{  
    MaHD = Convert.ToInt32(row["MaHD"]);  
    NgayTao = Convert.ToDateTime(row["NgayTao"]);  
    TenNV = row["TenNV"].ToString();  
    LoaiHD = row["LoaiHD"].ToString();  
    ChiPhi = row["ChiPhi"] != DBNull.Value ?  
Convert.ToDecimal(row["ChiPhi"]) : 0;  
    GiaMon = row["GiaMon"] != DBNull.Value ?  
Convert.ToDecimal(row["GiaMon"]) : 0;  
    ThueMay = row["ThueMay"] != DBNull.Value ?  
Convert.ToDecimal(row["ThueMay"]) : 0;  
    TongTien = row["TongTien"] != DBNull.Value ?  
Convert.ToDecimal(row["TongTien"]) : 0;  
}
```

Constructor với tham số DataRow row để khởi tạo một đối tượng hóa đơn từ một hàng dữ liệu.

4.4. XÂY DỰNG GIAO DIỆN FORM BẢO TRÌ

4.4.1. Yêu cầu và kịch bản xử lý của từng chức năng trên form Bảo trì

Khi máy gặp sự cố hoặc hoạt động bất thường, cần đánh dấu bảo trì và ghi chú chi tiết: người dùng mở form Bảo trì, chọn máy cần bảo trì từ danh sách, chọn bộ phận cần bảo trì từ danh sách bộ phận, chọn nhà bảo trì, và nhập ghi chú nếu cần, sau đó nhấn nút "Báo lỗi" để lưu thông tin bảo trì và đánh dấu máy ở trạng thái "Báo lỗi".

Khi bảo trì hoàn tất, cần lưu hóa đơn thanh toán và bỏ đánh dấu bảo trì: người dùng chọn các lỗi bảo trì đã hoàn tất từ danh sách trên form Bảo trì, kiểm tra tổng chi phí bảo trì, nhấn nút "Thanh toán" để lưu hóa đơn thanh toán (loại "Chi") và cập nhật trạng thái máy về "Trống" nếu không còn lỗi, hoặc giữ "Báo lỗi" nếu vẫn còn lỗi chưa hoàn tất.

4.4.2. Demo chức năng trên giao diện

- Báo lỗi cho máy gặp sự cố

Khi một máy trong cửa hàng gặp sự cố (ví dụ: hoạt động bất thường, hỏng hóc), nhân viên cần báo lỗi để bắt đầu quy trình bảo trì. Dưới đây là các bước thực hiện trên giao diện:

Tại giao diện của trang chủ, người dùng chọn Bảo trì để mở form Bảo trì (frmBaotri).

Hình 4.33. Giao diện Bảo trì

Trên giao diện form Bảo trì, bạn sẽ thấy một khu vực hiển thị danh sách máy (các nút bấm trong một FlowLayoutPanel).

Danh sách này hiển thị tất cả các máy trong cửa hàng, với màu sắc thể hiện trạng thái:

- Màu trắng: Máy trống (sẵn sàng sử dụng).
- Màu xanh: Máy đang sử dụng.
- Màu đỏ: Máy đang báo lỗi.

Tìm và xác định máy cần bảo trì (ví dụ: "Máy: 1-PC-Thuong-01" có dấu hiệu hỏng).

[Trang chủ](#)
[Đặt máy](#)
[Đặt món](#)
[Hóa đơn](#)
[Bảo trì](#)
[Đăng xuất](#)

Nhân viên: **Nguyen Thi Hai Yen**

Bảo trì

☐ máy còn trống: 29
 ☒ máy đang sử dụng: 1
 ☐ máy báo lỗi: 0

PC-Thuong-01	PC-Thuong-02	PC-Thuong-03	PC-Thuong-04	PC-Thuong-05
PC-Thuong-06	PC-Thuong-07	PC-Thuong-08	PC-Thuong-09	PC-Thuong-10
PC-Thuong-11	PC-Thuong-12	PC-Thuong-13	PC-Thuong-14	PC-Thuong-15
PC-Thuong-16	PC-Thuong-17	PC-Thuong-18	PC-Thuong-19	PC-Thuong-20
PC-Thuong-21	PC-Thuong-22	PC-Thuong-23	PC-Thuong-24	PC-Thuong-25
PC-VIP-01	PC-VIP-02	PC-VIP-03	PC-VIP-04	PC-VIP-05

Máy: 1 - PC-Thuong-01

Trạng thái máy:

Tình trạng máy:

Giải pháp:

Nhà Bảo trì:

Danh sách bảo trì

Bộ phận	Mô tả	Chi phí	Trạng
☐ Màn hình x...	Giải pháp: Cài lẹ...	200,000	Đã h
☐ Màn hình x...	Cài lại driver car...	200,000	Đã h
☐ Chơi game...	Nâng cấp RAM ...	1,500,000	Đã h
☐ Máy không...	Thay ổ cứng mới	1,200,000	Đã h

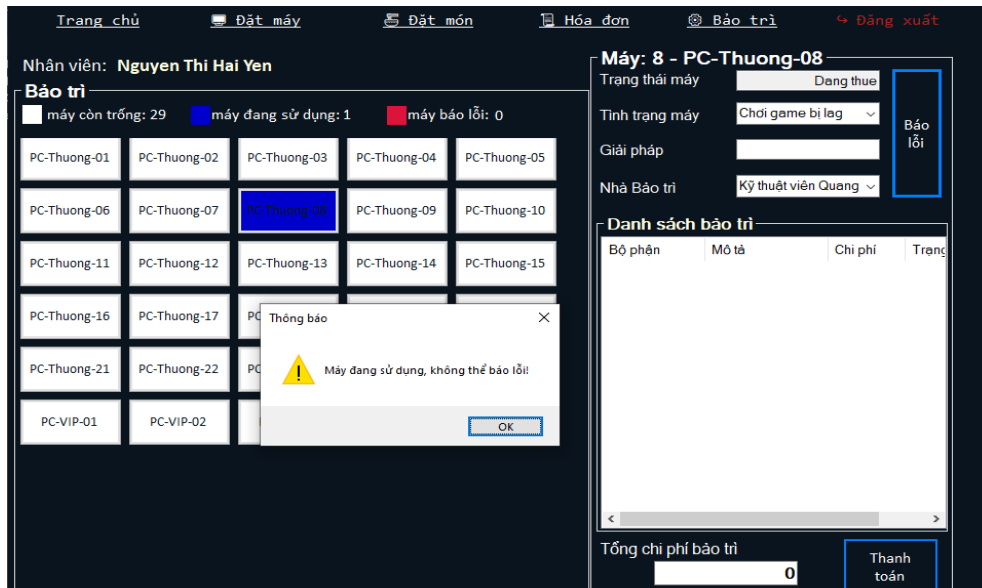
Tổng chi phí bảo trì:

Hình 4.34. Giao diện chọn máy để bảo trì

Chọn máy cần bảo trì: Nhấp vào nút tương ứng với máy gặp sự cố (ví dụ: nút "PC-Thuong-01").

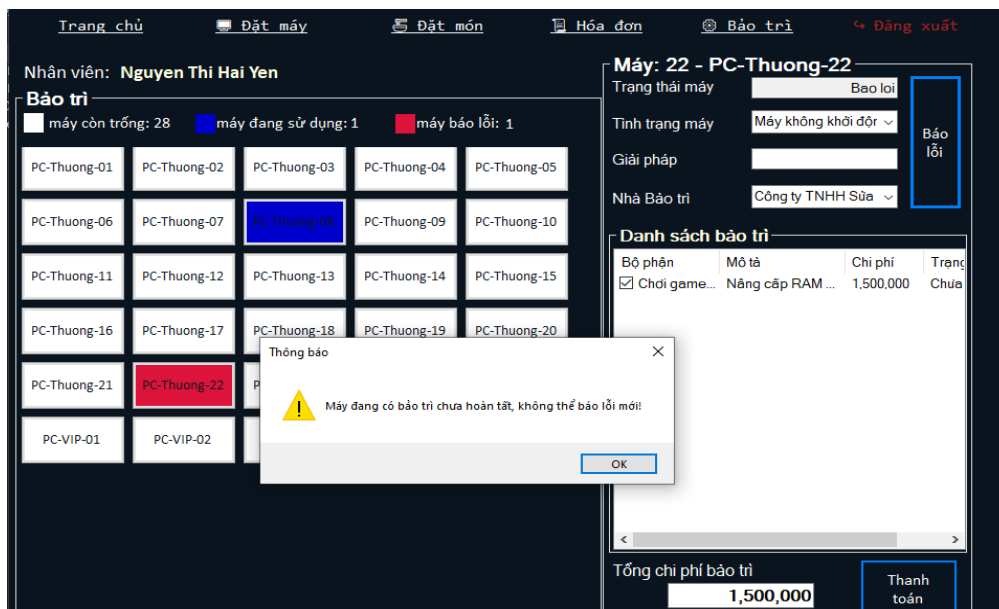
Sau khi chọn, thông tin máy sẽ hiển thị.

Kiểm tra trạng thái máy: Đảm bảo rằng máy không ở trạng thái "Đang sử dụng". Nếu máy đang sử dụng (màu xanh), hệ thống sẽ không cho phép báo lỗi và hiển thị thông báo: "Máy đang sử dụng, không thể báo lỗi!".



Hình 4.35. Thông báo máy đang sử dụng không thể báo lỗi

Nếu máy đã có bảo trì chưa hoàn tất, hệ thống cũng sẽ hiển thị thông báo: "Máy đang có bảo trì chưa hoàn tất, không thể báo lỗi mới!".



Hình 4.36. Giao diện thông báo máy đang bảo trì

Chọn Tình trạng máy, chọn Nhà bảo trì: Trên giao diện, tìm Tình trạng máy. Chọn bộ phận phù hợp với sự cố của máy và chọn một nhà bảo trì để giao nhiệm vụ sửa chữa tại Nhà Bảo trì.

Trang chủ Đặt máy Đặt món Hóa đơn Bảo trì Đăng xuất

Nhân viên: Nguyen Thi Hai Yen

Bảo trì

máy còn trống: 28 máy đang sử dụng: 1 máy báo lỗi: 1

PC-Thuong-01 PC-Thuong-02 PC-Thuong-03 PC-Thuong-04 PC-Thuong-05

PC-Thuong-06 PC-Thuong-07 PC-Thuong-08 PC-Thuong-09 PC-Thuong-10

PC-Thuong-11 PC-Thuong-12 PC-Thuong-13 PC-Thuong-14 PC-Thuong-15

PC-Thuong-16 PC-Thuong-17 PC-Thuong-18 PC-Thuong-19 PC-Thuong-20

PC-Thuong-21 PC-Thuong-22 PC-Thuong-23 PC-Thuong-24 PC-Thuong-25

PC-VIP-01 PC-VIP-02 PC-VIP-03 PC-VIP-04 PC-VIP-05

Máy: 1 - PC-Thuong-01

Trạng thái máy Trong

Tình trạng máy Chơi game bị lag

Giải pháp

Nhà Bảo trì Dịch vụ IT Minh Anh

Báo lỗi

Danh sách bảo trì

Bộ phận	Mô tả	Chi phí	Trạng
☐ Màn hình x...	Giải pháp: Cài lạ...	200,000	Đã h...
☐ Màn hình x...	Cài lại driver car...	200,000	Đã h...
☐ Chơi game...	Nâng cấp RAM ...	1,500,000	Đã h...
☐ Máy không...	Thay ổ cứng mới	1,200,000	Đã h...
☐ Máy không...	Thay ổ cứng mới	1,200,000	Đã h...

Tổng chi phí bảo trì 0

Thanh toán

Hình 4.37. Giao diện chọn nhà bảo trì

Nhấn nút "Báo lỗi": Tìm nút "Báo lỗi" (btnBaoloi) trên giao diện và nhấn vào đó.

Hệ thống sẽ kiểm tra các điều kiện (máy không đang sử dụng, không có bảo trì chưa hoàn tất, đã chọn bộ phận và nhà bảo trì).

Nếu mọi điều kiện đều thỏa mãn:

- Hệ thống lưu thông tin bảo trì vào cơ sở dữ liệu.
- Trạng thái máy được cập nhật thành "Báo lỗi" (nút của máy sẽ chuyển sang màu đỏ trên giao diện).
- Một thông báo sẽ hiện lên: "Báo lỗi thành công!".

Nếu có lỗi (ví dụ: thiếu thông tin), hệ thống sẽ hiển thị thông báo tương ứng (như "Vui lòng chọn bộ phận cần bảo trì!").

Trang chủ Đặt máy Đặt món Hóa đơn Bảo trì Đăng xuất

Nhân viên: **Nguyen Thi Hai Yen**

Bảo trì

☐ máy còn trống: 27 ☒ máy đang sử dụng: 1 ☐ máy báo lỗi: 2

PC-Thuong-01	PC-Thuong-02	PC-Thuong-03	PC-Thuong-04	PC-Thuong-05
PC-Thuong-06	PC-Thuong-07	PC-Thuong-08	PC-Thuong-09	PC-Thuong-10
PC-Thuong-11	PC-Thuong-12	PC-Thuong-13	PC-Thuong-14	PC-Thuong-15
PC-Thuong-16	PC-Thuong-17	PC-Thuong-18	PC-Thuong-19	PC-Thuong-20
PC-Thuong-21	PC-Thuong-22	PC-Thuong-23	PC-Thuong-24	PC-Thuong-25
PC-VIP-01	PC-VIP-02	PC-VIP-03	PC-VIP-04	PC-VIP-05

Máy: 1 - PC-Thuong-01

Trạng thái máy:

Tình trạng máy:

Giải pháp:

Nhà Bảo trì:

Danh sách bảo trì

Bộ phận	Mô tả	Chi phí	Trạng
☐ Màn hình x...	Giải pháp: Cài lạ...	200,000	Đã hç
☐ Màn hình x...	Cài lại driver car...	200,000	Đã hç
☐ Chơi game...	Nâng cấp RAM ...	1,500,000	Đã hç
☐ Máy không...	Thay ổ cứng mới	1,200,000	Đã hç
☐ Máy không...	Thay ổ cứng mới	1,200,000	Đã hç
☒ Chơi game...	Nâng cấp RAM ...	1,500,000	Chưa

Tổng chi phí bảo trì: **1,500,000**

Thanh toán

Hình 4.38. Giao diện bảo lỗi thành công

- Thanh toán và hoàn tất bảo trì

Sau khi nhà bảo trì hoàn thành việc sửa chữa, nhân viên cần thanh toán chi phí bảo trì và cập nhật trạng thái máy. Dưới đây là các bước thực hiện trên giao diện:

Chọn máy đã hoàn tất bảo trì: Từ danh sách máy trên giao diện (flpCom), nhấp vào nút của máy vừa được sửa chữa

Thông tin máy sẽ hiển thị lại trong khu vực "Thông tin máy" (gbMay).

Trang chủ Đặt máy Đặt món Hóa đơn Bảo trì Đăng xuất

Nhân viên: **Nguyen Thi Hai Yen**

Bảo trì

☐ máy còn trống: 27 ☒ máy đang sử dụng: 1 ☐ máy báo lỗi: 2

PC-Thuong-01	PC-Thuong-02	PC-Thuong-03	PC-Thuong-04	PC-Thuong-05
PC-Thuong-06	PC-Thuong-07	PC-Thuong-08	PC-Thuong-09	PC-Thuong-10
PC-Thuong-11	PC-Thuong-12	PC-Thuong-13	PC-Thuong-14	PC-Thuong-15
PC-Thuong-16	PC-Thuong-17	PC-Thuong-18	PC-Thuong-19	PC-Thuong-20
PC-Thuong-21	PC-Thuong-22	PC-Thuong-23	PC-Thuong-24	PC-Thuong-25
PC-VIP-01	PC-VIP-02	PC-VIP-03	PC-VIP-04	PC-VIP-05

Máy: 1 - PC-Thuong-01

Trạng thái máy:

Tình trạng máy:

Giải pháp:

Nhà Bảo trì:

Danh sách bảo trì

Bộ phận	Mô tả	Chi phí	Trạng
☐ Màn hình x...	Giải pháp: Cài lạ...	200,000	Đã hç
☐ Màn hình x...	Cài lại driver car...	200,000	Đã hç
☐ Chơi game...	Nâng cấp RAM ...	1,500,000	Đã hç
☐ Máy không...	Thay ổ cứng mới	1,200,000	Đã hç
☐ Máy không...	Thay ổ cứng mới	1,200,000	Đã hç
☒ Chơi game...	Nâng cấp RAM ...	1,500,000	Chưa

Tổng chi phí bảo trì: **1,500,000**

Thanh toán

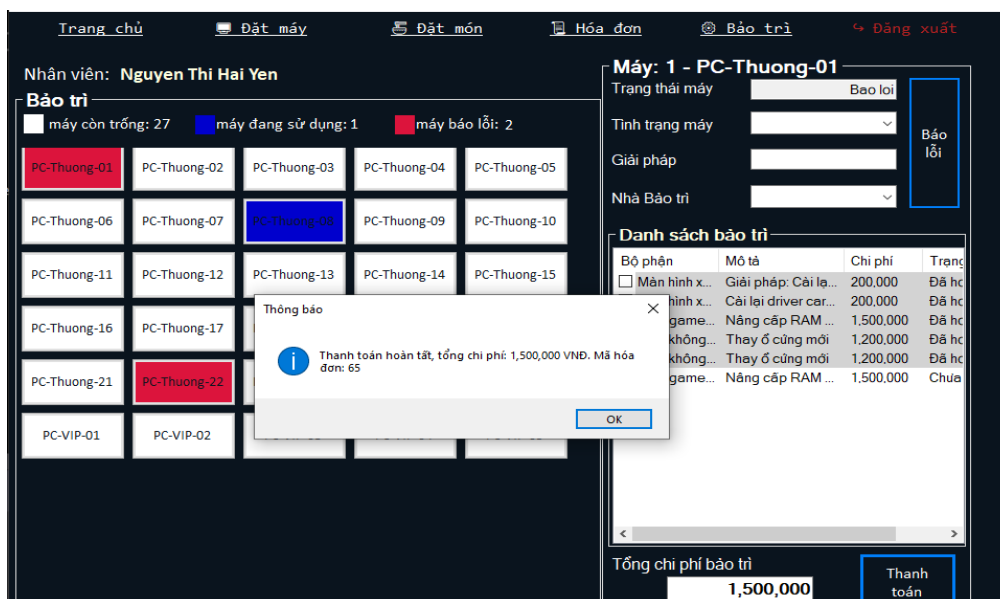
Hình 4.39. Giao diện hiển thị thông tin bảo trì

Nhấn nút "Thanh toán": Tìm nút "Thanh toán" (btnThanhtoan) trên giao diện và nhấn vào đó.

Hệ thống sẽ kiểm tra xem có ít nhất một lỗi được check hay không. Nếu không, thông báo sẽ hiện lên: "Vui lòng chọn ít nhất một lỗi để thanh toán!".

Nếu có lỗi được check:

- Các bản ghi bảo trì được chọn sẽ được cập nhật trạng thái thành "Đã hoàn tất".
- Một thông báo sẽ hiện lên, ví dụ: "Thanh toán hoàn tất, tổng chi phí: 1.000.000 VNĐ. Mã hóa đơn: 1234".



Hình 4.40. Giao diện thanh toán thành công

Trang chủ Đặt máy Đặt món Hóa đơn Bảo trì Đăng xuất

Nhân viên: **Nguyễn Thị Hai Yen**

Bảo trì

☐ máy còn trống: 28 ☒ máy đang sử dụng: 1 ☐ máy báo lỗi: 1

PC-Thuong-01	PC-Thuong-02	PC-Thuong-03	PC-Thuong-04	PC-Thuong-05
PC-Thuong-06	PC-Thuong-07	PC-Thuong-08	PC-Thuong-09	PC-Thuong-10
PC-Thuong-11	PC-Thuong-12	PC-Thuong-13	PC-Thuong-14	PC-Thuong-15
PC-Thuong-16	PC-Thuong-17	PC-Thuong-18	PC-Thuong-19	PC-Thuong-20
PC-Thuong-21	PC-Thuong-22	PC-Thuong-23	PC-Thuong-24	PC-Thuong-25
PC-VIP-01	PC-VIP-02	PC-VIP-03	PC-VIP-04	PC-VIP-05

Máy: 1 - PC-Thuong-01

Trạng thái máy:

Tình trạng máy:

Giải pháp:

Nhà Bảo trì:

Danh sách bảo trì

Bộ phận	Mô tả	Chi phí	Trạng
☐ Màn hình x...	Giải pháp: Cài la...	200,000	Đã hç
☐ Màn hình x...	Cài lại driver car...	200,000	Đã hç
☐ Chơi game...	Nâng cấp RAM ...	1,500,000	Đã hç
☐ Máy không...	Thay ổ cứng mới	1,200,000	Đã hç
☐ Máy không...	Thay ổ cứng mới	1,200,000	Đã hç
☐ Chơi game...	Nâng cấp RAM ...	1,500,000	Đã hç

Tổng chi phí bảo trì:

Hình 4.41. Máy trở lại trạng thái Trống (Trong) sau khi đã thanh toán thành công.

4.4.3 Code trên tầng GUI

- Hiển thị danh sách máy và thông tin bảo trì của máy

```
private void LoadComputerStatus()
{
    try
    {
        DataTable dt = DataProvider.Instance.ExecuteNonQuery(
            "SELECT Trangthai, COUNT(*) as SoLuong FROM dbo.Maytinh
GROUP BY Trangthai",
            new (string, object)[] { }, // Truyền mảng tham số rỗng
            CommandType.Text
        );
        int available = 0, usingCount = 0, errorCount = 0;
        foreach (DataRow row in dt.Rows)
        {
            string status = row["Trangthai"].ToString();
            int count = Convert.ToInt32(row["SoLuong"]);
            if (status == "Trong") available = count;
            else if (status == "Dang thue") usingCount = count;
            else if (status == "Bao loi") errorCount = count;
        }
        txtAvailable.Text = available.ToString();
    }
}
```

```

        txtUsing.Text = usingCount.ToString();
        txtError.Text = errorCount.ToString();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Lỗi khi tải trạng thái máy: {ex.Message}",
            "Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

Hàm LoadComputerStatus() truy vấn cơ sở dữ liệu để lấy số lượng máy theo từng trạng thái ("Trong", "Dang thue", "Bao loi") bằng câu lệnh SQL GROUP BY Trangthai, sau đó cập nhật các ô txtAvailable, txtUsing, và txtError tương ứng. Đoạn code này sử dụng DataProvider.Instance.ExecuteQuery() để thực hiện truy vấn và xử lý ngoại lệ để hiển thị thông báo lỗi nếu có vấn đề, giúp người dùng dễ dàng theo dõi số lượng máy theo từng trạng thái trên giao diện.

```

private void LoadComputers()
{
    flpCom.Controls.Clear();
    try
    {
        List<MaytinhDT00> computers = MaytinhDAL.GetAllComputers();
        foreach (var computer in computers)
        {
            Button btn = new Button
            {
                Text = computer.TenMay,
                Tag = computer.MaMay,
                Size = new System.Drawing.Size(100, 50),
                Font = new System.Drawing.Font("Calibri", 10)
            };

            if (computer.Trangthai == "Trong")
                btn.BackColor = System.Drawing.Color.White;
            else if (computer.Trangthai == "Dang thue")
                btn.BackColor = System.Drawing.Color.MediumBlue;
            else if (computer.Trangthai == "Bao loi")
                btn.BackColor = System.Drawing.Color.Crimson;
            else

```

```

        btn.BackColor = System.Drawing.Color.Gray;

        btn.Click += (s, e) =>
        {
            selectedMaMay = (int)btn.Tag;
            UpdateMachineInfo(computer.TenMay, computer.Trangthai);
            LoadMaintainListForSelectedMachine();
            UpdateTotalCost();
            cbBaotri.SelectedIndex = -1;
            txtDetail.Clear();
        };

        flpCom.Controls.Add(btn);
    }
}
catch (Exception ex)
{
    MessageBox.Show($"Lỗi khi tải danh sách máy: {ex.Message}",
"Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}

```

Hàm LoadComputers() hiển thị tất cả máy trong cửa hàng dưới dạng các nút bấm trong flpCom. Nó lấy danh sách máy từ cơ sở dữ liệu thông qua MaytinhDAL.GetAllComputers(), trả về một List. Với mỗi máy, hàm tạo một Button với Text là tên máy (TenMay), Tag là mã máy (MaMay), và màu nền dựa trên trạng thái (Trangthai): trắng cho "Trống", xanh cho "Đang thuê", đỏ cho "Báo lỗi", và xám cho trạng thái không xác định. Sự kiện click của mỗi nút được gán để cập nhật selectedMaMay, gọi UpdateMachineInfo(), LoadMaintainListForSelectedMachine(), UpdateTotalCost(), và reset cbBaotri cùng txtDetail.

```

private void LoadMaintainListForSelectedMachine()
{
    lvMaintain.Items.Clear();
    if (selectedMaMay == -1) return;

    try
    {

```

```

        DataTable dt = DataProvider.Instance.ExecuteQuery(
            "SELECT b.MaBT, b.GhiChu, b.TrangThai AS TrangThaiBaotri,
t.TenTT, g.ChiPhi " +
            "FROM dbo.Baotri b " +
            "JOIN dbo.ChitietBT c ON b.MaBT = c.MaBT " +
            "JOIN dbo.Tinhtrang t ON c.MaTT = t.MaTT " +
            "JOIN dbo.Giaiphap g ON c.MaGP = g.MaGP " +
            "WHERE c.MaMay = @MaMay",
            new (string, object)[] { ("@MaMay", selectedMaMay) });

        foreach (DataRow row in dt.Rows)
        {
            int maBT = Convert.ToInt32(row["MaBT"]);
            string tenTT = row["TenTT"].ToString();
            string ghiChu = row["GhiChu"].ToString();
            decimal chiPhi = row["ChiPhi"] != DBNull.Value ?
Convert.ToDecimal(row["ChiPhi"]) : 0;
            string trangThai = row["TrangThaiBaotri"].ToString();

            ListViewItem item = new ListViewItem(tenTT); // Cột "Bộ
phận"

            item.SubItems.Add(ghiChu); // Cột "Mô tả"
            item.SubItems.Add(chiPhi.ToString("N0")); // Cột "Chi phí"
            item.SubItems.Add(trangThai); // Cột "Trạng thái bảo trì"
            item.Tag = maBT;

            if (trangThai == "Đã hoàn tất")
            {
                item.Checked = false;
                item.BackColor = System.Drawing.Color.LightGray;
            }
            else
            {
                item.Checked = true;
            }

            lvMaintain.Items.Add(item);
        }
    }
    catch (Exception ex)
    {

```

```

        MessageBox.Show($"Lỗi khi tải danh sách bảo trì: {ex.Message}",
"Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    UpdateTotalCost();
}

```

Phương thức LoadMaintainListForSelectedMachine() hiển thị danh sách các bản ghi bảo trì của máy được chọn (qua truy vấn SQL với tham số MaMay) trong lvMaintain. Nó tạo các cột "Bộ phận", "Mô tả", "Chi phí", và "Trạng thái bảo trì", gán giá trị từ dữ liệu, và đánh dấu các bản ghi "Đã hoàn tất" bằng màu xám với checkbox tắt. Đoạn này cho phép người dùng xem chi tiết các lỗi bảo trì của máy và chuẩn bị cho quá trình thanh toán.

- Kết thúc bảo trì và lưu lại hóa đơn thanh toán

Khi máy đã được sửa chữa xong, người dùng cần thanh toán chi phí bảo trì và cập nhật trạng thái máy về trạng thái bình thường.

```

private void btnThanhtoan_Click(object sender, EventArgs e)
{
    List<int> maBTList = new List<int>();
    foreach (ListViewItem item in lvMaintain.CheckedItems)
    {
        if (item.SubItems[3].Text != "Đã hoàn tất")
        {
            maBTList.Add((int)item.Tag);
        }
    }

    if (maBTList.Count == 0)
    {
        MessageBox.Show("Vui lòng chọn ít nhất một lỗi để thanh toán!",
"Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    try
    {
        BaotriDAL.ThanhToanBaotri(currentMaNV, maBTList, out int maHD);
    }
}

```



```

        decimal totalCost = 0;
        foreach (int maBT in maBTList)
        {
            DataTable dt = DataProvider.Instance.ExecuteQuery(
                "SELECT g.ChiPhi FROM ChitietBT c JOIN Giaiphap g ON
c.MaGP = g.MaGP WHERE c.MaBT = @MaBT",
                new (string, object)[] { ("@MaBT", maBT) });
            if (dt.Rows.Count > 0)
            {
                totalCost += Convert.ToDecimal(dt.Rows[0]["ChiPhi"]);
            }
        }

        MessageBox.Show($"Thanh toán hoàn tất, tổng chi phí:
{totalCost.ToString("N0")} VNĐ. Mã hóa đơn: {maHD}", "Thông báo",
        MessageBoxButtons.OK, MessageBoxIcon.Information);

        LoadComputerStatus();
        LoadComputers();
        LoadMaintainListForSelectedMachine();
        selectedMaMay = -1;
        gbMay.Text = "Thông tin máy";
        txtTT.Text = "";
        txtDetail.Clear();
        cbBaotri.SelectedIndex = -1;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Lỗi khi thanh toán: {ex.Message}", "Lỗi",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

Thanh toán và hoàn tất bảo trì: Người dùng chọn một máy từ danh sách trên giao diện, sau đó xem danh sách bảo trì trong lvMaintain. Các lỗi chưa thanh toán sẽ được tự động check sẵn. Người dùng kiểm tra tổng chi phí hiển thị trong txtTotal (được tính tự động bởi UpdateTotalCost()). Nếu mọi thông tin đã đúng, người dùng nhấn nút "Thanh toán" (btnThanhtoan_Click).

Hệ thống kiểm tra các điều kiện:

- Nếu máy không ở trạng thái "Báo lỗi" (txtTT.Text là "Trống" hoặc "Đang thuê"), thông báo "Máy này không bảo trì!" sẽ hiển thị.
- Nếu chưa chọn máy (selectedMaMay = -1), thông báo "Chưa chọn máy!" sẽ hiển thị.
- Nếu không có lỗi nào được check trong lvMaintain, thông báo "Vui lòng chọn ít nhất một lỗi để thanh toán!" sẽ hiển thị.

Nếu không có lỗi, hệ thống thực hiện thanh toán:

- Lấy danh sách mã bảo trì (maBTList) từ các mục được check trong lvMaintain.
- Gọi BaotriDAL.ThanhToanBaotri() để lưu hóa đơn thanh toán (loại "Chi") và cập nhật trạng thái các bản ghi bảo trì thành "Đã hoàn tất".
- Tính lại tổng chi phí từ cơ sở dữ liệu và hiển thị thông báo: "Thanh toán hoàn tất, tổng chi phí: [số tiền] VNĐ. Mã hóa đơn: [maHD]".
- Trạng thái máy được cập nhật: nếu không còn lỗi chưa hoàn tất, máy trở về trạng thái "Trống"; nếu còn lỗi, máy giữ trạng thái "Báo lỗi".

Giao diện được làm mới: danh sách máy, danh sách bảo trì, và trạng thái tổng quan được cập nhật. Các ô nhập liệu và thông tin máy được reset (selectedMaMay = -1, gbMay.Text = "Thông tin máy", txtTT.Text = "", cbBaotri.SelectedIndex = -1, txtDetail.Clear()).

4.4.4. Code trên DAL

- **Hàm GetMaintainListForMachine(int maMay)**

```
public static List<BaotriDTO> GetMaintainListForMachine(int maMay)
{
    List<BaotriDTO> maintainList = new List<BaotriDTO>();
    using (SqlConnection conn = DataProvider.Instance.GetConnection())
    {
        conn.Open();
```

```

        using (SqlCommand cmd = new
SqlCommand("sp_GetMaintainListForMachine", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.AddWithValue("@MaMay", maMay);
            using (SqlDataReader reader = cmd.ExecuteReader())
            {
                while (reader.Read())
                {
                    maintainList.Add(new BaotriDTO
                    {
                        MaBT = Convert.ToInt32(reader["MaBT"]),
                        GhiChu = reader["GhiChu"].ToString(),
                        TrangThai =
reader["TrangThaiBaotri"].ToString(),
                        MaHD = Convert.ToInt32(reader["MaHD"])
                    });
                }
            }
        }
    }
    return maintainList;
}

```

Hàm GetMaintainListForMachine() truy xuất danh sách các bản ghi bảo trì của một máy tính cụ thể từ cơ sở dữ liệu dựa trên mã máy (maMay). Nó sử dụng stored procedure sp_GetMaintainListForMachine để thực hiện truy vấn, truyền tham số @MaMay và trả về một danh sách các đối tượng BaotriDTO. Mỗi BaotriDTO chứa các thông tin như mã bảo trì (MaBT), ghi chú (GhiChu), trạng thái bảo trì (TrangThai), và mã hóa đơn (MaHD). Hàm này hỗ trợ việc hiển thị chi tiết bảo trì của máy trên giao diện, cung cấp dữ liệu cần thiết để quản lý các lỗi bảo trì.

- **Hàm CreateBaotri(int maNV, int maMay, int maTT, int maNN, int maGP, string ghiChu, decimal chiPhi, int maNBT)**

```

public static int CreateBaotri(int maNV, int maMay, int maTT, int maNN, int
maGP, string ghiChu, decimal chiPhi, int maNBT)
{
    Dictionary<string, object> outputParams;

```

```

DataProvider.Instance.ExecuteStoredProcedure(
    "sp_CreateBaotri",
    out outputParams,
    ("@MaNV", maNV),
    ("@MaMay", maMay),
    ("@MaTT", maTT),
    ("@MaNN", maNN),
    ("@MaGP", maGP),
    ("@GhiChu", ghiChu ?? (object)DBNull.Value),
    ("@ChiPhi", chiPhi),
    ("@MaNBT", maNBT),
    ("@MaHD", null)
);
return Convert.ToInt32(outputParams["@MaBT"]);
}

```

Hàm CreateBaotri() tạo một bản ghi bảo trì mới trong cơ sở dữ liệu bằng cách gọi stored procedure sp_CreateBaotri. Nó nhận các tham số như mã nhân viên (maNV), mã máy (maMay), mã tình trạng (maTT), mã nguyên nhân (maNN), mã giải pháp (maGP), ghi chú (ghiChu), chi phí (chiPhi), và mã nhà bảo trì (maNBT). Hàm truyền các tham số này vào stored procedure và nhận về mã bảo trì (MaBT) thông qua tham số đầu ra @MaBT. Hàm này đảm nhiệm việc lưu thông tin bảo trì mới, bao gồm cả việc liên kết với hóa đơn và cập nhật trạng thái máy trong cơ sở dữ liệu.

4.4.5 Code trên DTO

```

public class BaotriDTO
{
    public int MaBT { get; set; }
    public DateTime NgayBT { get; set; }
    public int MaNBT { get; set; }
    public int MaHD { get; set; }
    public string GhiChu { get; set; }
    public string TrangThai { get; set; }
}

```

Lớp BaotriDTO đóng vai trò quan trọng như một đối tượng truyền dữ liệu, chứa các thuộc tính như MaBT, NgayBT, MaNBT, MaHD, GhiChu, và TrangThai để đại diện

cho một bản ghi bảo trì. Nó hỗ trợ các hoạt động truy xuất, lưu trữ, và hiển thị dữ liệu bảo trì trong hệ thống, đảm bảo tính nhất quán và dễ dàng tích hợp giữa các tầng.

KẾT LUẬN

Đối với bài tập lớn môn .NET, nhóm chúng em đã mô phỏng một hệ thống quản lý hoàn chỉnh với các chức năng cơ bản, áp dụng kiến thức đã học cũng như nghiên cứu tham khảo những kiến thức bên ngoài. Hệ thống này giúp quản lý thời gian chơi của khách, quản lý tài chính và các dịch vụ khác một cách hiệu quả. Chúng em đã dùng Visual Studio Code và SQL Server để phát triển ứng dụng này. Ứng dụng có giao diện dễ dùng, giúp người quản lý dễ dàng theo dõi và điều hành quán với các chức năng như đăng nhập, tính tiền, quản lý máy và đồ ăn hoạt động ổn định.

Dự án cũng giúp chúng em nâng cao kỹ năng lập trình, củng cố kiến thức và xây dựng được kiến thức mới. Trong quá trình làm dự án, chúng em gặp nhiều khó khăn và khúc mắc, đây cũng là dịp để nhóm chúng em có thêm kinh nghiệm giải quyết vấn đề kỹ thuật cũng như làm việc nhóm.

Nhìn chung, bài tập lớn này rất hữu ích, giúp chúng em học được nhiều kiến thức và kỹ năng thực tế. Chúng em tin rằng những gì học được từ dự án này sẽ là nền tảng tốt để chúng em phát triển thêm các ứng dụng phức tạp hơn sau này.

Hy vọng ứng dụng quản lý quán net này sẽ là một ý tưởng để các cửa hàng có thể áp dụng và mong rằng có thể phát triển trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] *IBM*. (2025). Được truy lục từ What is three-tier architecture?:
<https://www.ibm.com/think/topics/three-tier-architecture>
- [2] Microsoft. (2025). *Microsoft Learn*. Được truy lục từ N-tier architecture style:
<https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>
- [3] TS. Bùi Thị Hồng Nhung - Khoa CNTT&KTS - Học viện Ngân Hàng. (2024).
Giáo trình C#. Hà Nội.